

# **Data Science I with python**

**STAT 303-1-Sec20**

Lizhen Shi

## **Table of contents**

# Preface

Welcome to **STAT 303-1 (Data Science with Python I), Section 20** at Northwestern University.

This book is designed to support your journey into data science, blending foundational concepts with hands-on practice. It builds on the original work of **Professor Arvind Krishna**, whose materials inspired the structure and spirit of this resource. The content has been thoughtfully updated to meet the evolving needs of students and the goals of the course.

In this first course, you will:

- Learn to use essential Python libraries for data science, including but not limited to **NumPy**, **Pandas**, and **Matplotlib**. These three are the main focus, but you will also be introduced to other useful libraries and tools as needed for real-world data analysis.
- Develop skills in **Exploratory Data Analysis (EDA)**, transforming messy, real-world datasets into meaningful insights through visualization, summarization, and pattern discovery

The book is organized to guide you step-by-step:

- **Chapters 1–3:** Set up your coding environment in VS Code, understand Python environments, and enhance your workflow
- **Chapters 4–5:** Review key concepts from STAT 201. If you need a refresher, visit the [STAT 201 ebook](#).
- New material begins in **Chapter 6 – *Reading Data***

Throughout the quarter, this resource will be updated and refined to improve clarity, depth, and alignment with our teaching objectives.

As a **living document**, your feedback and suggestions are always welcome. Contributions from students, instructors, and the broader academic community help make this book stronger and more effective.

If you have ideas for improving the textbook, please share your feedback or suggestions using our dedicated [Textbook Improvement Form](#). Your input is invaluable in making this resource better for everyone.

Thank you for joining us on this learning adventure. We hope this book becomes a valuable companion as you build your skills and confidence in data science with Python.

# **Part I**

## **Getting started**

# 1 Setting up your environment with VS Code

<IPython.core.display.Image object>

## 1.1 Learning Objectives

Setting up your Python data science environment is the foundation for all your work in this course. A well-configured setup will save you time, prevent errors, and make your workflow smoother. Whether you're new to VS Code or Python, these steps will help you build a reliable environment for coding, analysis, and reproducibility.

By the end of this chapter, you will be able to:

- Install and configure **Visual Studio Code (VS Code)** for Python and Jupyter Notebooks.
- Create and manage a **virtual environment** for this course.
- Install essential **Python libraries** for data science.
- Verify that your environment is working properly.

## 1.2 Why Choose Visual Studio Code (VS Code)?

Choosing the right editor can make a big difference in your productivity and learning experience. VS Code is a top choice for data science and Python development because it combines power, flexibility, and ease of use.

- **Free and Lightweight**  
VS Code is free, open-source, and runs efficiently on most systems without being resource-heavy.
- **Cross-Platform**  
Works consistently on Windows, macOS, and Linux.

- **Rich Python Support**

With the official Python extension, you get:

- Syntax highlighting and IntelliSense (auto-completion, code hints).
- Built-in debugging tools.
- Easy integration with virtual environments and Jupyter Notebooks.

- **Integrated Jupyter Notebook Support**

You can open and run `.ipynb` notebooks directly in VS Code.

- **Customizable and Extensible**

Large marketplace of extensions (e.g., formatting, linting, Git, Docker, AI tools).

- **Version Control Integration**

Built-in Git and GitHub support for managing and sharing code.

- **Unified Workflow**

One place for editing, running, debugging, documenting, and version-controlling Python code.

Whether you're just starting out or working on advanced projects, VS Code provides a modern, unified environment for all your coding needs.

## 1.3 Quick Start: Setting Up Python & VS Code

VS Code is a versatile editor. To use it for data science, you'll complete a few extra setup steps (e.g., Python & Jupyter extensions). Follow this step-by-step setup guide to get started quickly and avoid common pitfalls.

### Step 1: Install Python

- Download and install the latest version from [python.org](https://python.org).
- On Windows, check the box “**Add Python to PATH.**” This makes Python available in your terminal.

### Step 2: Install VS Code & Extensions

- Download VS Code from [code.visualstudio.com](https://code.visualstudio.com).
- Open VS Code, go to the Extensions panel (**Ctrl+Shift+X**), and install:
  - **Python (Microsoft)**
  - **Jupyter (Microsoft)**

### Step 3: Create a Course Folder

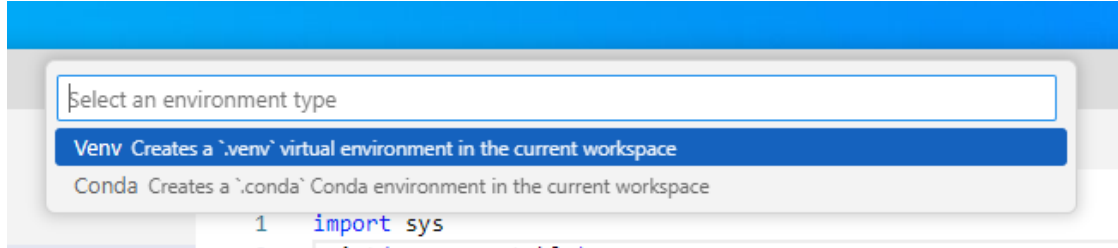
- Make a folder for your notebooks (e.g., `datasci` on Desktop or Documents).
- Open this folder in VS Code.

#### Step 4: Create Your First Notebook

- In VS Code, go to **File** → **New File** → **Save as** → `test.ipynb`.
- Add a code cell:

```
print("hello world")
```

- Run the cell. VS Code may prompt you to select or create a Python environment.



- If prompted, create a new environment using the GUI (recommended for beginners).
- Make sure the kernel (top right) shows **.venv (Python 3.xx.x)**.
- If you see `hello world` printed, your setup is working!

**If everything works, you can skip the Full Setup below.**

If you run into issues, continue with the detailed setup and troubleshooting steps in the next section.

This quick start helps you get coding fast, but the full setup ensures your environment is robust and reproducible for all course projects.

## 1.4 Full Setup: Reliable Python Environment in VS Code

If the quick start didn't work, or you want a more robust setup, follow these step-by-step instructions to ensure your Python environment is reproducible and ready for data science.

### Step 1: Install Python

- Go to the [official Python website](https://www.python.org/) and download the latest version for your operating system.
- **Important:** During installation, check the box “**Add Python to PATH**” so Python is available in your terminal.
- After installation, restart your terminal in VS Code.

- Verify installation by running:

```
python --version
```

You should see the installed Python version.

## Step 2: Install VS Code Extensions

- Open VS Code.
- Go to Extensions (Ctrl+Shift+X).
- Install:
  - **Python** (by Microsoft)
  - **Jupyter** (by Microsoft)

## Step 3: Set Up Your Workspace

- Create a new folder for your course work (e.g., STAT303-1 on Desktop or Documents).
- Open the folder in VS Code (File > Open Folder).

## Step 4: Create a Notebook for Your Work

- In VS Code, go to File > New File and select Jupyter Notebook.
- Save the file as `your_notebook.ipynb`.
- Add a code cell:

```
print("hello world")
```

- Make sure to select the `.venv` kernel for your notebook (see next step).

## Step 5A: Create a Virtual Environment (GUI Method)

- Run the cell. If prompted, create a Python environment using `venv` in the current workspace.
- Wait for Jupyter to start and install necessary components (e.g., ipykernel).
- After installation, ensure the selected kernel in the top right of the notebook says **.venv (Python 3.xx.x)**.
- If not, select the `.venv` kernel manually.

*If you successfully created the environment using the GUI, skip Step 5B.*

## Step 5B: Create a Virtual Environment (Command Line Method)

- If not prompted to create your environment, use the terminal:
  - Open the terminal in VS Code.



- Run:

```
python -m venv .venv
```

- Activate the environment:

- \* On Windows:

```
.\.venv\Scripts\activate
```

- \* On macOS/Linux:

```
source .venv/bin/activate
```

- You should see (.venv) at the start of your terminal prompt.
- Install essential packages:

```
pip install numpy pandas matplotlib
```

## Step 6: Verify Your Setup

- Run the code cell again. You should see `hello world` printed out.
- In the terminal, check your Python version and installed packages:

```
python --version  
pip list
```

## Troubleshooting Tips

- If the kernel does not show `.venv`, restart VS Code and ensure the environment is activated.
- Make sure the Python and Jupyter extensions are installed and enabled.
- If you see errors, check that your notebook is saved with the `.ipynb` extension and the correct kernel is selected.

A reliable environment is essential for reproducible data science. These steps will help you avoid common issues and set yourself up for success throughout the course.

## 1.5 Jupyter Notebooks in VS Code

After setting up your environment, you can take full advantage of Jupyter Notebooks directly in VS Code. This integration lets you:

- Create, edit, and run `.ipynb` notebooks without leaving VS Code
- Use interactive code cells for data analysis, visualization, and documentation
- Access rich outputs (plots, tables, markdown) inline with your code

- Switch kernels to use different Python environments for each notebook
- Debug, refactor, and version-control your notebooks with built-in tools

### Getting Started:

- Open or create a notebook file (.ipynb) in VS Code
- Make sure the correct Python kernel/environment is selected (top right corner)
- Add code and markdown cells as needed
- Run cells individually or all at once

### Learn More:

- Explore the official guide: [Jupyter Notebooks in VS Code](#)

Jupyter Notebooks in VS Code provide a powerful, flexible workflow for data science and Python development.

## 1.6 Rendering notebook as HTML using Quarto

Quarto is designed for high-quality, customizable, and publishable outputs, making it suitable for reports, blogs, or presentations. We'll use Quarto to print the \*.ipynb\* file as HTML for your assignment submission.

### 1.6.1 Installing and Verifying Quarto

- Download and install Quarto from the [official website](#).
- Follow the installation instructions for your operating system.
- Open your terminal within VS Code:
  - Go to **Terminal** -> **New Terminal**.
- Run the following command to verify that Quarto and its dependencies are correctly installed: `quarto --version`
- You should see the installed quarto version. If the command line doesn't work, you might see an error message like:
  - ``quarto is not recognized``
  - ``quarto command is not found``

This issue is often caused by Quarto not being added to the PATH environment variable. Similar to how you added the Python path to the environment variable above, you need to add the Quarto path to the system environment variable so that the command can be recognized by your operating system's shell.

On Windows, if you used the default installation path (without changing it), Quarto is installed in: `C:\Users\<USER>\AppData\Local\Programs\Quarto\bin`

**Note:**

Before moving forward, ensure that the command `quarto --version` successfully prints the version of your installed quarto. If it does not, you may need to troubleshoot your quarto installation or add it to the PATH environment variable.

## 1.6.2 Converting the Notebook to HTML

Check the procedure for rendering a notebook as HTML [here](#). You have several options to format the file. Here are some points to remember when using Quarto to render your notebook as HTML:

1. The `Raw NBConvert` cell type is used to render different code formats into HTML or LaTeX. This information is stored in the notebook metadata and converted appropriately. **Use this cell type to put the desired formatting settings for the HTML file.**
2. In the formatting settings, remember to use the setting `embed-resources: true`. This will ensure that the rendered HTML file is self-contained, and is not dependent on other files. This is especially important when you are sending the HTML file to someone, or uploading it somewhere. If the file is self-contained, then you can send the file by itself without having to attach the dependent files with it.

Once you have entered the desired formatting setting in the `Raw NBConvert` cell, you are ready to render the notebook to HTML. Open the terminal, navigate to the directory containing the notebook (*.ipynb file*), and use the command: `quarto render filename.ipynb --to html`.

## 1.7 Independent Study

**Goal:** Get hands-on experience creating a reproducible Python environment using `pip` and `.venv` in VS Code.

### 1.7.1 Step-by-Step Instructions

#### Step 1: Create Your Workspace

- Make a folder named STAT303\_1 for your course work.
- Open this folder in VS Code.
- Create a new notebook: `setup_test.ipynb`.
- Add a code cell:

```
print("Hello, VS Code & .venv!")
```

- Run the cell. If prompted, create/select a Python environment (choose `.venv`).

#### Step 2: Create the `.venv` Environment

- If VS Code does not prompt you, open the terminal in VS Code.
- Run:

```
python -m venv .venv
```

- Activate the environment:

```
.\.venv\Scripts\activate
```

- Install essential packages:

```
pip install numpy pandas matplotlib
```

#### Step 3: Verify Environment Consistency

- In the terminal, check your Python version:

```
python --version
```

- In the notebook, run:

```
import sys
print(sys.executable)
```

- Make sure the Python path in the notebook matches the one in your terminal. This confirms you are using the same environment.

#### Step 4: Generate html from the notebook using Quarto

- In the terminal, make sure you are in the directory that contains your notebook

```
quarto render setup_test.ipynb --to html
```

- Make sure the html file was generated in the same directory

## Troubleshooting Tips

If you run into issues (e.g., kernel not showing `.venv`, code cell errors):

- Restart VS Code.
- Deactivate/reactivate `.venv`.
- Make sure **Python** & **Jupyter** extensions are installed.
- Save your notebook with the `.ipynb` extension.
- If the kernel is missing, use the kernel picker (top right) to select `.venv`.
- If you see an error like *“No such file or directory”* or *“No valid input files passed to render”*:
  1. Check your current location:
    - macOS / Linux: `pwd`
    - Windows: `cd`
  2. List files in the current directory:
    - macOS / Linux: `ls`
    - Windows: `dir`
  3. If the notebook isn't there:
    - Use `cd` to move into the correct folder, or
    - Provide the full (absolute) path to the file.

## Extra:

- Try installing a package (e.g., `seaborn`) and import it in a new cell to test.
- Practice switching kernels to see how environments affect your code.
- Practice creating a `.conda` env and use `conda` to install the `seaborn` package

## 1.8 Reference

- [Getting Started with VS Code](#)
- [Jupyter Notebooks in VS Code](#)
- [Install Python Packages](#)
- [Managing environments with conda](#)

## 2 Python Environments and Package Management

<IPython.core.display.Image object>

### 2.1 Learning Objectives

Managing Python environments and packages is essential for reproducible, error-free data science. Understanding these concepts will help you avoid common pitfalls and collaborate more effectively. This chapter will guide you through the tools and best practices for setting up, sharing, and troubleshooting Python environments.

By the end of this chapter , you will be able to:

- Explain why virtual environments are important.
- Install essential data science packages in your Python environment.
- Create a `requirements.txt` file and use it to quickly recreate an environment.
- Recognize why dependency conflicts happen.
- Describe how tools like `conda` or `poetry` can help resolve dependency conflicts.

### 2.2 Data Science packages in Python

Python has a rich ecosystem of packages that make data analysis easier and more powerful. In this course, you will install and use several core packages:

- **NumPy** → numerical computing and array operations
- **pandas** → data manipulation and analysis

- **Matplotlib** → data visualization
- **Seaborn** → statistical data visualization (built on top of Matplotlib)

These packages give you the essential tools to load, clean, analyze, and visualize data.

Before using them, you need to install them in your Python environment.

In your previous VS Code setup lesson, you created a Python environment using `.venv` and selected it as the kernel to run your notebook. This created a folder called `.venv`, which contains:

- A dedicated Python interpreter for your project
- A `Lib` folder where all packages you install are stored

Any packages you install (e.g., with `pip install ...`) will only affect this environment, keeping your project isolated from others.

Using virtual environments is considered best practice for Python development, especially in data science projects. This isolation helps you:

- Avoid version conflicts between packages
- Keep project dependencies separate
- Make your code more reproducible and shareable

Before we dive in, let's clarify what a Python environment is and why isolation matters.

## 2.3 Python Virtual Environments

A Python virtual environment is an **isolated workspace** with its own Python interpreter and its own set of installed packages.

This isolation makes it easy to manage dependencies for different projects and prevents package version conflicts.

**Why use virtual environments?**

- Keep each project's dependencies separate
- Avoid breaking code by accidentally upgrading/downgrading packages
- Share your code with others and ensure it works the same way on their machine

Virtual environments are a key tool for professional, reproducible Python workflows.

## 2.4 Install Data Science Packages Within Your Environment

Python's power for data science comes from its rich ecosystem of packages. These packages provide essential tools for scientific computing, data analysis, visualization, and machine learning.

Packages like `numpy`, `pandas`, and `matplotlib` are not included in the Python standard library—you need to install them in your environment before you can use them.

**Test your setup:** Add the following code to your `test.ipynb` notebook to check if your environment is ready:

```
import numpy as np
import pandas as pd
print("Setup complete!")
```

If you see a `ModuleNotFoundError`, it means the package is not installed in your current environment. Use `pip install` to add missing packages.

**Tip:** Always install packages inside your active virtual environment to keep your project isolated and reproducible.

## 2.4.1 How to Install Packages

There are two main ways to install data science packages in your environment:

### 2.4.1.1 Installing from the Terminal

- **Open a New Terminal:** Check that your terminal is using the same environment as your notebook kernel.
  - If you see `(.venv)` at the start of the prompt, your virtual environment is active and matches the notebook kernel.
  - If you see `(base)`, it means the base conda environment is active (common if you have Anaconda installed).
  - To confirm which Python is being used, run:
    - \* On macOS/Linux: `which python`
    - \* On Windows: `where.exe python`

**Note:** If you have both Anaconda and VS Code installed, environments can sometimes conflict. If the terminal and notebook use different environments, packages may be installed in the wrong place, causing import errors in your notebook.

- **Install packages with `pip`:**

```
pip install numpy pandas
```



### 2.4.1.2 Installing from the Notebook

You can also install packages directly from a Jupyter Notebook cell using a magic command. This is convenient for quick installs without leaving the notebook interface.

- Add a new code cell and run:

```
pip install numpy pandas
```

**Best Practice:** Always make sure you are installing packages into the correct environment. Double-check your kernel and terminal environment before running installation commands.

## 2.4.2 Backing Up and Sharing Your Environment

Reproducibility is a cornerstone of good data science. To ensure your code works for you, your collaborators, and on other machines, you need a way to recreate your Python environment exactly.

Backing up your environment makes it easy to share your work and avoid “it works on my machine” problems.

**How to back up your environment:**

Step 1: **Create a `requirements.txt` file**

- This file lists all installed packages and their versions, so anyone can recreate your environment.

Run the following command in your terminal to list all installed packages and their versions:

```
pip freeze
```

```
asttokens==2.4.1
colorama==0.4.6
comm==0.2.2
contourpy==1.3.0
cyclor==0.12.1
debugpy==1.8.6
decorator==5.1.1
executing==2.1.0
fonttools==4.54.1
ipykernel==6.29.5
ipython==8.27.0
jedi==0.19.1
```

```
jupyter_client==8.6.3
jupyter_core==5.7.2
kiwisolver==1.4.7
matplotlib==3.9.2
matplotlib-inline==0.1.7
nest-asyncio==1.6.0
numpy==2.1.1
packaging==24.1
pandas==2.2.3
parso==0.8.4
pillow==10.4.0
platformdirs==4.3.6
prompt_toolkit==3.0.48
psutil==6.0.0
pure_eval==0.2.3
Pygments==2.18.0
pyparsing==3.1.4
python-dateutil==2.9.0.post0
pytz==2024.2
pywin32==306
pyzmq==26.2.0
six==1.16.0
stack-data==0.6.3
tornado==6.4.1
traitlets==5.14.3
tzdata==2024.2
wcwidth==0.2.13
```

Note: you may need to restart the kernel to use updated packages.

Using the redirection operator `>`, you can save the output of `pip freeze` to a `requirement.txt`. This file can be used to install the same versions of packages in a different environment.

```
pip freeze > requirement.txt
```

Note: you may need to restart the kernel to use updated packages.

Let's check whether the `requirement.txt` is in the current working directory

```
%ls
```

Volume in drive C is Windows  
Volume Serial Number is A80C-7DEC

Directory of c:\Users\lsi8012\OneDrive - Northwestern University\FA24\303-1\test\_env

```
09/27/2024  02:25 PM    <DIR>          .
09/27/2024  02:25 PM    <DIR>          ..
09/27/2024  07:44 AM    <DIR>          .venv
09/27/2024  01:42 PM    <DIR>          images
09/27/2024  02:25 PM                695 requirement.txt
09/27/2024  02:25 PM            21,352 venv_setup.ipynb
                2 File(s)            22,047 bytes
                4 Dir(s)  166,334,562,304 bytes free
```

## Step 2: Share the file

- Send the `requirements.txt` file to your collaborator, or save it for future use.

## Step 3: Recreate the environment elsewhere

- On a new machine or environment, run:

```
pip install -r requirements.txt
```

```
pip install -r requirement.txt
```

```
Requirement already satisfied: asttokens==2.4.1 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: colorama==0.4.6 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: comm==0.2.2 in c:\users\lsi8012\onedrive - northwestern unive
Requirement already satisfied: contourpy==1.3.0 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: cycler==0.12.1 in c:\users\lsi8012\onedrive - northwestern un
Requirement already satisfied: debugpy==1.8.6 in c:\users\lsi8012\onedrive - northwestern un
Requirement already satisfied: decorator==5.1.1 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: executing==2.1.0 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: fonttools==4.54.1 in c:\users\lsi8012\onedrive - northwestern
Requirement already satisfied: ipykernel==6.29.5 in c:\users\lsi8012\onedrive - northwestern
Requirement already satisfied: ipython==8.27.0 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: jedi==0.19.1 in c:\users\lsi8012\onedrive - northwestern unive
Requirement already satisfied: jupyter_client==8.6.3 in c:\users\lsi8012\onedrive - northwes
Requirement already satisfied: jupyter_core==5.7.2 in c:\users\lsi8012\onedrive - northwester
Requirement already satisfied: kiwisolver==1.4.7 in c:\users\lsi8012\onedrive - northwestern
Requirement already satisfied: matplotlib==3.9.2 in c:\users\lsi8012\onedrive - northwestern
Requirement already satisfied: matplotlib-inline==0.1.7 in c:\users\lsi8012\onedrive - north
```

```

Requirement already satisfied: nest-asyncio==1.6.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: numpy==2.1.1 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: packaging==24.1 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pandas==2.2.3 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: parso==0.8.4 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pillow==10.4.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: platformdirs==4.3.6 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: prompt_toolkit==3.0.48 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: psutil==6.0.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pure_eval==0.2.3 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: Pygments==2.18.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pyparsing==3.1.4 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: python-dateutil==2.9.0.post0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pytz==2024.2 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pywin32==306 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pyzmq==26.2.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: six==1.16.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: stack-data==0.6.3 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: tornado==6.4.1 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: traitlets==5.14.3 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: tzdata==2024.2 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: wcwidth==0.2.13 in c:\users\lsi8012\onedrive - northwestern uni
Note: you may need to restart the kernel to use updated packages.

```

Once you run the install command, all packages listed in your `requirements.txt` file will be installed, matching the exact versions you used in your original environment.

This ensures your code runs the same way everywhere—whether on your computer, a collaborator’s machine, or a server.

### Why is this important?

- Guarantees everyone is using the same package versions, reducing errors and “works on my machine” problems.
- Makes it easy to set up your project on a new computer, server, or cloud environment.
- Simplifies troubleshooting and collaboration—everyone starts from the same setup.

### Pro Tip:

- Update your `requirements.txt` after installing or upgrading packages to keep it current.
- Consider version-controlling your `requirements.txt` file (e.g., with Git) so changes are tracked and shared with collaborators.

A well-maintained environment file is essential for reproducible, shareable, and professional data science projects.

## 2.5 Dependency Conflicts

A **dependency conflict** occurs when two or more packages in your Python environment require different or incompatible versions of the same library.

This can cause errors, unexpected behavior, or even break your code.

### 2.5.1 Why do dependency conflicts happen?

- Many Python packages rely on other packages (called *dependencies*) to work.
- If you install packages that depend on different versions of the same dependency, they may not work together.
- This is especially common in data science, where libraries evolve quickly and have complex interdependencies.

**Example:** Suppose you install two packages:

- PackageA requires `numpy==1.24.0`
- PackageB requires `numpy==2.2.0`

If you install both with `pip`, the **last specified version wins**. One of the packages may then fail because it cannot use the version of `numpy` that was actually installed.

### 2.5.2 How to avoid dependency conflicts

- Use **virtual environments** to isolate dependencies for each project.
- Check package requirements before installing new libraries.
- Use tools like **Poetry** or **Conda** to detect and resolve conflicts automatically.

## 2.6 How to Resolve Conflicts and Manage Environments

### 2.6.1 Use conda for Robust Environment Management

`conda` is a powerful tool for managing both Python environments and packages, especially in data science workflows. It helps you avoid and resolve dependency conflicts by handling package versions and system dependencies more effectively than `pip` alone.

## Why use conda?

- Create fully isolated environments for different projects
- Install packages and all their dependencies from the Anaconda repository
- Easily switch between environments for different tasks
- Export and share environment configurations for reproducibility

## How conda prevents dependency conflicts:

- When you install a package, conda automatically checks for compatible versions of all dependencies and installs them together.
- If a conflict is detected, conda will warn you, suggest solutions, or prevent incompatible installations.

## Essential conda commands:

- Create a new environment:

```
conda create --name myenv numpy pandas matplotlib
```

- Activate an environment:

```
conda activate myenv
```

- Install a package in an environment:

```
conda install seaborn
```

- List all environments:

```
conda env list
```

- Export environment configuration:

```
conda env export > environment.yml
```

- Recreate an environment from a file:

```
conda env create -f environment.yml
```

**Example:** Suppose you need to work on two projects that require different versions of `scikit-learn`. You can create two separate environments:

```
conda create --name projectA scikit-learn=0.24  
conda create --name projectB scikit-learn=1.2
```

Each environment will have its own compatible dependencies, so you avoid conflicts.

**Summary:** Using `conda` is highly recommended for managing complex dependencies, system-level packages, and reproducible environments in data science.

## 2.6.2 Use poetry for Modern Python Projects

poetry is a modern tool for managing Python dependencies and packaging. It streamlines environment setup, ensures reproducibility, and makes sharing your project easy.

### Why use poetry?

- Automatically creates and manages virtual environments for each project
- Uses a `pyproject.toml` file to specify and lock project requirements
- Prevents dependency conflicts with a lock file (`poetry.lock`)
- Simplifies publishing packages to PyPI and sharing with others

### How poetry helps with dependency management:

- Resolves and installs compatible versions of all dependencies automatically
- Keeps your environment reproducible and up-to-date
- Makes it easy to add, update, or remove packages

### Essential poetry commands:

| Command / File                                             | Description                                                                                                                                                                                                                |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>poetry install</code>                                | <b>Installs all dependencies.</b> If a <code>poetry.lock</code> file exists, installs exact versions from it. If not, resolves dependencies, creates the <code>poetry.lock</code> file, and installs.                      |
| <code>poetry add</code><br><code>&lt;package&gt;</code>    | Adds a package, resolves the new dependency tree, updates the <code>pyproject.toml</code> and updates the <code>poetry.lock</code> file.                                                                                   |
| <code>poetry remove</code><br><code>&lt;package&gt;</code> | Removes a package, resolves the new dependency tree, updates the <code>pyproject.toml</code> and updates the <code>poetry.lock</code> file.                                                                                |
| <code>poetry show</code>                                   | Lists all installed packages (and their versions) as recorded in the <code>poetry.lock</code> file.                                                                                                                        |
| <code>poetry show</code><br><code>--tree</code>            | Displays the dependency tree based on the resolved dependencies in the <code>poetry.lock</code> file.                                                                                                                      |
| <code>poetry show</code><br><code>--outdated</code>        | Compares versions in the <code>poetry.lock</code> file against the latest available on the remote repository.                                                                                                              |
| <code>poetry run</code><br><code>&lt;command&gt;</code>    | Runs a command in the virtual environment whose packages are defined by the <code>poetry.lock</code> file.                                                                                                                 |
| <code>poetry.lock</code> (File)                            | <b>The most important file for reproducibility.</b> Pins exact versions of every package and dependency. <b>Always commit this to version control</b> to ensure all developers and deployments use identical dependencies. |

### Core Files to Know

- **pyproject.toml:**  
The *declarative* file where you define your project’s metadata, dependencies, and build system.  
This file is meant to be edited by humans.
- **poetry.lock:**  
The *definitive record* of the exact versions of every package that makes your project work.  
This file is generated and managed by Poetry.  
**You should commit this to version control** to ensure everyone (and your deployments) uses identical dependencies.

**Summary:** `poetry` is highly recommended for modern Python projects where you want reliable dependency management, easy environment setup, and reproducible results.

### 2.6.3 Difference Between `conda` and `poetry`

Both `conda` and `poetry` are tools for managing Python environments and dependencies, but they have different strengths and use cases.

#### **conda:**

- Manages both Python environments and packages, including non-Python dependencies (e.g., C libraries, compilers)
- Works with multiple languages (Python, R, etc.)
- Uses the Anaconda repository, which includes many scientific and data science packages
- Great for data science workflows, especially when you need packages with compiled code or system-level dependencies
- Handles complex dependency resolution and environment isolation

#### **poetry:**

- Focuses on Python projects and packages
- Uses `pyproject.toml` and `poetry.lock` for reproducible builds
- Automatically creates and manages virtual environments
- Excellent for modern Python application development and publishing to PyPI
- Handles dependency resolution and version locking for Python packages only

#### **Key differences:**

- `conda` can install system-level and non-Python dependencies; `poetry` only manages Python packages
- `conda` environments can include packages from the Anaconda repository; `poetry` uses PyPI
- `poetry` is ideal for pure Python projects and reproducible builds; `conda` is better for scientific computing and mixed-language projects



### When to use each:

- Use **conda** if you need scientific libraries, compiled code, or non-Python dependencies
- Use **poetry** for modern Python projects, apps, and libraries where you want easy dependency management and publishing

### Summary Table:

| Feature                 | conda           | poetry      |
|-------------------------|-----------------|-------------|
| Language support        | Python, R, more | Python only |
| Non-Python dependencies | Yes             | No          |
| Environment isolation   | Yes             | Yes         |
| Dependency resolution   | Excellent       | Excellent   |
| Reproducibility         | Good            | Excellent   |
| Publishing to PyPI      | No              | Yes         |

Choose the tool that best fits your project needs!

## 2.6.4 Modern Python Package Managers: `pip`, `conda`, `poetry`, `uv`

Python package management has evolved rapidly, making it easier to install, update, and manage dependencies for any project.

- **pip** is the standard tool for installing Python packages from PyPI. It works well for most projects and is simple to use.
- **.venv** helps you create isolated environments so each project has its own dependencies.
- **conda** offers advanced environment and package management, especially for scientific computing and projects with non-Python dependencies.
- **poetry** provides modern dependency management, automatic environment creation, and reproducibility for Python projects.
- **uv** is a new, high-performance package manager written in Rust. It's designed for speed and supports modern workflows. Learn more: [uv GitHub page](#)

### For this course:

- You'll use **pip** and **.venv** for installing packages and managing your environment.
- As you tackle larger or more complex projects, consider exploring **conda**, **poetry**, and **uv** for better performance, reproducibility, and ease of use.

### Summary:

- Choose the tool that fits your workflow and project needs.

- Stay curious—new tools like `uv` are making Python development faster and more reliable.

A good package manager and environment strategy will save you time, prevent headaches, and make your code more reproducible and shareable.

## 2.7 Independent Study: Practice Environment Setup with `pip`

**Objective:** Build hands-on skills in creating, managing, and verifying Python environments and packages using `pip`.

### 2.7.1 Instructions

#### Step 1: Create Your Workspace

- Make a folder named `test_pip_env`.
- Open the folder in VS Code.

#### Step 2: Create a Virtual Environment

- create a `.venv` environment

#### Step 3: Install Required Packages

- With the environment active, install packages using `pip`:

```
pip install numpy pandas
```

#### Step 4: Export Your Environment Configuration

- Save your environment's package list to a file:

```
pip freeze > stat303_env.txt
```

#### Step 5: Deactivate and Remove the Environment

- Deactivate the environment:

```
deactivate
```

- Remove the environment folder to clean up.

#### Step 6: Recreate the Environment from the Exported File

- Create a new environment and activate it.

- Install packages from your saved file:

```
pip install -r stat303_env.txt
```

- Verify that `numpy`, `pandas`, `matplotlib`, and (if installed) `scikit-learn` are present.

### Step 7: Run a Jupyter Notebook

- Create a notebook in the recreated environment.
- Import `numpy`, `pandas`, `matplotlib`, and `scikit-learn`.
- Print the versions of these packages to confirm setup.
- Generate html file from the notebook using `quarto`

This exercise will help you master environment management and reproducibility—key skills for any data scientist.

## 2.8 Resources

- [Tutorials in Python Packaging User Guide](#)
- [Poetry Documentation](#)
- [uv GitHub page](#)

## 3 Enhancing Workflow in Jupyter Notebooks

<IPython.core.display.Image object>

### 3.1 Learning Objectives

In this lecture, we'll explore how to optimize your workflow in Jupyter notebooks by leveraging **magic commands** and **shell commands**, understanding **file paths**, and interacting with the **filesystem** using the **os** module.

By completing this lecture, you will be able to:

- Run shell commands directly within a notebook.
- Navigate and manipulate files and directories efficiently.
- Integrate external tools and scripts seamlessly into your workflow.

By mastering these techniques, you'll be able to work more efficiently and handle complex data science tasks with ease.

### 3.2 Magic Commands

#### 3.2.1 What are Magic Commands?

Magic commands in Jupyter are shortcuts that extend the functionality of the notebook environment. There are two types of magic commands:

- **Line magics:** Commands that operate on a single line.
- **Cell magics:** Commands that operate on the entire cell.

You can access the full list of magic commands by typing:

```
# show all the available magic commands on the system
%lsmagic
```

Available line magics:

```
%alias %alias_magic %autoawait %autocall %automagic %autosave %bookmark %cd %clear %
```

Available cell magics:

```
%%! %%HTML %%SVG %%bash %%capture %%cmd %%code_wrap %%debug %%file %%html %%javasc
```

Automagic is ON, % prefix IS NOT needed for line magics.

### 3.2.2 Line Magic

In Jupyter notebooks, line magic commands are invoked by placing a single percentage sign (%) in front of the statement, allowing for quick, inline operations.

#### 3.2.2.1 %time: Timing the execution of code

In data science, it is often crucial to evaluate the performance of specific code snippets or algorithms, and the %time magic command provides a simple and efficient way to measure the execution time of individual statements, helping you identify bottlenecks and optimize your code for better performance.

```
def my_dot(a, b):
    """
    Compute the dot product of two vectors

    Args:
        a (ndarray (n,)): input vector
        b (ndarray (n,)): input vector with same dimension as a

    Returns:
        x (scalar):
    """
    x=0
    for i in range(a.shape[0]):
        x = x + a[i] * b[i]
    return x
```

```
import numpy as np
np.random.seed(1)
a = np.random.rand(10000000) # very large arrays
b = np.random.rand(10000000)
```

Let's use `%time` to measure the execution time of a single line of code.

```
# Example: Timing a list comprehension
%time np.dot(a, b)
```

```
CPU times: total: 15.6 ms
Wall time: 32.9 ms
```

```
2501072.5816813153
```

```
%time my_dot(a, b)
```

```
CPU times: total: 1.88 s
Wall time: 1.86 s
```

```
2501072.5816813707
```

To capture the output of `%time` (or `%timeit`), you cannot directly assign it to a variable as it's a magic command that prints the result to the notebook's output. However, you can use Python's built-in `time` module to manually time your code and assign the execution time to a variable.

Here's how you can do it using the `time` module:

```
import time
tic = time.time() # capture start time
c = np.dot(a, b)
toc = time.time() # capture end time

print(f"np.dot(a, b) = {c:.4f}")
print(f"Vectorized version duration: {1000*(toc-tic):.4f} ms ")

tic = time.time() # capture start time
c = my_dot(a,b)
toc = time.time() # capture end time

print(f"my_dot(a, b) = {c:.4f}")
print(f"loop version duration: {1000*(toc-tic):.4f} ms ")

del(a);del(b) #remove these big arrays from memory
```

```
np.dot(a, b) = 2501072.5817
Vectorized version duration: 2.9922 ms
my_dot(a, b) = 2501072.5817
loop version duration: 2004.3225 ms
```

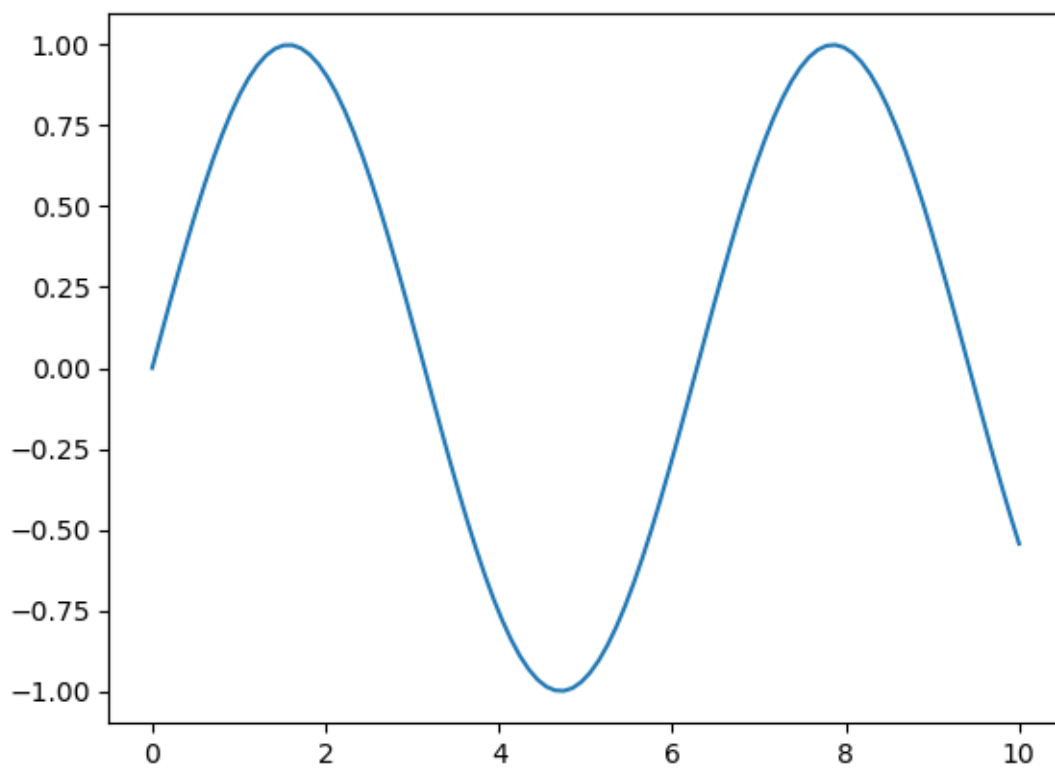
### 3.2.2.2 %matplotlib inline: Displaying plots inline

This command allows you to embed plots within the notebook.

```
# Example: Using %matplotlib inline to display a plot
import matplotlib.pyplot as plt
import numpy as np

%matplotlib inline

x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y);
```



### 3.2.3 Cell Magic Commands

While cell magic commands are denoted with double percentage signs (%%) at the beginning of the cell, enabling you to apply commands to the entire cell for more complex tasks. Note that a cell magic command in Jupyter notebook has to be the first line in the code cell!

#### 3.2.3.1 %%time: Timing cell execution

This cell magic is useful for measuring the execution time of an entire cell.

```
%%time
# Example: Timing a cell with matrix multiplication

A = np.random.rand(1000, 1000)
B = np.random.rand(1000, 1000)
C = np.dot(A, B)
```

```
CPU times: total: 219 ms
Wall time: 23 ms
```

Question: there are several timing magic commands that can be confusing due to their similarities, they are %time,%timeit, %%time, and %timeit. Do your own research on the differences among them

## 3.3 Shell Commands in Jupyter Notebooks

### What are Shell Commands?

Shell commands let you interact directly with your computer's operating system from within a Jupyter notebook. This means you can manage files, check your environment, and run system tools without leaving your notebook. To run a shell command in Jupyter, start the line with an exclamation mark (!). For example, !ls lists files on macOS/Linux, while !dir does the same on Windows.

### How does it work?

- Python code is executed by the IPython kernel inside the notebook.
- Shell commands (lines starting with !) are sent to your computer's shell (like Bash, Command Prompt, or PowerShell), not to Python.
- This separation means you can use both Python and shell commands in the same notebook, but they run in different environments.



### 3.3.1 Using Shell Commands

#### 3.3.1.1 Print the current working directory

To see where your notebook is running, use: - On macOS/Linux: `!pwd` - On Windows: `!cd`

This helps you understand where files will be saved or loaded from.

```
!cd
```

```
!cd
```

```
c:\Users\lsi8012\mydev\STAT303-1-class-notes
```

#### 3.3.1.2 `!ls` (`!dir` on Windows): Listing files and directories

To view all files and folders in your current directory, use a shell command:

- On macOS/Linux: `!ls`
- On Windows: `!dir`

This is useful for quickly checking what data, scripts, or notebooks are available in your workspace. If you want to see hidden files (those starting with a dot), use `!ls -a` on macOS/Linux or `!dir /a` on Windows.

**Tip:** If you get an error, double-check that you are using the correct command for your operating system.

## 3.4 Installing Required Packages Within Your Notebook

When working in Jupyter notebooks, you often need to install new Python packages. There are two main ways to do this:

#### 1. Shell Command (not recommend):

- Use an exclamation mark (!) before the command:
  - `!pip install package_name`
- This runs the command in your system shell, which may not always install the package in the same environment as your notebook kernel.

#### 2. Magic Command:

- Use a percent sign (%) before the command:
  - `%pip install package_name`
- This is Jupyter-specific and ensures the package is installed in the environment where your notebook is running.

### Example:

- `%pip install numpy` # Installs numpy in the notebook’s environment
- `!pip install numpy` # Installs numpy using the system shell (may differ from notebook environment)

Unlike shell commands, the `%pip` magic command is designed specifically for Jupyter notebooks. It automatically installs packages into the exact environment your notebook kernel is using, reducing confusion and installation errors.

```
%pip install numpy
```

```
Requirement already satisfied: numpy in c:\users\lsi8012\appdata\local\anaconda3\lib\site-pa
Note: you may need to restart the kernel to use updated packages.
```

In most cases, you don’t need to type % before `pip install numpy` because Jupyter’s “automagic” feature is enabled by default. Automagic lets you use line magics without the % (unless their name conflicts with Python variables or commands).

### How automagic works:

- If you type `pip install numpy`, Jupyter will interpret it as `%pip install numpy` automatically.

### Best practice:

- Use `%pip install ...` for clarity and to avoid conflicts.
- Automagic is convenient, but explicit magic commands are safer in shared or complex notebooks.

**Note:** Cell magics still require the %% prefix.

## 3.5 File Paths in Python

Understanding how to specify file paths in Python is essential for loading and saving data. File paths tell Python where to find or store your files (e.g., datasets, results, or scripts).

There are two main types of file paths: **absolute paths** and **relative paths**.

- **Absolute Path:** Gives the complete location of a file or folder from the root of your file system. It always points to the same place, no matter where your code is running.
- **Relative Path:** Specifies the location of a file or folder in relation to your current working directory. It is shorter and more flexible, making your code easier to share and reuse.

Choosing the right type of path helps avoid errors and makes your code more portable across different computers and operating systems.

### 3.5.1 Absolute Path

An **absolute path** provides the full address to a file or directory, starting from the root of your system. It does **not** depend on where your code is running.

**Example (Windows):**

```
# Absolute path example (Windows)
file_path = r"C:\Users\Username\Documents\data.csv"
```

```
!conda env list
```

```
# conda environments:
#
base                  *  C:\Users\lsi8012\AppData\Local\anaconda3
```

The path associated with each conda env is absolute path.

### 3.5.2 Relative Path

A **relative path** describes the location of a file or folder in relation to your current working directory. It is shorter, more flexible, and makes your code easier to share and run on different computers.

Relative paths are especially useful in projects with organized folder structures, because they allow you to move your code and data together without changing file references.

To find your current working directory in your notebook, you can use a magic command:

- Magic command: `%pwd`

Knowing your current working directory helps you understand where Python will look for files when using relative paths.

```
# Magic command
%pwd
```

```
'c:\\Users\\lsi8012\\Documents\\Courses\\FA24\\DataScience_Intro_python_fa24_Sec20_21'
```

Example of a Relative Path:

```
# Example of relative path
file_path = "sample.txt" # Relative to the current directory
```

The relative path `sample.txt` means that there is a file in the current working directory.

### 3.5.3 Methods to Specify File Paths in Windows

Specifying file paths correctly is crucial for avoiding errors and making your code portable. Windows uses backslashes (`\`) to separate folders, but in Python, a single backslash is an escape character (e.g., `\n` for newline), which can lead to mistakes if not handled properly.

Here are four reliable ways to specify file paths in Windows:

#### 3.5.3.1 Method 1: Using Escaped Backslashes

Use double backslashes (`\\`) to prevent Python from interpreting them as escape characters.

```
file_path = "C:\\Users\\Username\\Documents\\data.csv"
```

#### 3.5.3.2 Method 2: Using Raw Strings

Prefix the path with `r` to tell Python to treat backslashes as literal characters.

```
file_path = r"C:\Users\Username\Documents\data.csv"
```

### 3.5.3.3 Method 3: Using forward slashes (/)

Python accepts forward slashes (/) in file paths, even on Windows. This is often the simplest and most portable method.

```
file_path = "C:/Users/Username/Documents/data.csv"
```

### 3.5.3.4 Method 4: Using `os.path.join`

Use `os.path.join()` to build paths programmatically. This method automatically uses the correct separator for your operating system, making your code cross-platform.

Using these methods helps prevent bugs, makes your code easier to share, and ensures it works on different computers and operating systems.

```
import os
file_path = os.path.join("C:", "Users", "Username", "Documents", "data.csv")
```

This method works across different operating systems because `os.path.join` automatically uses the correct path separator (\ for Windows and /for Linux/Mac).

## 3.5.4 File paths in macOS and Linux

macOS and Linux use forward slashes (/) as path separators, which is exactly what Python expects by default. This means you can specify file paths directly, like `/Users/yourname/Documents/data.csv`, without worrying about escape characters or compatibility issues.

**Why is this helpful?** - Forward slashes work seamlessly in Python on macOS, Linux, and even Windows. - You avoid common errors caused by backslashes (which are escape characters in Python). - Code written with forward slashes is portable and recommended for all platforms.

### 3.5.4.1 Best Practices for File Paths in Data Science

- Prefer **relative paths** within your project folders—this makes your code portable and easy to share or move.
- Use **absolute paths** only for files outside your project or when you need a fixed location.
- Always check your **current working directory** before reading or writing files to avoid confusion and errors.

- Avoid hardcoding file paths directly in your code; use variables or configuration files for flexibility.
- For cross-platform compatibility, use **forward slashes (/)** in file paths or build paths programmatically with `os.path.join()`.
- Document your file structure and path conventions in your project README to help collaborators.
- When sharing code, test file paths on both Windows and macOS/Linux to ensure portability.

## 3.6 Interacting with the OS and Filesystem

In data science projects, you often work with data files (such as CSV, Excel, or JSON) stored in various folders. Managing these files efficiently is essential for reproducible workflows and organized projects.

The Python `os` module provides powerful tools to interact with your operating system and manage files and directories directly from your notebook. With `os`, you can: - Check your current working directory - List files and folders - Create, rename, or delete directories - Move between folders - Check if files or folders exist before using them

Let's import the `os` module and explore some of its most useful functions for data science tasks.

```
import os
```

We can get the location of the current working directory using the `os.getcwd()` function.

```
os.getcwd()
```

```
'c:\\Users\\lsi8012\\mydev\\STAT303-1-class-notes'
```

The command `os.chdir('..')` in Python changes the current working directory to the parent directory of the current one.

Note that `..` as the path notation for the parent directory is universally true across all major operating systems, including Windows, macOS, and Linux. It allows you to move one level up in the directory hierarchy, which is very useful when navigating directories programmatically, especially in scripts where directory traversal is needed.

```
os.chdir('..')
```

```
os.getcwd()
```

```
'c:\\Users\\lsi8012\\mydev'
```

`os.chdir()` is used to change the current working directory.

For example: `os.chdir('./week2')`

`./week2` is a relative path:

- `.` refers to the current directory.
- `week2` is a folder inside the current directory.

The `os.listdir()` function in Python returns a list of all files and directories in the specified path. If no path is provided, it returns the contents of the current working directory.

```
os.listdir()
```

```
['limonstar-web',  
'llm2005spring',  
'llm2005spring_back',  
'mcdc_2025su',  
'mcdc_2025su_new',  
'STAT201',  
'STAT303-1-class-notes',  
'STAT303-2-class-notes',  
'STAT303-3-class-notes',  
'stat359_su25',  
'stat362']
```

Check whether a specific folder/file exist in the current working directory

```
'data' in os.listdir('.')
```

```
False
```

## 3.7 Navigate between directories and understand . and .. in paths

In Python, you can use the `os` module to work with directories and paths, and it's important to understand how relative paths work:

- `.` refers to the **current directory**
- `..` refers to the **parent directory**

This knowledge is essential for organizing files and writing portable code.

From the command line, you can use the `cd` (change directory) command to move between folders:

- `cd <folder_name>` moves **down** into a child folder
- `cd ..` moves **up** to the parent folder

By combining these commands, you can navigate to the desired location.

## 3.8 Independent Study

### 3.8.1 Setting Up Your Data Science Workspace

To reinforce and apply the skills from this lecture, complete the following hands-on tasks:

#### 1. Set Up Your Workspace

- Create a folder named `STAT303-1` to organize all course materials.
- Set up a dedicated `.venv` environment for your coursework to keep dependencies isolated.
- Organize your files into subfolders for datasets, assignments, project, quizzes, and lectures. The layout looks like below

```
stat303-1/  
  .venv/           # Python virtual environment  
  datasets/  
  project/  
  homework assignments/  
  small assignments/  
  lectures/
```

- Use the `os` module or shell commands in your notebook to create these directories programmatically.



## 2. Practice Magic Commands

- Use `%timeit` to measure the execution time of a simple Python function in your notebook.
- Explore `%lsmagic` to discover all available magic commands and experiment with a few.

## 3. Run Shell Commands

- Use `!ls` (or `!dir` on Windows) to list the contents of the directories you created.
- Use `!pwd` (or `!cd` on Windows) to print your current working directory.

## 4. Navigate between directories using `.` and `..`

**Tip:** Document your process and any issues you encounter. This will help you troubleshoot and share your workflow with others.

## 3.9 References and Further Learning

- **IPython Magic Commands**  
[IPython Documentation – Magic Commands](#)  
A complete list of available line and cell magics, such as `%time`, `%pip`, and `%%bash`.
- **File and Directory Management in Python**  
[Python `os` module documentation](#)  
Covers functions like `os.getcwd()`, `os.chdir()`, and path handling.

## **Part II**

# **Python refresher**

## 4 Python Basics

<IPython.core.display.Image object>

Before diving into new material, this chapter provides a quick refresher on key Python concepts covered in STAT201, which is the prerequisite for this course. If you have not taken STAT201, or if you feel you need to strengthen your Python skills, please review chapters 3–8 in the [STAT201 book](#). A solid understanding of Python basics will help you succeed in this course and make it easier to follow the advanced topics ahead.

### 4.1 Python Variables

Variables are fundamental building blocks in Python—they allow you to store, update, and reference data throughout your code. Choosing clear and descriptive variable names makes your code easier to read, debug, and share with others.

#### 4.1.1 Rules for variable names

- Variable names must start with a letter (a–z, A–Z) or an underscore (\_). They cannot begin with a number.
- Names can include letters, digits, and underscores (`a_variable`, `profit_margin`, `the_3_musketeers`).
- Variable names are case-sensitive: `score`, `Score`, and `SCORE` are all different variables.
- Avoid using Python reserved words (like `for`, `if`, `class`, etc.) as variable names.

**Tip:** Use descriptive names that reflect the purpose of the variable. This helps you and others understand your code at a glance.

Here are some examples of good variable names:

```
# Examples of valid variable names

a_variable = 23
is_today_Saturday = False
my_favorite_car = "Delorean"
the_3_musketeers = ["Athos", "Porthos", "Aramis"]
```

If you use an invalid variable name, Python will raise a `SyntaxError` and stop running your code. Always follow the naming rules to avoid these errors and keep your code readable.

### 4.1.2 Dynamic Typing in Python Variables

Python variables are dynamically typed, meaning you don't need to declare their type before using them. You can assign a value of any type to a variable, and even change its type later in your code:

```
x = 5          # x is an integer
x = "cat"      # now x is a string
```

To check the type of a variable, use the built-in `type()` function:

```
type(x)
```

This flexibility makes Python easy to use, but it's important to keep track of your variable types to avoid confusion in your code.

```
# Print the variables defined above their values and their types
print("a_variable:", a_variable, "| type:", type(a_variable))
print("is_today_Saturday:", is_today_Saturday, "| type:", type(is_today_Saturday))
print("my_favorite_car:", my_favorite_car, "| type:", type(my_favorite_car))
print("the_3_musketeers:", the_3_musketeers, "| type:", type(the_3_musketeers))
```

```
a_variable: 23 | type: <class 'int'>
is_today_Saturday: False | type: <class 'bool'>
my_favorite_car: Delorean | type: <class 'str'>
the_3_musketeers: ['Athos', 'Porthos', 'Aramis'] | type: <class 'list'>
```

### 4.1.3 Multiple Variable Assignment in Python

Python allows you to assign values to several variables at once in a single line. This technique is especially useful for initializing related variables and can make your code cleaner and more efficient.

**Example:**

```
color1, color2, color3 = "red", "green", "blue"
```

After this assignment: - color1 is “red” - color2 is “green” - color3 is “blue”

This approach works for any number of variables, as long as the number of values matches the number of variable names.

```
color1, color2, color3 = "red", "green", "blue"
print(color1)
print(color3)
```

```
red
blue
```

The same value can be assigned to multiple variables by chaining multiple assignment operations within a single statement.

```
color4 = color5 = color6 = "magenta"
print(color4)
print(color6)
```

```
magenta
magenta
```

## 4.2 Built-in data types

Python has several built-in data types for storing different kinds of information in variables.

```
<IPython.core.display.Image object>
```

### Primitive Types

In Python, **integers, floats, booleans, and None** are often called *primitive data types* because they represent a single value.

### Container (Data Structure) Types

Types such as **strings, lists, tuples, sets, and dictionaries** are *containers* because they can hold multiple values (characters in a string, items in a list, key-value pairs in a dictionary, etc.). We'll explore these container types in more detail in the next chapter.

### Identifying Types

You can check the type of any object using Python's built-in `type()` function. For example:

```
print(type(42))          # int
print(type(3.14))        # float
print(type(True))        # bool
print(type(None))        # NoneType
print(type("hello"))     # str (a sequence / container of characters)
print(type([1, 2, 3]))   # list
```

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'NoneType'>
<class 'str'>
<class 'list'>
```

## 4.3 Python Standard Library

The [Python Standard Library](#) provides a wide range of modules and built-in functions that support everyday programming tasks. These tools allow you to write code that is both efficient and readable without reinventing common functionality.

**Examples of built-in functions:** - `print()`: Displays output to the screen. - `len()`: Returns the length of an object (like a list or string). - `type()`: Shows the type of a variable. - `sum()`: Adds up all items in an iterable (like a list). - `range()`: Generates a sequence of numbers, often used in loops.

You can explore more built-in functions and modules in the official documentation. Using these tools makes your code more readable and powerful.

### Example:

**`range()`:** The `range()` function returns a sequence of evenly-spaced integer values. It is commonly used in `for` loops to define the sequence of elements over which the iterations are performed.

Below is an example where the `range()` function is used to create a sequence of whole numbers upto 10:

```
print(list(range(1,10)))
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

## Date and Time:

Python includes a powerful built-in module called `datetime` for working with dates and times. This module lets you create, manipulate, and format date/time objects easily.

- You can get the current date and time.
- You can perform arithmetic with dates (e.g., add days, subtract dates).
- You can format dates and times for display or parsing.

This is essential for tasks like timestamping data, scheduling, or analyzing time series.

```
import datetime as dt
```

```
#Defining a date-time object  
dt_object = dt.datetime(2022, 9, 20, 11,30,0)
```

Information about date and time can be accessed with the relevant attribute of the `datetime` object.

```
dt_object.day, dt_object.year
```

```
(20, 2022)
```

## Formatting Dates and Times:

The `strftime` method in the `datetime` module lets you convert a `datetime` object into a readable string using custom formats. This is useful for displaying dates in a way that matches your needs (e.g., for reports, logs, or user interfaces).

Common format codes include: - `%Y`: 4-digit year (e.g., 2025) - `%m`: 2-digit month (01-12) - `%d`: 2-digit day (01-31) - `%H`: Hour (00-23) - `%M`: Minute (00-59) - `%S`: Second (00-59)

See the [Python documentation](#) for more formatting options.

```
dt_object.strftime('%m/%d/%Y')
```

```
'09/20/2022'
```

```
dt_object.strftime('%m/%d/%y %H:%M')
```

```
'09/20/22 11:30'
```

```
dt_object.strftime('%h-%d-%Y')
```

'Sep-20-2022'

## 4.4 Third-Party Packages (Libraries)

While Python has many useful [built-in functions](#) like `print()`, `abs()`, `max()`, and `sum()`, these are often not enough for data analysis. Third-party libraries extend Python's capabilities and are essential for scientific computing and data science.

**Popular libraries and their main uses:**

1. **NumPy**: Efficient numerical operations, arrays, and mathematical functions. Essential for scientific and data analysis tasks.
2. **Pandas**: Powerful data manipulation and analysis. DataFrames and Series make reading, cleaning, and transforming data easy.
3. **Matplotlib & Seaborn**: Data visualization. Matplotlib creates a wide range of plots; Seaborn builds on Matplotlib for attractive statistical graphics.
4. **SciPy**: Advanced scientific computing—optimization, integration, statistics, and more.
5. **Scikit-learn**: Machine learning—tools for preprocessing, classification, regression, clustering, and model evaluation.
6. **Statsmodels**: Statistical modeling and inference (focuses on explanation, not just prediction).

**How to use these libraries:** 1. **Install the libraries** : Covered by Chapter 2 and 3 2. **Import the libraries** in your Python script or Jupyter notebook. 3. **Use their functions and classes** to analyze data, visualize results, and build models.

These libraries form the foundation of modern data science workflows in Python. You will gain plenty of hands-on experience with each of them throughout this course sequence.

### 4.4.1 Importing a Library

Use the `import` keyword to bring a library into your Python code.

Example:

```
import numpy as np
```

Aliases like `np` make code shorter and easier to read.



```
import numpy as np
np.arange(8)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

## Importing in Python: Key Styles

- Import a whole module:

```
import math
```

- Import specific items:

```
from random import randint
```

- Use an alias:

```
import pandas as pd
```

- Rename an imported item:

```
from os.path import join as join_path
```

Pick the style that fits your needs and keeps your code readable.

## 4.5 User-defined functions

A function is a reusable set of instructions that takes one or more inputs, performs some operations, and often returns an output. **Indeed, while python's standard library and ecosystem libraries offer a wealth of pre-defined functions for a wide range of tasks, there are situations where defining your own functions is not just beneficial but necessary.**

### 4.5.1 Creating and using functions

You can define a new function using the `def` keyword.

```
def say_hello():
    print('Hello there!')
    print('How are you?')
```

Note the round brackets or parentheses () and colon : after the function's name. Both are essential parts of the syntax. The function's *body* contains an indented block of statements. The statements inside a function's body are not executed when the function is defined. To execute the statements, we need to *call* or *invoke* the function.

```
say_hello()
```

```
Hello there!  
How are you?
```

```
def say_hello_to(name):  
    print('Hello ', name)  
    print('How are you?')
```

```
say_hello_to('Lizhen')
```

```
Hello Lizhen  
How are you?
```

```
name = input ('Please enter your name: ')  
say_hello_to(name)
```

```
Please enter your name: George  
Hello George  
How are you?
```

## 4.5.2 Variable scope: Local and global Variables

**Local variable:** When we declare variables inside a function, these variables will have a local scope (within the function). We cannot access them outside the function. These types of variables are called local variables. For example,

```
def greet():  
    message = 'Hello' # local variable  
    print('Local', message)  
greet()
```

```
Local Hello
```

```
# print(message) # try to access message variable outside greet() function, uncomment this line
```

As `message` was defined within the function `greet()`, it is local to the function, and cannot be called outside the function.

**Global variable:** A variable declared outside of the function or in global scope is known as a global variable. This means that a global variable can be accessed inside or outside of the function.

Let's see an example of how a global variable is created.

```
message = 'Hello' # declare global variable

def greet():
    print('Local', message) # declare local variable

greet()
print('Global', message)
```

```
Local Hello
Global Hello
```

## 4.5.3 Function Arguments

### 4.5.3.1 Named Arguments

When calling functions with multiple arguments, using *named* arguments improves clarity and reduces mistakes. You can also split long function calls across multiple lines for readability.

**Example:**

```
def greet(name, message):
    print(f"{message}, {name}!")

greet(name="Alice", message="Hello")
```

Named arguments make your code easier to understand and maintain.

Here is an example:

```
def loan_emi(amount, duration, rate, down_payment=0):  
    loan_amount = amount - down_payment  
    emi = loan_amount * rate * ((1+rate)**duration) / (((1+rate)**duration)-1)  
    return emi
```

```
emi1 = loan_emi(  
    amount=1260000,  
    duration=8*12,  
    rate=0.1/12,  
    down_payment=3e5  
)
```

```
emi1
```

```
14567.19753389219
```

#### 4.5.3.2 Optional Arguments

Functions with optional arguments offer more flexibility in how you can use them. You can call the function with or without the argument, and if there is no argument in the function call, then a default value is used.

```
emi2 = loan_emi(  
    amount=1260000,  
    duration=8*12,  
    rate=0.1/12)
```

```
emi2
```

```
19119.4467632335
```

#### 4.5.3.3 \*args and \*\*kwargs

We can pass a variable number of arguments to a function using two special symbols in Python:

- \*args for variable-length **positional arguments**
- \*\*kwargs for variable-length **keyword arguments**

This is useful when you want your function to accept a variety of arguments.

```
def myFun(*args,**kwargs):
    print("args: ", args)
    print("kwargs: ", kwargs)

# Now we can use both *args ,**kwargs
# to pass arguments to this function :
myFun('John',22,'cs',name="John",age=22,major="cs")
```

```
args: ('John', 22, 'cs')
kwargs: {'name': 'John', 'age': 22, 'major': 'cs'}
```

## 4.6 Control Flow

Control flow statements (like if, for, and while) let you decide how your code runs based on conditions and loops. Control flow in Python includes both conditional statements and iteration loops to manage program logic.

### 4.6.1 Conditional Statements

As in other languages, python has [built-in keywords](#) that provide conditional flow of control in the code.

#### 4.6.1.1 Branching with if, else and elif

One of the most powerful features of programming languages is *branching*: the ability to make decisions and execute a different set of statements based on whether one or more conditions are true.

##### The if statement

In Python, branching is implemented using the if statement, which is written as follows:

```
if condition:
    statement1
    statement2
```

The `condition` can be a value, variable or expression. If the condition evaluates to `True`, then the statements within the *if block* are executed. Notice the four spaces before `statement1`, `statement2`, etc. The spaces inform Python that these statements are associated with the `if` statement above. This technique of structuring code by adding spaces is called *indentation*.

**Indentation:** Python relies heavily on *indentation* (white space before a statement) to define code structure. This makes Python code easy to read and understand. You can run into problems if you don't use indentation properly. Indent your code by placing the cursor at the start of the line and pressing the `Tab` key once to add 4 spaces. Pressing `Tab` again will indent the code further by 4 more spaces, and press `Shift+Tab` will reduce the indentation by 4 spaces.

For example, let's write some code to check and print a message if a given number is even.

```
a_number = 34
```

```
if a_number % 2 == 0:
    print("We're inside an if block")
    print('The given number {} is even.'.format(a_number))
```

```
We're inside an if block
The given number 34 is even.
```

### The else statement

We may want to print a different message if the number is not even in the above example. This can be done by adding the `else` statement. It is written as follows:

```
if condition:
    statement1
    statement2
else:
    statement4
    statement5
```

If `condition` evaluates to `True`, the statements in the `if` block are executed. If it evaluates to `False`, the statements in the `else` block are executed.

```
if a_number % 2 == 0:
    print('The given number {} is even.'.format(a_number))
else:
    print('The given number {} is odd.'.format(a_number))
```

The given number 34 is even.

### The elif statement

Python also provides an `elif` statement (short for “else if”) to chain a series of conditional blocks. The conditions are evaluated one by one. For the first condition that evaluates to `True`, the block of statements below it is executed. The remaining conditions and statements are not evaluated. So, in an `if`, `elif`, `elif...` chain, at most one block of statements is executed, the one corresponding to the first condition that evaluates to `True`.

```
today = 'Wednesday'
```

```
if today == 'Sunday':
    print("Today is the day of the sun.")
elif today == 'Monday':
    print("Today is the day of the moon.")
elif today == 'Tuesday':
    print("Today is the day of Tyr, the god of war.")
elif today == 'Wednesday':
    print("Today is the day of Odin, the supreme diety.")
elif today == 'Thursday':
    print("Today is the day of Thor, the god of thunder.")
elif today == 'Friday':
    print("Today is the day of Frigga, the goddess of beauty.")
elif today == 'Saturday':
    print("Today is the day of Saturn, the god of fun and feasting.")
```

Today is the day of Odin, the supreme diety.

In the above example, the first 3 conditions evaluate to `False`, so none of the first 3 messages are printed. The fourth condition evaluates to `True`, so the corresponding message is printed. The remaining conditions are skipped. Try changing the value of `today` above and re-executing the cells to print all the different messages.

### Using if, elif, and else together

You can also include an `else` statement at the end of a chain of `if`, `elif...` statements. This code within the `else` block is evaluated when none of the conditions hold true.

```
a_number = 49
```

```

if a_number % 2 == 0:
    print('{} is divisible by 2'.format(a_number))
elif a_number % 3 == 0:
    print('{} is divisible by 3'.format(a_number))
elif a_number % 5 == 0:
    print('{} is divisible by 5'.format(a_number))
else:
    print('All checks failed!')
    print('{} is not divisible by 2, 3 or 5'.format(a_number))

```

```

All checks failed!
49 is not divisible by 2, 3 or 5

```

## Non-Boolean Conditions

Note that conditions do not necessarily have to be booleans. In fact, a condition can be any value. The value is converted into a boolean automatically using the `bool` operator. Any value in Python can be converted to a Boolean using the `bool` function.

Only the following values evaluate to **False** (they are often called *falsy* values):

1. The value **False** itself
2. The integer **0**
3. The float **0.0**
4. The empty value **None**
5. The empty text **""**
6. The empty list **[]**
7. The empty tuple **()**
8. The empty dictionary **{}**
9. The empty set **set()**
10. The empty range **range(0)**

Everything else evaluates to **True** (a value that evaluates to **True** is often called a *truthy* value).

```

if '':
    print('The condition evaluted to True')
else:
    print('The condition evaluted to False')

```

```
The condition evaluted to False
```



```
if 'Hello':  
    print('The condition evaluted to True')  
else:  
    print('The condition evaluted to False')
```

The condition evaluted to True

```
if { 'a': 34 }:  
    print('The condition evaluted to True')  
else:  
    print('The condition evaluted to False')
```

The condition evaluted to True

```
if None:  
    print('The condition evaluted to True')  
else:  
    print('The condition evaluted to False')
```

The condition evaluted to False

### Nested conditional statements

The code inside an if block can also include an if statement inside it. This pattern is called **nesting** and is used to check for another condition after a particular condition holds true.

```
a_number = 15
```

```
if a_number % 2 == 0:  
    print("{} is even".format(a_number))  
    if a_number % 3 == 0:  
        print("{} is also divisible by 3".format(a_number))  
    else:  
        print("{} is not divisibule by 3".format(a_number))  
else:  
    print("{} is odd".format(a_number))  
    if a_number % 5 == 0:  
        print("{} is also divisible by 5".format(a_number))  
    else:  
        print("{} is not divisibule by 5".format(a_number))
```

```
15 is odd
15 is also divisible by 5
```

Notice how the `print` statements are indented by 8 spaces to indicate that they are part of the inner `if/else` blocks.

Nested `if, else` statements are often confusing to read and prone to human error. It's good to avoid nesting whenever possible, or limit the nesting to 1 or 2 levels.

### Shorthand if conditional expression

A frequent use case of the `if` statement involves testing a condition and setting a variable's value based on the condition.

Python provides a shorter syntax, which allows writing such conditions in a single line of code. It is known as a *conditional expression*, sometimes also referred to as a *ternary operator*. It has the following syntax:

```
x = true_value if condition else false_value
```

It has the same behavior as the following `if-else` block:

```
if condition:
    x = true_value
else:
    x = false_value
```

Let's try it out for the example above.

```
parity = 'even' if a_number % 2 == 0 else 'odd'

print('The number {} is {}'.format(a_number, parity))
```

The number 15 is odd.

### The `pass` statement

`if` statements cannot be empty, there must be at least one statement in every `if` and `elif` block. We can use the `pass` statement to do nothing and avoid getting an error.

```
a_number = 9
```

```
# please uncomment the code below and see the error message
# if a_number % 2 == 0:

# elif a_number % 3 == 0:
#     print('{} is divisible by 3 but not divisible by 2')
```

As there must be at least one statement within the `if` block, the above code throws an error.

```
if a_number % 2 == 0:
    pass
elif a_number % 3 == 0:
    print('{} is divisible by 3 but not divisible by 2'.format(a_number))
```

9 is divisible by 3 but not divisible by 2

## 4.6.2 Loops

### 4.6.2.1 while loops

Another powerful feature of programming languages, closely related to branching, is running one or more statements multiple times. This feature is often referred to as *iteration* or *looping*, and there are two ways to do this in Python: using `while` loops and `for` loops.

`while` loops have the following syntax:

```
while condition:
    statement(s)
```

Statements in the code block under `while` are executed repeatedly as long as the `condition` evaluates to `True`. Generally, one of the statements under `while` makes some change to a variable that causes the condition to evaluate to `False` after a certain number of iterations.

Let's try to calculate the factorial of 100 using a `while` loop. The factorial of a number `n` is the product (multiplication) of all the numbers from 1 to `n`, i.e.,  $1*2*3*\dots*(n-2)*(n-1)*n$ .

```

result = 1
i = 1

while i <= 10:
    result = result * i
    i = i+1

print('The factorial of 100 is: {}'.format(result))

```

The factorial of 100 is: 3628800

#### 4.6.2.2 Infinite Loops

Suppose the condition in a `while` loop always holds true. In that case, Python repeatedly executes the code within the loop forever, and the execution of the code never completes. This situation is called an infinite loop. It generally indicates that you’ve made a mistake in your code. For example, you may have provided the wrong condition or forgotten to update a variable within the loop, eventually falsifying the condition.

If your code is *stuck* in an infinite loop during execution, just press the “Stop” button on the toolbar (next to “Run”) or select “Kernel > Interrupt” from the menu bar. This will *interrupt* the execution of the code. The following two cells both lead to infinite loops and need to be interrupted.

```
# INFINITE LOOP - INTERRUPT THIS CELL
```

```

result = 1
i = 1

while i <= 100:
    result = result * i
    # forgot to increment i

```

```
# INFINITE LOOP - INTERRUPT THIS CELL
```

```

result = 1
i = 1

while i > 0 : # wrong condition
    result *= i
    i += 1

```

### 4.6.2.3 break and continue statements

In Python, `break` and `continue` statements can alter the flow of a normal loop.

We can use the `break` statement within the loop's body to immediately stop the execution and `break` out of the loop. with the `continue` statement. If the condition evaluates to `True`, then the loop will move to the next iteration.

```
i = 1
result = 1

while i <= 100:
    result *= i
    if i == 42:
        print('Magic number 42 reached! Stopping execution..')
        break
    i += 1

print('i:', i)
print('result:', result)
```

```
Magic number 42 reached! Stopping execution..
i: 42
result: 1405006117752879898543142606244511569936384000000000
```

```
i = 1
result = 1

while i < 8:
    i += 1
    if i % 2 == 0:
        print('Skipping {}'.format(i))
        continue
    print('Multiplying with {}'.format(i))
    result = result * i

print('i:', i)
print('result:', result)
```

```
Skipping 2
Multiplying with 3
Skipping 4
```

```
Multiplying with 5
Skipping 6
Multiplying with 7
Skipping 8
i: 8
result: 105
```

In the example above, the statement `result = result * i` inside the loop is skipped when `i` is even, as indicated by the messages printed during execution.

**Logging:** The process of adding `print` statements at different points in the code (often within loops and conditional statements) for inspecting the values of variables at various stages of execution is called logging. As our programs get larger, they naturally become prone to human errors. Logging can help in verifying the program is working as expected. In many cases, `print` statements are added while writing & testing some code and are removed later.

Task: Guess the output and explain it.

```
# Use of break statement inside the loop

for val in "string":
    if val == "i":
        break
    print(val)

print("The end")
```

```
s
t
r
The end
```

```
# Program to show the use of continue statement inside loops

for val in "string":
    if val == "i":
        continue
    print(val)

print("The end")
```

```
s
t
r
n
g
The end
```

#### 4.6.2.4 for loops

A `for` loop is used for iterating or looping over sequences, i.e., lists, tuples, dictionaries, strings, and *ranges*. `For` loops have the following syntax:

```
for value in sequence:
    statement(s)
```

The statements within the loop are executed once for each element in `sequence`. Here's an example that prints all the element of a list.

```
days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']

for day in days:
    print(day)
```

```
Monday
Tuesday
Wednesday
Thursday
Friday
```

```
# Looping over a string
for char in 'Monday':
    print(char)
```

```
M
o
n
d
a
y
```

```
# Looping over a dictionary
person = {
    'name': 'John Doe',
    'sex': 'Male',
    'age': 32,
    'married': True
}

for key, value in person.items():
    print("Key:", key, ",", "Value:", value)
```

```
Key: name , Value: John Doe
Key: sex , Value: Male
Key: age , Value: 32
Key: married , Value: True
```

#### 4.6.2.5 Iterating using range

The `range` function is used to create a sequence of numbers that can be iterated over using a `for` loop. It can be used in 3 ways:

- `range(n)` - Creates a sequence of numbers from 0 to `n-1`
- `range(a, b)` - Creates a sequence of numbers from `a` to `b-1`
- `range(a, b, step)` - Creates a sequence of numbers from `a` to `b-1` with increments of `step`

Let's try it out.

```
for i in range(4):
    print(i)
```

```
0
1
2
3
```

```
for i in range(3, 8):
    print(i)
```



```
3
4
5
6
7
```

```
for i in range(3, 14, 4):
    print(i)
```

```
3
7
11
```

Ranges are used for iterating over lists when you need to track the index of elements while iterating.

```
a_list = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']

for i in range(len(a_list)):
    print('The value at position {} is {}'.format(i, a_list[i]))
```

```
The value at position 0 is Monday.
The value at position 1 is Tuesday.
The value at position 2 is Wednesday.
The value at position 3 is Thursday.
The value at position 4 is Friday.
```

## 4.7 Object Oriented Programming

Python is an object-oriented programming language. In layman terms, it means that every number, string, data structure, function, class, module, etc., exists in the python interpreter as a python object. An object may have attributes and methods associated with it. For example, let us define a variable that stores an integer:

```
var = 2
```

The variable `var` is an object that has attributes and methods associated with it. For example a couple of its attributes are `real` and `imag`, which store the real and imaginary parts respectively, of the object `var`:

```
print("Real part of 'var': ",var.real)
print("Real part of 'var': ",var.imag)
```

```
Real part of 'var':  2
Real part of 'var':  0
```

**Attribute:** An attribute is a value associated with an object, defined within the class of the object.

**Method:** A method is a function associated with an object, defined within the class of the object, and has access to the attributes associated with the object.

For looking at attributes and methods associated with an object, say `obj`, press tab key after typing `obj.`

Consider the example below of a class *example\_class*:

```
class example_class:
    class_name = 'My Class'
    def my_method(self):
        print('Hello World!')

e = example_class()
```

In the above class, `class_name` is an attribute, while `my_method` is a method.

#### 4.7.0.1 Call by Reference in Python

Python uses *call by object reference* (also called *call by sharing*). When you assign an object to a variable, the variable points to the object in memory—not a copy.

Changes made through one reference affect the original object. This is important to remember when working with mutable types like lists and dictionaries.

```
x = [5,3]
```

The variable name `x` is a reference to the memory location where the object `[5, 3]` is stored. Now, suppose we assign `x` to a new variable `y`:

```
y = x
```

In the above statement the variable name `y` now refers to the same object `[5,3]`. The object `[5,3]` does **not** get copied to a new memory location referred by `y`. To prove this, let us add an element to `y`:

```
y.append(4)
print(y)
```

```
[5, 3, 4]
```

```
print(x)
```

```
[5, 3, 4]
```

When we changed `y`, note that `x` also changed to the same object, showing that `x` and `y` refer to the same object, instead of referring to different copies of the same object.

## 5 Data Structures

<IPython.core.display.Image object>

In data science, you'll shape raw data **before modeling**. These built-ins are the foundation that explain why `Series/DataFrame` (pandas) and `ndarray` (NumPy) behave the way they do.

### 5.1 Primitives vs. Containers

- **Primitive (single value):** `int`, `float`, `bool`, `None`
- **Containers (hold multiple values):** `str`, `list`, `tuple`, `set`, `dict`

Tip: Check any object's type with `type(x)`.

### 5.2 Core Built-in Data Structures

#### 5.2.1 Sequences (ordered, indexable)

Types: `list`, `tuple`, `str`

```
# list (mutable)
nums = [10, 20, 20, 30]
nums.append(40)      # mutate in place
nums[0] = 11         # item assignment
nums_slice = nums[1:3] # slicing

# tuple (immutable)
pt = (42.0, -1.5)     # cannot reassign pt[0]

# string (immutable sequence of characters)
s = "data"
s2 = s.upper()        # returns a new string; s unchanged
```

**When to use:**

- **list**: grow/shrink, frequent edits, ordered data
- **tuple**: fixed-size records, function returns, hashable as dict keys
- **str**: text processing (we'll also use `nltk` later)

### 5.2.2 Sets (unordered, unique)

**Types:** set

```
a = {1, 2, 2, 3}      # {1, 2, 3}
b = {3, 4}
a | b                # union -> {1, 2, 3, 4}
a & b                # intersection -> {3}
a - b                # difference -> {1, 2}
```

{1, 2}

**When to use:**

- Deduplication, membership tests, fast set algebra.

### 5.2.3 Mappings (key–value)

**Types:** dict

```
student = {"name": "Alex", "year": 3, "major": "Stats"}
student["year"] = 4          # update
student["gpa"] = 3.7         # insert
for k, v in student.items(): # iterate keys & values
    print(k, "->", v)
```

```
name -> Alex
year -> 4
major -> Stats
gpa -> 3.7
```

**When to use:**

- Labeled data, lookups, configuration/state.

## 5.3 Iterables vs. Iterators

In Python, an **iterable** is any object capable of returning its members one at a time, such as lists, tuples, strings, and dictionaries. You can loop over iterables using a **for** loop.

An **iterator** is an object that represents a stream of data; it produces the next value when you call `next()` on it. Iterators are created from iterables using the `iter()` function.

**Key differences:** - Iterables can be looped over, but do not remember their position. - Iterators remember their position and can only be advanced one item at a time.

**Example:**

```
my_list = [1, 2, 3]
my_iter = iter(my_list)
print(next(my_iter)) # 1
print(next(my_iter)) # 2
print(next(my_iter)) # 3
```

Understanding the difference helps you write efficient loops and work with data streams in Python.

## 5.4 Common, Efficient Operations

### 5.4.1 Comprehensions

Comprehensions are a concise way to create lists, sets, or dictionaries from iterables by applying an expression to each item in an iterable (such as a list, tuple, or range) and optionally filtering the items based on a condition. They are a powerful and efficient way to generate new collections without the need for explicit loops.

**Basic Syntax:**

```
new_list = [expression for item in iterable if condition]
```

- **expression:** What you want to include in the new list (or set/dict).
- **item:** Represents each element in the iterable as the comprehension iterates through it.
- **iterable:** The source of elements (list, tuple, range, etc.).
- **condition (optional):** A filter to control which items are included. If omitted, all items are included.

**Why use comprehensions?** - More readable and concise than loops. - Often faster than equivalent for-loops. - Preferred for simple transformations and filtering.

### List comprehension example:

Create a list that has squares of natural numbers from 5 to 15.

```
sqrt_natural_no_5_15 = [(x**2) for x in range(5,16)]  
print(sqrt_natural_no_5_15)
```

```
[25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225]
```

Create a list of tuples, where each tuple consists of a natural number and its square, for natural numbers ranging from 5 to 15.

```
sqrt_natural_no_5_15 = [(x,x**2) for x in range(5,16)]  
print(sqrt_natural_no_5_15)
```

```
[(5, 25), (6, 36), (7, 49), (8, 64), (9, 81), (10, 100), (11, 121), (12, 144), (13, 169), (14, 196)]
```

Creating a list of words that start with the letter 'a' in a given list of words.

```
words = ['apple', 'banana', 'avocado', 'grape', 'apricot']  
a_words = [word for word in words if word.startswith('a')]  
print(a_words)
```

```
['apple', 'avocado', 'apricot']
```

### Set comprehension:

```
unique_lengths = {len(word) for word in ['cat', 'dog', 'mouse']}  
uniq_initials = {name[0].upper() for name in ["amy","ann","bob","amy"]}  
print(uniq_initials)  
print(unique_lengths)
```

```
{'B', 'A'}  
{3, 5}
```

### Dictionary comprehension:

```
word_lengths = {word: len(word) for word in ['cat', 'dog', 'mouse']}
print(word_lengths)
```

```
{'cat': 3, 'dog': 3, 'mouse': 5}
```

Comprehensions are preferred for simple transformations and filtering.

### 5.4.2 Membership & Lookups

Membership operations let you check if an item exists in a collection (like a list, set, or dictionary). Use `in` and `not in` for these checks. Lookups retrieve values by key or index, and are fastest in sets and dictionaries.

#### Examples:

- `3 in [1, 2, 3] → True` (checks if 3 is in the list)
- `'a' in {'a': 1, 'b': 2} → True` (checks if 'a' is a key in the dictionary)
- `5 not in {1, 2, 3} → True` (checks if 5 is not in the set)

#### Tip:

- Dictionary and set lookups are very fast (constant time).
- List and tuple membership checks are slower (linear time).
- For safe dictionary lookups, use `.get(key)` to avoid errors if the key is missing.

### 5.4.3 Unpacking

Unpacking lets you assign elements of a collection (like a list, tuple, or string) to multiple variables in a single step. This makes your code more readable and concise, especially when working with structured data.

#### Basic Syntax:

```
pt = (3, 4)
x, y = pt # x=3, y=4

a, b, c = [1, 2, 3] # a=1, b=2, c=3
```

You can also use unpacking in loops and with function arguments.

**Extended Unpacking:** Python allows you to use the `*` operator to capture multiple elements:



```
# Extended unpacking:
first, *rest = [10, 20, 30, 40] # first=10, rest=[20, 30, 40]

a, *mid, z = [1,2,3,4]

print(rest)
print(mid)
```

```
[20, 30, 40]
[2, 3]
```

### Unpacking in Loops:

```
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
for num, char in pairs:
    print(num, char)
```

```
1 a
2 b
3 c
```

### Unpacking with Dictionaries:

```
student = {"name": "Alex", "year": 3}
for key, value in student.items():
    print(key, value)
```

```
name Alex
year 3
```

### Why use unpacking?

- Makes code cleaner and more readable
- Quick variable assignment
- Useful for working with structured data (e.g., tuples, lists, dicts)
- Avoids manual indexing

**Tip:** Unpacking works with any iterable, including lists, tuples, strings, and even the results of functions that return multiple values.

## 5.4.4 Sorting in Python Iterables

Sorting is a common task when working with data. Python provides flexible ways to order lists, tuples, and other iterables.

### 5.4.4.1 `sorted()` (built-in function)

- Returns a **new sorted list** from any iterable.
- Works with lists, tuples, strings, sets, and more.
- Does **not modify** the original iterable.

```
nums = [3, 1, 4, 1, 5]
print(sorted(nums))      # [1, 1, 3, 4, 5]
print(nums)              # [3, 1, 4, 1, 5] (unchanged)

word = "python"
print(sorted(word))      # ['h', 'n', 'o', 'p', 't', 'y']
```

```
[1, 1, 3, 4, 5]
[3, 1, 4, 1, 5]
['h', 'n', 'o', 'p', 't', 'y']
```

### 5.4.4.2 `.sort()` (list method)

- **In-place sort:** modifies the list itself.
- Only available for lists (not other iterables).
- Returns `None`.

```
nums = [3, 1, 4, 1, 5]
nums.sort()
print(nums)      # [1, 1, 3, 4, 5]
```

```
[1, 1, 3, 4, 5]
```

#### 5.4.4.3 Reverse Sorting

Both `sorted()` and `.sort()` accept a `reverse` argument.

```
nums = [3, 1, 4, 1, 5]
print(sorted(nums, reverse=True))
```

```
[5, 4, 3, 1, 1]
```

#### 5.4.4.4 Sorting with a key

The `key` parameter lets you customize sorting logic.

```
# Sort by string length
words = ["pear", "apple", "banana", "kiwi"]
print(sorted(words, key=len))
# ['kiwi', 'pear', 'apple', 'banana']

# Sort by last character
print(sorted(words, key=lambda w: w[-1]))
# ['banana', 'pear', 'apple', 'kiwi']
```

```
['pear', 'kiwi', 'apple', 'banana']
['banana', 'apple', 'kiwi', 'pear']
```

#### 5.4.4.5 Sorting Complex Data

For lists of tuples or dicts, use `key`.

```
# Sort by the second element of each tuple
pairs = [("a", 3), ("b", 1), ("c", 2)]
print(sorted(pairs, key=lambda t: t[1]))
# [('b', 1), ('c', 2), ('a', 3)]

# Sort list of dicts by a field
students = [
    {"name": "Alice", "grade": 85},
    {"name": "Bob", "grade": 92},
    {"name": "Chen", "grade": 78}
]
print(sorted(students, key=lambda s: s["grade"]))
```

```
[('b', 1), ('c', 2), ('a', 3)]
[{'name': 'Chen', 'grade': 78}, {'name': 'Alice', 'grade': 85}, {'name': 'Bob', 'grade': 92}]
```

Sorting strings alphabetically vs. numerically:

```
nums = ["10", "2", "1"]
print(sorted(nums))
print(sorted(nums, key=int))
```

```
['1', '10', '2']
['1', '2', '10']
```

### 5.4.5 Lambda Functions in Python

Sometimes you only need a **tiny one-off function** for a specific task (like sorting by length or filtering items). Writing a full `def` feels heavy.

That's where **lambda functions** help.

#### Syntax

```
lambda parameters: expression
```

- Creates an anonymous function (no name required).
- Must contain a single expression (not multiple statements).
- Automatically returns the value of the expression.

```
square = lambda x: x**2
print(square(5))    # 25
```

25

Using Lambda Functions

- With `sorted()`

```
words = ["pear", "apple", "banana", "kiwi"]

# Sort by word length
print(sorted(words, key=lambda w: len(w)))

# Sort by last character
print(sorted(words, key=lambda w: w[-1]))
```

```
['pear', 'kiwi', 'apple', 'banana']  
['banana', 'apple', 'kiwi', 'pear']
```

- With `map()` and `filter()`

```
nums = [1, 2, 3, 4, 5]  
  
# Square each number  
print(list(map(lambda x: x**2, nums)))  
  
# Keep only even numbers  
print(list(filter(lambda x: x % 2 == 0, nums)))
```

```
[1, 4, 9, 16, 25]  
[2, 4]
```

- With `reduce()` (from `functools`)

```
from functools import reduce  
  
nums = [1, 2, 3, 4, 5]  
product = reduce(lambda x, y: x * y, nums)  
print(product)    # 120
```

120

### Key Takeaways:

- `lambda` = quick, throwaway function for one-liners
- use `def` if the function
  - has multiple steps
  - is reused often

#### 5.4.6 `enumerate()`

The `enumerate()` function is a built-in Python tool that adds a counter to any iterable, returning pairs of (index, item) as you loop. This is especially useful when you need both the item and its position in a loop.

#### Syntax:

```
for index, value in enumerate(iterable, start=0):  
    # use index and value
```

- **iterable**: Any sequence (list, tuple, string, etc.)
- **start**: Optional, sets the starting index (default is 0)

#### Why use `enumerate()`?

- Makes code cleaner and less error-prone
- Avoids manual index tracking with a separate variable
- Works with lists, tuples, strings, and more

#### Example:

```
fruits = ['apple', 'banana', 'cherry']  
for idx, fruit in enumerate(fruits):  
    print(idx, fruit)
```

```
0 apple  
1 banana  
2 cherry
```

**Tip:** You can set the starting index with the `start` argument, e.g. `enumerate(my_list, start=1)`.

Use `enumerate()` for readable, efficient loops when you need both index and value.

### 5.4.7 `zip()`

The `zip()` function combines two or more iterables (like lists, tuples, or strings) into tuples, pairing elements by their position. This is useful for parallel iteration, creating pairs, or merging data from multiple sources.

#### Syntax:

```
zip(iterable1, iterable2, ...)
```

- Each tuple contains one element from each iterable, matched by position.
- Stops at the shortest iterable.

**Why use zip()? -** Parallel iteration over multiple sequences - Pairing related data (e.g., names and scores) - Creating dictionaries from two lists

**Example:**

```
names = ['Alice', 'Bob', 'Chen']
scores = [85, 92, 78]
for name, score in zip(names, scores):
    print(name, score)
```

```
Alice 85
Bob 92
Chen 78
```

```
# Creating a dictionary from two lists:
gradebook = dict(zip(names, scores))
print(gradebook)
```

```
{'Alice': 85, 'Bob': 92, 'Chen': 78}
```

**Tip:** - You can use `zip(*zipped)` to unzip a list of tuples back into separate lists. - If the input iterables are different lengths, `zip()` stops at the shortest one.

### 5.4.8 Common Functions for Iterables

Python provides a variety of built-in functions to operate on iterables, making it easy to manipulate, process, and analyze collections like lists, tuples, strings, sets, and dictionaries. Below is a list of commonly used built-in functions specifically designed for iterables.

| Function                | Description                                                                   | Example                                            |
|-------------------------|-------------------------------------------------------------------------------|----------------------------------------------------|
| <code>len()</code>      | Returns the number of elements in an iterable.                                | <code>len([1, 2, 3]) → 3</code>                    |
| <code>min()</code>      | Returns the smallest element in an iterable.                                  | <code>min([3, 1, 4]) → 1</code>                    |
| <code>max()</code>      | Returns the largest element in an iterable.                                   | <code>max([3, 1, 4]) → 4</code>                    |
| <code>sum()</code>      | Returns the sum of elements in an iterable (numeric types only).              | <code>sum([1, 2, 3]) → 6</code>                    |
| <code>sorted()</code>   | Returns a sorted list from an iterable (does <b>not</b> modify the original). | <code>sorted([3, 1, 2]) → [1, 2, 3]</code>         |
| <code>reversed()</code> | Returns an iterator that accesses the elements of an iterable in reverse.     | <code>list(reversed([1, 2, 3])) → [3, 2, 1]</code> |

| Function                        | Description                                                                                                        | Example                                                                        |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>enumerate()</code>        | Returns an iterator of tuples containing indices and elements of the iterable.                                     | <code>list(enumerate(['a', 'b', 'c'])) → [(0, 'a'), (1, 'b'), (2, 'c')]</code> |
| <code>all()</code>              | Returns <b>True</b> if all elements of the iterable are true (or if empty).                                        | <code>all([True, 1, 'a']) → True</code>                                        |
| <code>any()</code>              | Returns <b>True</b> if <b>any</b> element of the iterable is true.                                                 | <code>any([False, 0, 'b']) → True</code>                                       |
| <code>str.join(iterable)</code> | Joins the elements of an iterable (e.g., list, tuple) into a single string, using the given string as a separator. | <code>''.join(['a', 'b', 'c']) → 'abc'</code>                                  |

## 5.5 Independent Study

### 5.5.1 Sorting the data

- Sort [10, 2, 33, 25, 7] in descending order.
- Given words = ["data", "python", "AI", "science"], sort alphabetically ignoring case.
- Sort [("Ann", 22), ("Bob", 19), ("Chen", 22)] by age, preserving name order when ages match.

### 5.5.2 lambda

- Use lambda with `sorted()` to order the list:

```
nums = [-3, 1, -2, 5, 0]
```

- Use lambda with `filter()` to keep only names starting with a vowel:

```
names = ["Alice", "Bob", "Eve", "Uma", "Sam"]
```

- Use lambda with `map()` to convert Celsius to Fahrenheit:

```
temps_c = [0, 20, 37, 100]
```



### 5.5.3

You are given a list of strings representing student names. Some students appear more than once because they signed up for multiple activities.

Write a Python function that:

1. **Removes duplicate names** while *preserving the first occurrence order*.
2. **Returns a list of tuples** (index, name\_length) for each **unique name**, where:
  - index is the original position of the name in the list (first occurrence).
  - name\_length is the length of the name.

Finally, sort the output list by name\_length in descending order.

```
names = ["Alice", "Bob", "Alice", "Charlie", "Bob", "Dave"]
```

### 5.5.4 Student data

```
# CELL 1: Student Data Setup
students = [
    (101, "Alice", "STAT303", 85),
    (102, "Bob", "STAT301", 92),
    (103, "Charlie", "STAT303", 78),
    (104, "Diana", "STAT301", 95),
    (105, "Eve", "STAT303", 88),
    (106, "Frank", "STAT304", 82),
    (107, "Grace", "STAT301", 90),
    (108, "Hank", "STAT304", 87),
    (109, "Ivy", "STAT303", 91),
    (101, "Alice", "STAT301", 88),
    (101, "Alice", "STAT304", 82),
    (101, "Alice", "STAT302", 86),
    (102, "Bob", "STAT303", 79),
    (102, "Bob", "STAT304", 85),
    (102, "Bob", "STAT302", 83),
    (103, "Charlie", "STAT301", 91),
    (103, "Charlie", "STAT304", 84),
    (103, "Charlie", "STAT302", 80),
    (104, "Diana", "STAT303", 89),
```

```
(104, "Diana", "STAT304", 93),
(104, "Diana", "STAT302", 96),
(105, "Eve", "STAT301", 87),
(105, "Eve", "STAT304", 90),
(105, "Eve", "STAT302", 85),
(106, "Frank", "STAT303", 76),
(106, "Frank", "STAT301", 81),
(106, "Frank", "STAT302", 79),
(107, "Grace", "STAT303", 84),
(107, "Grace", "STAT304", 88),
(107, "Grace", "STAT302", 91),
(108, "Hank", "STAT303", 80),
(108, "Hank", "STAT301", 83),
(108, "Hank", "STAT302", 82),
(109, "Ivy", "STAT301", 94),
(109, "Ivy", "STAT304", 89),
(109, "Ivy", "STAT302", 92),
(110, "Jack", "STAT303", 83),
(110, "Jack", "STAT301", 86),
(110, "Jack", "STAT304", 79),
(110, "Jack", "STAT302", 81),
(111, "Karen", "STAT303", 96),
(111, "Karen", "STAT301", 92),
(111, "Karen", "STAT304", 94),
(111, "Karen", "STAT302", 98),
(112, "Leo", "STAT303", 72),
(112, "Leo", "STAT301", 75),
(112, "Leo", "STAT304", 78),
(112, "Leo", "STAT302", 74)
```

]

- What is the data type for this variable
- How many unique students in the dataset
- How many unique courses in the dataset
- what is the grade range?
- What is the average grade?

## **Part III**

# **Core library foundations**

## 6 Reading Data

<IPython.core.display.Image object>

### 6.1 Types of Data: Structured vs. Unstructured

Understanding the format of your data is crucial before you can analyze it effectively. Data exists in two primary formats:

#### 6.1.1 Structured Data

**Structured data** is organized in a tabular format with clearly defined relationships between data elements. Key characteristics include:

- **Rows** represent individual **observations** (records or data points)
- **Columns** represent **variables** (attributes or features)
- Data follows a consistent schema or format
- Easy to search, query, and analyze using standard tools

**Example:** The dataset below contains movie information where each row represents one movie (observation) and each column contains specific attributes like title, rating, or budget (variables). Since all data in a row relates to the same movie, this is also called **relational data**.

|   | Title                           | US Gross | Production Budget | Release Date | Major Genre | Creative Type | Rotten Tomatoes R |
|---|---------------------------------|----------|-------------------|--------------|-------------|---------------|-------------------|
| 0 | The Shawshank Redemption        |          | 28241469          | 25000000     |             | Sep 23 1994   | Drama             |
| 1 | Inception                       |          | 285630280         | 160000000    |             | Jul 16 2010   | Horror/Thriller   |
| 2 | One Flew Over the Cuckoo's Nest |          | 108981275         | 4400000      |             | Nov 19 1975   | Comedy            |
| 3 | The Dark Knight                 |          | 533345358         | 185000000    |             | Jul 18 2008   | Action/Adventure  |
| 4 | Schindler's List                |          | 96067179          | 25000000     |             | Dec 15 1993   | Drama             |

*Common formats:* CSV files, Excel spreadsheets, SQL databases

### 6.1.2 Unstructured Data

**Unstructured data** lacks a predefined organizational structure, making it more challenging to analyze with traditional methods.

*Examples include:* Text documents, images, audio files, videos, social media posts, sensor data

While harder to analyze directly, unstructured data can often be converted into structured formats for analysis (e.g., extracting features from images or sentiment scores from text).

**In this sequence, we will focus on analyzing structured data.**

## 6.2 Introduction to Pandas

**Pandas** is Python's premier library for data manipulation and analysis. Think of it as Excel for Python - it provides powerful tools for working with tabular data (like spreadsheets) and supports reading from various file formats:

- **CSV** (Comma-Separated Values)
- **Excel** spreadsheets (.xlsx, .xls)
- **JSON** (JavaScript Object Notation)
- **HTML** tables
- **SQL** databases
- And many more!

## 6.3 Reading CSV Files with Pandas

### 6.3.1 Understanding CSV Files

**CSV (Comma-Separated Values)** is the most popular format for storing structured data. Here's what makes CSV files special:

- **Simple structure:** Each row represents one record/observation
- **Comma delimiter:** Values within a row are separated by commas
- **Plain text:** Human-readable and lightweight
- **Universal compatibility:** Supported by virtually all data tools

**Pro Tip:** When you open a CSV file in Excel, the commas are automatically converted to column separators, so you won't see them visually!

### 6.3.2 CSV File Structure Example

Here's a sample CSV file containing daily COVID-19 data for Italy:

```
date,new_cases,new_deaths,new_tests
2020-04-21,2256.0,454.0,28095.0
2020-04-22,2729.0,534.0,44248.0
2020-04-23,3370.0,437.0,37083.0
2020-04-24,2646.0,464.0,95273.0
2020-04-25,3021.0,420.0,38676.0
2020-04-26,2357.0,415.0,24113.0
2020-04-27,2324.0,260.0,26678.0
2020-04-28,1739.0,333.0,37554.0
...
```

**Structure breakdown:** - **Header row:** Column names (date, new\_cases, new\_deaths, new\_tests) - **Data rows:** Each row contains values for one day - **Delimiter:** Commas separate each value within a row

### 6.3.3 Getting Started: Importing Pandas

Before we can work with data, we need to import the pandas library. By convention, pandas is imported with the alias `pd` - this is a universal practice that makes your code more readable and concise.

**Why use `pd`?**

- **Shorter syntax:** `pd.read_csv()` instead of `pandas.read_csv()`
- **Industry standard:** Everyone in the data science community uses this alias
- **Cleaner code:** Makes your scripts more readable

```
import pandas as pd
import os
```

### 6.3.4 Loading Data with `pd.read_csv()`

**What is `pd.read_csv()`?**

The `pd.read_csv()` function is your gateway to loading CSV data into Python. It converts CSV files into a pandas **DataFrame** - think of it as a powerful, interactive spreadsheet that you can manipulate with code.

**Key Features:**

- **Automatic data type detection:** Pandas intelligently guesses column types (numbers, text, dates)
- **Header recognition:** Automatically uses the first row as column names
- **Missing data handling:** Gracefully handles empty cells
- **Flexible parsing:** Can handle various CSV formats and delimiters

```
movie_ratings = pd.read_csv('./datasets/movie_ratings.csv')
movie_ratings.head()
```

|   | Title              | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source   | Major |
|---|--------------------|----------|-----------------|-------------------|--------------|-------------|----------|-------|
| 0 | Opal Dreams        |          | 14443           | 14443             | 9000000      | Nov 22 2006 | PG/PG-13 |       |
| 1 | Major Dundee       |          | 14873           | 14873             | 3800000      | Apr 07 1965 | PG/PG-13 |       |
| 2 | The Informers      |          | 315000          | 315000            | 18000000     | Apr 24 2009 | R        |       |
| 3 | Buffalo Soldiers   |          | 353743          | 353743            | 15000000     | Jul 25 2003 | R        |       |
| 4 | The Last Sin Eater |          | 388390          | 388390            | 2200000      | Feb 09 2007 | PG/PG-13 |       |

### Understanding the DataFrame Display:

When you display a DataFrame, you'll notice several key components:

1. **The Index Column (leftmost):** The first column shows row indices (0, 1, 2, 3...). When you create a DataFrame without specifying an index, pandas automatically assigns a **RangeIndex**, which is a sequence of integers starting from 0.
2. **Column Headers:** The top row shows column names from your CSV file
3. **Data Cells:** The intersection of rows and columns contains your actual data
4. **Data Types:** Pandas automatically detects whether columns contain numbers, text, dates, etc.

**Key Point:** The index is **not** part of your original data - it's added by pandas to help identify and access rows efficiently.

```
# Examine the DataFrame index in detail
print(" DataFrame Index Information:")
print(f"Index type: {type(movie_ratings.index)}")
print(f"Index values: {movie_ratings.index}")
print(f"Index range: {movie_ratings.index[0]} to {movie_ratings.index[-1]}")
print(f"Total rows: {len(movie_ratings.index)}")

# Show the index as a list for first 10 rows
print(f"\nFirst 10 index values: {movie_ratings.index[:10].tolist()}")
```

```
DataFrame Index Information:
Index type: <class 'pandas.core.indexes.range.RangeIndex'>
Index values: RangeIndex(start=0, stop=2228, step=1)
Index range: 0 to 2227
Total rows: 2228
```

```
First 10 index values: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

### 6.3.5 Indexing

Alternatively, we can use the `index_col` parameter in the `pd.read_csv()` function to set a specific column as the `index` when reading a CSV file in pandas. This allows us to directly specify which column should be used as the index of the resulting DataFrame.

```
movie_ratings = pd.read_csv('./datasets/movie_ratings.csv', index_col='Title')
movie_ratings.head()
```

| Title              | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source      | Major G  |     |
|--------------------|----------|-----------------|-------------------|--------------|-------------|-------------|----------|-----|
| Opal Dreams        |          | 14443           | 14443             |              | 9000000     | Nov 22 2006 | PG/PG-13 | Ada |
| Major Dundee       |          | 14873           | 14873             |              | 3800000     | Apr 07 1965 | PG/PG-13 | Ada |
| The Informers      |          | 315000          | 315000            |              | 18000000    | Apr 24 2009 | R        | Ada |
| Buffalo Soldiers   |          | 353743          | 353743            |              | 15000000    | Jul 25 2003 | R        | Ada |
| The Last Sin Eater |          | 388390          | 388390            |              | 2200000     | Feb 09 2007 | PG/PG-13 | Ada |

Let's check out the index again

```
movie_ratings.index
```

```
Index(['Opal Dreams', 'Major Dundee', 'The Informers', 'Buffalo Soldiers',
      'The Last Sin Eater', 'The City of Your Final Destination', 'The Claim',
      'Texas Rangers', 'Ride With the Devil', 'Karakter',
      ...,
      'A Cinderella Story', 'D-War', 'Sinbad: Legend of the Seven Seas',
      'Hoodwinked', 'First Knight', 'King Arthur', 'Mulan', 'Robin Hood',
      'Robin Hood: Prince of Thieves', 'Spiceworld'],
      dtype='object', name='Title', length=2228)
```



### 6.3.5.1 Indexing with `set_index()` and `reset_index()`

- Use `set_index()` to assign an existing column as the DataFrame's index.
- Use `reset_index()` to revert back to the default integer index.

```
movie_ratings.reset_index(inplace=True)
```

```
movie_ratings.index
```

```
RangeIndex(start=0, stop=2228, step=1)
```

```
movie_ratings.set_index("Title").head()
```

| Title              | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source | Major G |
|--------------------|----------|-----------------|-------------------|--------------|-------------|--------|---------|
| Opal Dreams        | 14443    | 14443           | 9000000           | Nov 22 2006  | PG/PG-13    | Ada    |         |
| Major Dundee       | 14873    | 14873           | 3800000           | Apr 07 1965  | PG/PG-13    | Ada    |         |
| The Informers      | 315000   | 315000          | 18000000          | Apr 24 2009  | R           | Ada    |         |
| Buffalo Soldiers   | 353743   | 353743          | 15000000          | Jul 25 2003  | R           | Ada    |         |
| The Last Sin Eater | 388390   | 388390          | 2200000           | Feb 09 2007  | PG/PG-13    | Ada    |         |

```
movie_ratings.index
```

```
RangeIndex(start=0, stop=2228, step=1)
```

- The `index` can act like a primary key.
- It controls how `.loc[]`, slicing, and filtering behave.
- Choosing a good index (e.g., IDs, timestamps) makes subsetting much easier and more readable.

The built-in python function `type` can be used to check the datatype of an object:

```
type(movie_ratings)
```

```
pandas.core.frame.DataFrame
```

**The Result: A DataFrame**

### 6.3.6 Understanding Pandas Data Structures

Before diving deeper into data manipulation, it's crucial to understand the **two fundamental data structures** that pandas provides. Think of these as the building blocks for all data science operations.

### 6.3.7 Series - Your First Pandas Data Structure

A `pandas.Series` is like a **smart column** of data - imagine a single column from an Excel spreadsheet, but with superpowers!

**Definition:** A Series is a **one-dimensional array** of data with an associated **index** (labels for each data point).

#### Key Characteristics:

| Feature                           | Description                                                                          | Example                                                |
|-----------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>One-dimensional Indexed</b>    | Holds data in a single column/row<br>Each element has a label/index                  | [85, 92, 78, 95]<br>['Alice', 'Bob', 'Carol', 'David'] |
| <b>Homogeneous Size immutable</b> | Ideally same data type (but flexible)<br>Fixed size once created (values can change) | All numbers or all text<br>4 elements stay 4 elements  |

**\*\* Think of it as\*\*:** A single column from your DataFrame with smart labeling capabilities.

```
# Creating a Series from the 'Title' column
print(" Extracting a Series from our DataFrame:")
print("=" * 50)

movie_titles = movie_ratings['Title']
print(" Movie Titles Series (first 10):")
print(movie_titles.head(10))
print()

# Let's explore what we just created
print(" Series Details:")
print(f" Type: {type(movie_titles)}")
print(f" Shape: {movie_titles.shape}")
print(f" Size: {movie_titles.size}")
print(f" Data Type: {movie_titles.dtype}")
print(f" Name: {movie_titles.name}")
```

Extracting a Series from our DataFrame:

=====

Movie Titles Series (first 10):

```
0          Opal Dreams
1        Major Dundee
2        The Informers
3    Buffalo Soldiers
4    The Last Sin Eater
5    The City of Your Final Destination
6          The Claim
7        Texas Rangers
8    Ride With the Devil
9          Karakter
```

Name: Title, dtype: object

Series Details:

Type: <class 'pandas.core.series.Series'>

Shape: (2228,)

Size: 2228

Data Type: object

Name: Title

### 6.3.7.1 DataFrame: Two-Dimensional Data

A **DataFrame** is the star of pandas - it's like a complete spreadsheet or database table:

**Key Characteristics:**

- **Two-dimensional:** Organized in rows and columns
- **Heterogeneous:** Different columns can have different data types
- **Labeled axes:** Both rows (index) and columns have names/labels
- **Flexible:** Easy to filter, sort, group, and analyze

**Structure:**

- **Rows (Index):** Each row represents one **observation** (e.g., one movie, customer, transaction)
- **Columns:** Each column represents one **variable/feature** (e.g., title, price, rating, date)
- **Cells:** Individual data points where rows and columns intersect

**Default Behavior:**

- **Row indices:** Automatically numbered 0, 1, 2, 3... (but customizable)
- **Column names:** Taken from the first row of your CSV file (the header)

- **Data types:** Automatically detected (numbers, text, dates, etc.)

## 6.4 DataFrame Exploration: Attributes and Methods

Now that you understand what DataFrames and Series are, let's learn how to **explore and understand** your data. Think of these tools as your data detective kit!

### 6.4.1 Attributes vs Methods: What's the Difference?

| Type              | How to Use                                  | What It Does     | Example                |
|-------------------|---------------------------------------------|------------------|------------------------|
| <b>Attributes</b> | <code>df.attribute</code> (no parentheses)  | Shows properties | <code>df.shape</code>  |
| <b>Methods</b>    | <code>df.method()</code> (with parentheses) | Performs actions | <code>df.head()</code> |

**Pro Tip:** Use `dir(your_dataframe)` to see all available attributes and methods!

### 6.4.2 Essential DataFrame Attributes

These attributes help you quickly understand your dataset's structure and characteristics.

#### 6.4.2.1 Data Types with `dtypes`

**What it shows:** The data type of each column

**Why it matters:** Understanding data types helps you: - Identify if numbers are stored as text (and need conversion) - Know which operations work with each column

- Optimize memory usage and performance
- Catch data quality issues early

```
# Examine data types of all columns
print(" Data Types in Our Movie Dataset:")
print("=" * 40)
print(movie_ratings.dtypes)
print()

# Count how many columns of each type we have
print(" Data Type Summary:")
dtype_counts = movie_ratings.dtypes.value_counts()
```

```
for dtype, count in dtype_counts.items():
    print(f" {dtype}: {count} columns")
```

```
Title                object
US Gross             int64
Worldwide Gross      int64
Production Budget    int64
Release Date         object
MPAA Rating          object
Source              object
Major Genre          object
Creative Type        object
IMDB Rating          float64
IMDB Votes           int64
dtype: object
```

#### 6.4.2.2 Understanding Pandas Data Types

| Pandas Type       | Python Equivalent | Real-World Example                   | When You'll See It             |
|-------------------|-------------------|--------------------------------------|--------------------------------|
| <b>object</b>     | string/mixed      | Movie titles, categories, mixed data | Text columns, categorical data |
| <b>int64</b>      | int               | Year (2023), count (50 movies)       | Whole numbers, counts, IDs     |
| <b>float64</b>    | float             | Rating (7.5), budget (\$50.5M)       | Decimal numbers, measurements  |
| <b>bool</b>       | True/False        | Is_Popular, Has_Sequel               | Yes/no questions               |
| <b>datetime64</b> | datetime          | Release dates, timestamps            | Dates and times                |

**Common Data Type Issues:** - **Numbers stored as 'object':** Often means there are non-numeric characters (like '\$' or ',') - **Dates as 'object':** Need to convert using `pd.to_datetime()` - **Missing values in integers:** Forces conversion to float64

#### 6.4.2.3 Dataset Dimensions with [shape](#)

**What it shows:** Returns dimensions as (rows, columns)

**Why it matters:**

- Understand your dataset size at a glance
- Know if you have a “tall” (many rows) or “wide” (many columns) dataset
- Estimate memory requirements
- Spot issues (e.g., fewer rows than expected after filtering)

```
# Check the dimensions of our movie dataset
print(f"Dataset shape: {movie_ratings.shape}")
print(f"Number of rows (movies): {movie_ratings.shape[0]}")
print(f"Number of columns (features): {movie_ratings.shape[1]}")
```

```
Dataset shape: (2228, 11)
Number of rows (movies): 2228
Number of columns (features): 11
```

#### 6.4.2.3.1 columns

**Purpose:** Returns the column names as an Index object

**Why useful:** See what variables/features are available in your dataset

```
# View all column names
print("Available columns:")
for i, col in enumerate(movie_ratings.columns, 1):
    print(f"{i}. {col}")
```

```
Available columns:
1. Title
2. US Gross
3. Worldwide Gross
4. Production Budget
5. Release Date
6. MPAA Rating
7. Source
8. Major Genre
9. Creative Type
10. IMDB Rating
11. IMDB Votes
```

#### 6.4.2.3.2 index

**Purpose:** Returns the row labels (index) of the DataFrame

**Why useful:** Understand how rows are labeled and indexed

```
# Examine the index
print(f"Index type: {type(movie_ratings.index)}")
print(f"Index range: {movie_ratings.index[0]} to {movie_ratings.index[-1]}")
print(f"First 10 index values: {movie_ratings.index[:10].tolist()}")
```

Index type: <class 'pandas.core.indexes.range.RangeIndex'>

Index range: 0 to 2227

First 10 index values: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

#### 6.4.2.4 Essential DataFrame Methods

##### 6.4.2.4.1 info()

**Purpose:** Provides a comprehensive summary of the DataFrame

**Returns:** Information about data types, non-null counts, and memory usage

```
# Get comprehensive dataset information
movie_ratings.info()
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2228 entries, 0 to 2227

Data columns (total 11 columns):

| #  | Column            | Non-Null Count | Dtype   |
|----|-------------------|----------------|---------|
| 0  | Title             | 2228 non-null  | object  |
| 1  | US Gross          | 2228 non-null  | int64   |
| 2  | Worldwide Gross   | 2228 non-null  | int64   |
| 3  | Production Budget | 2228 non-null  | int64   |
| 4  | Release Date      | 2228 non-null  | object  |
| 5  | MPAA Rating       | 2228 non-null  | object  |
| 6  | Source            | 2228 non-null  | object  |
| 7  | Major Genre       | 2228 non-null  | object  |
| 8  | Creative Type     | 2228 non-null  | object  |
| 9  | IMDB Rating       | 2228 non-null  | float64 |
| 10 | IMDB Votes        | 2228 non-null  | int64   |

dtypes: float64(1), int64(4), object(6)

memory usage: 191.6+ KB

What `info()` tells you:

- **Data types** for each column
- **Non-null count** (helps identify missing data)
- **Memory usage** (useful for large datasets)
- **Total entries** and columns

#### 6.4.2.4.2 `describe()`

**Purpose:** Generate descriptive statistics for numerical columns

**Returns:** Count, mean, std, min, 25%, 50%, 75%, max for each numeric column

```
# Get statistical summary of numerical columns
movie_ratings.describe()
```

|       | US Gross     | Worldwide Gross | Production Budget | IMDB Rating | IMDB Votes    |
|-------|--------------|-----------------|-------------------|-------------|---------------|
| count | 2.228000e+03 | 2.228000e+03    | 2.228000e+03      | 2228.000000 | 2228.000000   |
| mean  | 5.076370e+07 | 1.019370e+08    | 3.816055e+07      | 6.239004    | 33585.154847  |
| std   | 6.643081e+07 | 1.648589e+08    | 3.782604e+07      | 1.243285    | 47325.651561  |
| min   | 0.000000e+00 | 8.840000e+02    | 2.180000e+02      | 1.400000    | 18.000000     |
| 25%   | 9.646188e+06 | 1.320737e+07    | 1.200000e+07      | 5.500000    | 6659.250000   |
| 50%   | 2.838649e+07 | 4.266892e+07    | 2.600000e+07      | 6.400000    | 18169.000000  |
| 75%   | 6.453140e+07 | 1.200000e+08    | 5.300000e+07      | 7.100000    | 40092.750000  |
| max   | 7.601676e+08 | 2.767891e+09    | 3.000000e+08      | 9.200000    | 519541.000000 |

#### Key Statistics Explained:

- **count:** Number of non-missing values
- **mean:** Average value
- **std:** Standard deviation (measure of spread)
- **min/max:** Smallest and largest values
- **25%, 50%, 75%:** Quartiles (percentiles)

**Tip:** Use `describe(include='all')` to include non-numeric columns in the summary!



#### 6.4.2.4.3 `head()` and `tail()`

**Purpose:** View the first or last few rows of your dataset

**Default:** Shows 5 rows, but you can specify any number

```
# View first 3 rows
print("First 3 movies:")
movie_ratings.head(3)
```

First 3 movies:

|   | Title         | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source | Major |
|---|---------------|----------|-----------------|-------------------|--------------|-------------|--------|-------|
| 0 | Opal Dreams   | 14443    | 14443           | 9000000           | Nov 22 2006  | PG/PG-13    |        | Adap  |
| 1 | Major Dundee  | 14873    | 14873           | 3800000           | Apr 07 1965  | PG/PG-13    |        | Adap  |
| 2 | The Informers | 315000   | 315000          | 18000000          | Apr 24 2009  | R           |        | Adap  |

```
# View last 3 rows
print("Last 3 movies:")
movie_ratings.tail(3)
```

Last 3 movies:

|      | Title                         | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source      | Major |
|------|-------------------------------|----------|-----------------|-------------------|--------------|-------------|-------------|-------|
| 2225 | Robin Hood                    |          |                 | 105269730         | 310885538    | 210000000   | May 14 2010 | I     |
| 2226 | Robin Hood: Prince of Thieves |          |                 | 165493908         | 390500000    | 50000000    | Jun 14 1991 | I     |
| 2227 | Spiceworld                    |          |                 | 29342592          | 56042592     | 25000000    | Jan 23 1998 | I     |

#### 6.4.2.4.4 `sample()`

**Purpose:** Get a random sample of rows from your DataFrame **Why useful:** Explore data patterns without bias toward top/bottom rows

```
# Get 5 random movies from the dataset
print("Random sample of 5 movies:")
movie_ratings.sample(5)
```

### 6.4.2.5 Quick Reference: Most Used Attributes & Methods

| Attribute/Method         | Purpose                     | Example Usage              |
|--------------------------|-----------------------------|----------------------------|
| <code>.shape</code>      | Get dimensions (rows, cols) | <code>df.shape</code>      |
| <code>.dtypes</code>     | Check data types            | <code>df.dtypes</code>     |
| <code>.columns</code>    | View column names           | <code>df.columns</code>    |
| <code>.info()</code>     | Comprehensive overview      | <code>df.info()</code>     |
| <code>.describe()</code> | Statistical summary         | <code>df.describe()</code> |
| <code>.head(n)</code>    | First n rows                | <code>df.head(10)</code>   |
| <code>.tail(n)</code>    | Last n rows                 | <code>df.tail(3)</code>    |
| <code>.sample(n)</code>  | Random n rows               | <code>df.sample(5)</code>  |

## 6.5 Putting It All Together: Understanding Pandas Data Structures

Now that we've explored DataFrames and Series in detail, let's consolidate your understanding with a comprehensive comparison and key takeaways.

### 6.5.1 DataFrame vs Series: The Complete Picture

| Aspect            | DataFrame                                           | Series                                 |
|-------------------|-----------------------------------------------------|----------------------------------------|
| ** Dimensions**   | 2D (rows × columns)                                 | 1D (single column)                     |
| ** Purpose**      | Complete dataset/table                              | Single variable/column                 |
| ** Structure**    | Multiple columns, each can be different type        | Single column of one data type         |
| ** Access**       | <code>df['column']</code> or <code>df.column</code> | Direct indexing <code>series[0]</code> |
| ** Example**      | Entire movie database                               | Just the 'Rating' column               |
| ** Relationship** | Contains multiple Series                            | One column extracted from DataFrame    |

### 6.5.2 Key Concepts You Now Understand

#### 6.5.2.1 DataFrames are Collections of Series

```
# When you do this:
ratings = movie_ratings['Rating'] # You get a Series
titles = movie_ratings['Title']   # You get another Series
```

```
# The DataFrame is made up of these Series working together!
```

### 6.5.2.2 Index: The Hidden Hero

- **Every row has a label** (0, 1, 2, 3... by default)
- **Links related data** across different columns
- **Enables precise data selection** and alignment

### 6.5.2.3 Data Types Matter

- **Numerical columns** (int64, float64) → Mathematical operations
- **Text columns** (object) → String operations, categories
- **Date columns** (datetime64) → Time-based analysis
- **Boolean columns** (bool) → True/False filtering

## 6.5.3 What You Can Do Now

**Load data** from CSV files into DataFrames

**Understand the structure** using `.shape`, `.dtypes`, `.info()`

**Extract specific columns** as Series for focused analysis

**Identify data quality issues** through data type inspection

**Navigate between 1D and 2D data** structures confidently

## 6.6 Writing Data to CSV Files

After cleaning, analyzing, or transforming your data, you'll often need to **save your results** for future use or sharing. Pandas makes this easy with the `to_csv()` method.

### 6.6.1 Basic CSV Export

The simplest way to export a DataFrame is using the `to_csv()` method with just a filename:

```
# Basic export - saves DataFrame to CSV file
movie_ratings.to_csv('./datasets/movie_ratings_exported.csv')
print(" Data exported successfully!")
```

Data exported successfully!

```
# Verify the file was created
if './datasets/movie_ratings_exported.csv' in [f'./datasets/{f}' for f in os.listdir('./datasets')]:
    print(" File successfully exported!")
else:
    print(" Export failed")
```

File successfully exported!

## 6.6.2 Important CSV Export Parameters

By default, pandas includes row indices in the exported CSV. Often you don't want this:

The `to_csv()` method has many useful parameters to customize your output:

### 6.6.2.1 Index Control

```
# Export WITHOUT row indices (most common)
movie_ratings.to_csv('./datasets/movies_no_index.csv', index=False)

# Export WITH row indices (default behavior)
movie_ratings.to_csv('./datasets/movies_with_index.csv', index=True)
```

### 6.6.2.2 Column Selection

Export only specific columns:

```
# Export only selected columns
columns_to_export = ['Title', 'IMDB Rating', 'Worldwide Gross']
movie_ratings[columns_to_export].to_csv('./datasets/movies_selected_columns.csv', index=False)
```

### 6.6.2.3 Custom Separators

Change the delimiter (useful for different regional standards):

```
# Use semicolon instead of comma (common in European locales)
movie_ratings.to_csv('./datasets/movies_semicolon.csv', sep=';', index=False)

# Use tab separator (creates TSV file)
movie_ratings.to_csv('./datasets/movies_tab.txt', sep='\t', index=False)
```

#### 6.6.2.4 Handling Missing Values

Control how missing/null values are represented:

```
# Replace NaN values with custom text
movie_ratings.to_csv('./datasets/movies_custom_na.csv',
                    index=False,
                    na_rep='Missing')
```

#### 6.6.2.5 Encoding Specification

Ensure proper character encoding (important for international data):

```
# Specify UTF-8 encoding (recommended for international characters)
movie_ratings.to_csv('./datasets/movies_utf8.csv',
                    index=False,
                    encoding='utf-8')
```

### 6.6.3 Best Practices for CSV Export

| Practice                                   | Why Important                                      | Example                                             |
|--------------------------------------------|----------------------------------------------------|-----------------------------------------------------|
| <b>Always use <code>index=False</code></b> | Prevents unnecessary row numbers                   | <code>df.to_csv('file.csv', index=False)</code>     |
| <b>Specify encoding</b>                    | Ensures international characters display correctly | <code>encoding='utf-8'</code>                       |
| <b>Use descriptive filenames</b>           | Makes files easy to identify later                 | <code>'cleaned_movie_data_2024.csv'</code>          |
| <b>Check file paths</b>                    | Avoid overwriting important files                  | Use relative or absolute paths carefully            |
| <b>Handle missing data</b>                 | Define how NaN values should appear                | <code>na_rep='N/A'</code> or <code>na_rep=''</code> |

### 6.6.4 Complete Export Example

Here's a comprehensive example combining multiple parameters:

```
# Professional CSV export with all best practices
export_filename = './datasets/movie_analysis_final.csv'

# Select relevant columns for export
columns_for_analysis = [
    'Title', 'IMDB Rating', 'IMDB Votes',
    'US Gross', 'Worldwide Gross', 'Production Budget'
]

# Export with professional settings
movie_ratings[columns_for_analysis].to_csv(
    export_filename,
    index=False,          # No row numbers
    encoding='utf-8',     # International characters
    na_rep='N/A',        # Clear missing value indicator
    float_format='%.2f'   # 2 decimal places for numbers
)

print(f" Analysis data exported to: {export_filename}")
print(f" Exported {len(columns_for_analysis)} columns for {len(movie_ratings)} movies")
```

```
Analysis data exported to: ./datasets/movie_analysis_final.csv
Exported 6 columns for 2228 movies
```

### 6.6.5 Verifying Your Export

Always verify your exported data looks correct:

```
# Read back the exported file to verify
verification_df = pd.read_csv(export_filename)

print(f"\n Exported dimensions: {verification_df.shape}")

print(" Exported file preview:")
verification_df.head(3)
```

Exported dimensions: (2228, 6)

Exported file preview:

|   | Title         | IMDB Rating | IMDB Votes | US Gross | Worldwide Gross | Production Budget |
|---|---------------|-------------|------------|----------|-----------------|-------------------|
| 0 | Opal Dreams   | 6.5         | 468        | 14443    | 14443           | 9000000           |
| 1 | Major Dundee  | 6.7         | 2588       | 14873    | 14873           | 3800000           |
| 2 | The Informers | 5.2         | 7595       | 315000   | 315000          | 18000000          |

## 6.7 Reading Other Data Formats

While **CSV is the most popular format** for structured data, real-world data comes in many different formats. As a data analyst, you'll frequently encounter various file types and data sources that require different reading approaches.

### 6.7.1 Common Data Formats Overview

| Format | Extension   | Use Case                           | Pandas Function                                            |
|--------|-------------|------------------------------------|------------------------------------------------------------|
| CSV    | .csv        | Most common tabular data           | <code>pd.read_csv()</code>                                 |
| Text   | .txt        | Tab-separated or custom delimiters | <code>pd.read_csv()</code> with <code>sep</code> parameter |
| JSON   | .json       | API data, nested structures        | <code>pd.read_json()</code>                                |
| Excel  | .xlsx, .xls | Spreadsheet data                   | <code>pd.read_excel()</code>                               |
| URLs   | Various     | Live data from web APIs            | Various functions                                          |

### 6.7.2 What You'll Learn

In this section, we'll explore how to:

- **Read text files** with different delimiters (tabs, semicolons, etc.)
- **Handle JSON data** from APIs and web services
- **Access live data** directly from URLs and APIs
- **Extract tables** from HTML web pages

**Key Insight:** The same `pd.read_csv()` function can read many text-based formats by adjusting parameters!

### 6.7.3 Reading Text Files (.txt)

Text files offer **maximum flexibility** for storing tabular data with custom delimiters.

#### Key Differences from CSV:

- **CSV files:** Always use commas (,) as delimiters. Let's read a file where values are separated by tab characters:
- **Text files:** Can use **any character** as a delimiter

#### 6.7.3.1 Reading Tab-Separated Files

##### Common Delimiters in Text Files:

- **Space ( )** → Simple space-separated files
- **Tab character (\t)** → Creates TSV (Tab-Separated Values) files- **Pipe symbol (|)** → Used when data contains commas
- **Semicolon (;)** → Common in European data files

```
# Read tab-separated file using \t as separator
# Note: We still use pd.read_csv() but specify the separator
movie_ratings_txt = pd.read_csv('./datasets/movie_ratings.txt', sep='\t')

movie_ratings_txt.head()
```

| Unnamed: 0 |   | Title              | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | S |
|------------|---|--------------------|----------|-----------------|-------------------|--------------|-------------|---|
| 0          | 0 | Opal Dreams        |          | 14443           | 14443             | 9000000      | Nov 22 2006 | P |
| 1          | 1 | Major Dundee       |          | 14873           | 14873             | 3800000      | Apr 07 1965 | P |
| 2          | 2 | The Informers      |          | 315000          | 315000            | 18000000     | Apr 24 2009 | R |
| 3          | 3 | Buffalo Soldiers   |          | 353743          | 353743            | 15000000     | Jul 25 2003 | R |
| 4          | 4 | The Last Sin Eater |          | 388390          | 388390            | 2200000      | Feb 09 2007 | P |

#### Key Points:

- Use the **same** `pd.read_csv()` function for text files
- Specify the delimiter with the **sep** parameter
- `\t` represents the tab character



### 6.7.3.2 Common Separator Examples

- The function automatically handles headers and data types

```
# Different separator examples (these are demonstrations)

# Semicolon-separated (common in Europe)
# df_semicolon = pd.read_csv('file.txt', sep=';')

# Pipe-separated (when data contains commas)
# df_pipe = pd.read_csv('file.txt', sep='|')

# Space-separated
# df_space = pd.read_csv('file.txt', sep=' ')

# Multiple spaces or mixed whitespace
# df_whitespace = pd.read_csv('file.txt', sep='\s+')

print(" Tip: Use sep='\s+' for files with irregular spacing!")
```

Tip: Use sep='\s+' for files with irregular spacing!

### 6.7.3.3 Automatic Delimiter Detection

Pandas can automatically detect delimiters using the Python engine:

```
# Let pandas automatically detect the delimiter
# This uses the 'python' engine with a 'sniffer' tool
try:
    auto_detected = pd.read_csv('./datasets/movie_ratings.txt',
                                sep=None,                    # Auto-detect
                                engine='python')             # Use Python engine
    print(" Delimiter detected automatically!")
    print(f" Shape: {auto_detected.shape}")
except FileNotFoundError:
    print(" File not found - this is just a demonstration")
```

```
Delimiter detected automatically!
Shape: (2228, 12)
```

**When to use automatic detection:**

- When you're unsure about the delimiter
- For quick data exploration
- Avoid in production code (be explicit for reliability)

**Pro Tip:** Always check the [pandas documentation](#) for parameter details!

## 6.7.4 Reading JSON Data

**JSON (JavaScript Object Notation)** is the **standard format for web APIs** and modern data exchange.

**Why JSON is Everywhere:**

- **Web APIs:** Most modern APIs return JSON data
- **Mobile Apps:** Standard format for app-server communication
- **Cloud Services:** AWS, Google Cloud, Azure all use JSON
- **IoT Devices:** Smart devices send data in JSON format

**Key Advantages over CSV:**

| Feature                         | CSV                                             | JSON                                                                 |
|---------------------------------|-------------------------------------------------|----------------------------------------------------------------------|
| <b>Structure</b>                | Flat tables only                                | <b>Nested / hierarchical</b>                                         |
| <b>Data Types</b>               | Text and numbers                                | <b>Objects, arrays, booleans</b>                                     |
| <b>Metadata</b>                 | Must be stored separately or encoded in headers | <b>Supports metadata inline (keys + values in the same document)</b> |
| <b>Complex / Nested Records</b> | Must be flattened into multiple columns         | <b>Native support for nested objects and arrays</b>                  |

### 6.7.4.1 Reading JSON with Pandas

- **Data type inference** (numbers, dates, etc.)

`pd.read_json()` automatically converts JSON data into pandas DataFrames:

- **DataFrame conversion** when possible
- **Smart detection** of JSON structure

**Automatic Processing:**

- **Local files:** `pd.read_json('data.json')`
- **URLs:** `pd.read_json('https://api.example.com/data')`

- **Various orientations:** Records, index, values, etc.
- **JSON strings:** Direct JSON text

#### 6.7.4.2 Real Example: TED Talks Dataset

Let's load a real JSON dataset containing TED Talks information:

```
# Load TED Talks data from GitHub (JSON format)

url = 'https://raw.githubusercontent.com/cwkenwaysun/TEDmap/master/data/TED_Talks.json'
print(" Loading TED Talks data from JSON...")
tedtalks_data = pd.read_json(url)
print(f" Loaded {tedtalks_data.shape[0]} TED Talks with {tedtalks_data.shape[1]} features!")
```

```
Loading TED Talks data from JSON...
Loaded 2474 TED Talks with 16 features!
```

```
# Explore the TED Talks dataset

print(f" Dataset Info:")
tedtalks_data.head(3)

print(f"Shape: {tedtalks_data.shape}")
print("\n First few TED Talks:")
print(f"Columns: {list(tedtalks_data.columns)}")
```

```
Dataset Info:
Shape: (2474, 16)
```

```
First few TED Talks:
Columns: ['id', 'speaker', 'headline', 'URL', 'description', 'transcript_URL', 'month_filmed']
```

#### 6.7.4.3 Reading Data from URLs and APIs

Modern data science often involves accessing live data from web APIs (Application Programming Interfaces). This allows you to get real-time information directly into your analysis pipeline.

Using the `requests` Library

The `requests` library is the standard tool for making HTTP requests in Python. You'll need to install it if not already available:

```
pip install requests
```

**Basic API Request Pattern:** 1. **Send request** to the API URL 2. **Check response** status (success/error) 3. **Parse JSON data** into usable format 4. **Convert to DataFrame** for analysis

Let's demonstrate with the **CoinGecko API**, which provides cryptocurrency market data:

```
import requests

# Define the API endpoint
api_url = 'https://api.coingecko.com/api/v3/coins/markets'

# Add parameters to get USD prices for top 10 cryptocurrencies
params = {
    'vs_currency': 'usd', # Price in US Dollars
    'order': 'market_cap_desc', # Sort by market cap
    'per_page': 10, # Get top 10 coins
    'page': 1, # First page
    'sparkline': False # Don't include price history
}

print(" Requesting cryptocurrency data from CoinGecko API...")

# Send GET request with parameters
response = requests.get(api_url, params=params)

# Check HTTP status codes
print(f" Response Status Code: {response.status_code}")

if response.status_code == 200:
    print(" Request successful!")

    # Parse JSON response
    crypto_data = response.json()

    # Display basic info about the response
    print(f" Received data for {len(crypto_data)} cryptocurrencies")

    # Show raw JSON structure (first item only)
```

```

print(f"\n Sample JSON structure:")
print(f"Keys available: {list(crypto_data[0].keys())}")

else:
    # Handle different error codes
    error_messages = {
        400: "Bad Request - Check your parameters",
        401: "Unauthorized - API key might be required",
        403: "Forbidden - Access denied",
        404: "Not Found - Invalid endpoint",
        429: "Rate Limited - Too many requests",
        500: "Server Error - Try again later"
    }

    error_msg = error_messages.get(response.status_code, "Unknown error")
    print(f" Request failed: {error_msg}")
    crypto_data = None

```

Requesting cryptocurrency data from CoinGecko API...

Response Status Code: 200

Request successful!

Received data for 10 cryptocurrencies

Sample JSON structure:

Keys available: ['id', 'symbol', 'name', 'image', 'current\_price', 'market\_cap', 'market\_cap

```

# Convert JSON data to pandas DataFrame (if successful)
if crypto_data:
    # Method 1: Convert JSON directly to DataFrame
    crypto_df = pd.DataFrame(crypto_data)

    print(f"\n Cryptocurrency Market Data:")
    print(f"Dataset shape: {crypto_df.shape}")

    # Select and display key columns
    key_columns = ['name', 'symbol', 'current_price', 'market_cap', 'price_change_percentage']
    display_df = crypto_df[key_columns].copy()

    # Format for better readability
    display_df['current_price'] = display_df['current_price'].apply(lambda x: f"${x:,.2f}")
    display_df['market_cap'] = display_df['market_cap'].apply(lambda x: f"${x:,.0f}")

```

```

display_df['price_change_percentage_24h'] = display_df['price_change_percentage_24h'].ap

# Rename columns for display
display_df.columns = ['Name', 'Symbol', 'Current Price', 'Market Cap', '24h Change %']

print(f"\n Top 10 Cryptocurrencies by Market Cap:")
display(display_df)

# Method 2: Manual processing for learning
print(f"\n Price Summary:")
for coin in crypto_data[:5]: # Show first 5
    name = coin['name']
    symbol = coin['symbol'].upper()
    price = coin['current_price']
    change_24h = coin['price_change_percentage_24h']

    # Format change with color indicator
    change_indicator = " " if change_24h > 0 else " "
    print(f"{change_indicator} {name} ({symbol}): ${price:,.2f} | 24h: {change_24h:.2f}%")

else:
    print(" No data to process due to API request failure")

```

Cryptocurrency Market Data:  
Dataset shape: (10, 26)

Top 10 Cryptocurrencies by Market Cap:

|   | Name              | Symbol | Current Price | Market Cap          | 24h Change % |
|---|-------------------|--------|---------------|---------------------|--------------|
| 0 | Bitcoin           | btc    | \$114,167.00  | \$2,275,903,872,681 | 0.13%        |
| 1 | Ethereum          | eth    | \$4,133.97    | \$499,142,371,776   | -1.06%       |
| 2 | Tether            | usdt   | \$1.00        | \$174,703,594,573   | -0.04%       |
| 3 | XRP               | xrp    | \$2.86        | \$170,592,210,519   | -1.02%       |
| 4 | BNB               | bnb    | \$1,006.92    | \$140,084,147,935   | -1.12%       |
| 5 | Solana            | sol    | \$208.39      | \$113,247,264,117   | -1.54%       |
| 6 | USDC              | usdc   | \$1.00        | \$73,396,062,741    | 0.00%        |
| 7 | Lido Staked Ether | steth  | \$4,133.28    | \$35,272,520,571    | -0.90%       |
| 8 | Dogecoin          | doge   | \$0.23        | \$34,944,052,528    | -0.81%       |
| 9 | TRON              | trx    | \$0.33        | \$31,555,873,939    | -0.92%       |

#### Price Summary:

Bitcoin (BTC): \$114,167.00 | 24h: 0.13%  
Ethereum (ETH): \$4,133.97 | 24h: -1.06%  
Tether (USDT): \$1.00 | 24h: -0.04%  
XRP (XRP): \$2.86 | 24h: -1.02%  
BNB (BNB): \$1,006.92 | 24h: -1.12%

#### 6.7.4.4 Read data from wikipedia

```
api_url = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)_per_capita'
# Use pandas to read all tables from the webpage
wiki_response = requests.get(api_url)
# Check HTTP status codes
print(f" Response Status Code: {wiki_response.status_code}")
if wiki_response.status_code == 200:
    print(" Request successful!")
else:
    # output error message based on the status code
    error_messages = {
        400: "Bad Request - Check your parameters",
        401: "Unauthorized - API key might be required",
        403: "Forbidden - Access denied",
        404: "Not Found - Invalid endpoint",
        429: "Rate Limited - Too many requests",
        500: "Server Error - Try again later"
    }
    error_msg = error_messages.get(wiki_response.status_code, "Unknown error")
    print(f" Request failed: {error_msg}")
```

Response Status Code: 403

Request failed: Forbidden - Access denied

##### 6.7.4.4.1 Understanding HTTP 403 Errors with Wikipedia

#### Why Wikipedia Returns 403 “Forbidden” Errors:

Wikipedia and many other websites block requests that appear to be automated bots or scrapers. When you make a request without proper headers, the server sees it as suspicious and blocks it with a 403 error.

### Common Causes of 403 Errors:

- **Missing User-Agent:** No browser identification
- **Suspicious headers:** Looks like automated scraping
- **Rate limiting:** Too many requests too quickly
- **IP blocking:** Your IP has been flagged

#### 6.7.4.4.2 Solution: Add Proper Headers

The key is to make your request look like it's coming from a real web browser by adding appropriate headers:

```
headers = {
    'User-Agent': 'My Data Analysis Project (your_email@example.com)'
}
url = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)'

# Pass the headers to read_html
tables = pd.read_html(url, header=0, attrs=headers)
```

Let's define a helper function for this purpose

```
def read_html_tables_from_url(url, match=None, user_agent=None, timeout=10, **kwargs):
    """
    Read HTML tables from a website URL with error handling and customizable options.

    Parameters:
    -----
    url : str
        The website URL to scrape tables from
    match : str, optional
        String or regex pattern to match tables (passed to pd.read_html)
    user_agent : str, optional
        Custom User-Agent header. If None, uses a default Chrome User-Agent
    timeout : int, default 10
        Request timeout in seconds
    **kwargs : dict
        Additional arguments passed to pd.read_html()

    Returns:
    -----
```



```

list of DataFrames or None
    List of pandas DataFrames (one per table found), or None if error occurs

Examples:
-----

# Basic usage
tables = read_html_tables_from_url("https://en.wikipedia.org/wiki/List_of_countries_by_GL

# Filter tables containing specific text
tables = read_html_tables_from_url(url, match="Country")

# With custom user agent
tables = read_html_tables_from_url(url, user_agent="MyBot/1.0")
"""

import pandas as pd
import requests

# Default User-Agent if none provided
if user_agent is None:
    user_agent = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, 1.

headers = {'User-Agent': user_agent}

try:
    print(f" Fetching data from: {url}")

    # Make the HTTP request
    response = requests.get(url, headers=headers, timeout=timeout)
    response.raise_for_status() # Raise an exception for bad status codes (4xx or 5xx)

    print(f" Successfully retrieved webpage (Status: {response.status_code})")

    # Parse HTML tables
    read_html_kwargs = {'match': match} if match else {}
    read_html_kwargs.update(kwargs) # Add any additional arguments

    tables = pd.read_html(response.content, **read_html_kwargs)

    print(f" Found {len(tables)} table(s) on the page")

    # Display table information
    if tables:

```

```

        print(f"\n Table Overview:")
        for i, table in enumerate(tables):
            print(f" Table {i}: {table.shape} (rows × columns)")
            if i >= 4: # Limit output for readability
                print(f" ... and {len(tables) - 5} more tables")
                break

    return tables

except requests.exceptions.HTTPError as err:
    print(f" HTTP Error: {err}")
    return None
except requests.exceptions.ConnectionError as err:
    print(f" Connection Error: {err}")
    return None
except requests.exceptions.Timeout as err:
    print(f" Timeout Error: Request took longer than {timeout} seconds")
    return None
except requests.exceptions.RequestException as err:
    print(f" Request Error: {err}")
    return None
except ValueError as err:
    print(f" No tables found on the page or parsing error: {err}")
    return None
except Exception as e:
    print(f" An unexpected error occurred: {e}")
    return None

```

Now let's demonstrate how to use our new function with different options:

```

# Example 1: Basic usage - Get all tables from GDP Wikipedia page
url = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)_per_capita'
tables = read_html_tables_from_url(url)

if tables:
    print(f"\n Successfully retrieved {len(tables)} tables!")

    # Display info about the largest table
    largest_table = max(tables, key=len)
    print(f"\n Largest table has {largest_table.shape[0]} rows and {largest_table.shape[1]} columns")
    print(f"Columns: {list(largest_table.columns)}")
else:

```

```
print(" Failed to retrieve tables")
```

```
Fetching data from: https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)_per_capita
Successfully retrieved webpage (Status: 200)
Found 6 table(s) on the page
```

```
Table Overview:
```

```
Table 0: (1, 2) (rows × columns)
Table 1: (224, 4) (rows × columns)
Table 2: (9, 2) (rows × columns)
Table 3: (13, 2) (rows × columns)
Table 4: (2, 2) (rows × columns)
... and 1 more tables
```

```
Successfully retrieved 6 tables!
```

```
Largest table has 224 rows and 4 columns
```

```
Columns: ['Country/Territory', 'IMF (2025)[a][5]', 'World Bank (2022-24)[6]', 'United Nations']
Successfully retrieved webpage (Status: 200)
Found 6 table(s) on the page
```

```
Table Overview:
```

```
Table 0: (1, 2) (rows × columns)
Table 1: (224, 4) (rows × columns)
Table 2: (9, 2) (rows × columns)
Table 3: (13, 2) (rows × columns)
Table 4: (2, 2) (rows × columns)
... and 1 more tables
```

```
Successfully retrieved 6 tables!
```

```
Largest table has 224 rows and 4 columns
```

```
Columns: ['Country/Territory', 'IMF (2025)[a][5]', 'World Bank (2022-24)[6]', 'United Nations']
```

```
# Example 2: Filtered search - Only get tables containing 'Country'
filtered_tables = read_html_tables_from_url(url, match='Country')

if filtered_tables:
    print(f"\n Found {len(filtered_tables)} table(s) containing 'Country'")

    # Examine the first filtered table
```

```

gdp_table = filtered_tables[0]
print(f"\n GDP Table Details:")
print(f"Shape: {gdp_table.shape}")
print(f"Columns: {list(gdp_table.columns)}")
print("\n Sample data:")
display(gdp_table.head(3))
else:
    print(" No tables found containing 'Country'")

```

Fetching data from: [https://en.wikipedia.org/wiki/List\\_of\\_countries\\_by\\_GDP\\_\(nominal\)\\_per\\_capita](https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)_per_capita)  
 Successfully retrieved webpage (Status: 200)  
 Found 1 table(s) on the page

Table Overview:  
 Table 0: (224, 4) (rows × columns)

Found 1 table(s) containing 'Country'

GDP Table Details:  
 Shape: (224, 4)  
 Columns: ['Country/Territory', 'IMF (2025)[a][5]', 'World Bank (2022–24)[6]', 'United Nations (2023)[7]']

Sample data:  
 Successfully retrieved webpage (Status: 200)  
 Found 1 table(s) on the page

Table Overview:  
 Table 0: (224, 4) (rows × columns)

Found 1 table(s) containing 'Country'

GDP Table Details:  
 Shape: (224, 4)  
 Columns: ['Country/Territory', 'IMF (2025)[a][5]', 'World Bank (2022–24)[6]', 'United Nations (2023)[7]']

Sample data:

|   | Country/Territory | IMF (2025)[a][5] | World Bank (2022–24)[6] | United Nations (2023)[7] |
|---|-------------------|------------------|-------------------------|--------------------------|
| 0 | Monaco            | —                | 256581                  | 256581                   |
| 1 | Liechtenstein     | —                | 207973                  | 207150                   |

|   | Country/Territory | IMF (2025)[a][5] | World Bank (2022–24)[6] | United Nations (2023)[7] |
|---|-------------------|------------------|-------------------------|--------------------------|
| 2 | Luxembourg        | 140941           | 137516                  | 128936                   |

## 6.8 Independent Study

### 6.8.1 Practice exercise 1: Reading .csv data

Read the file *Top 10 Albums By Year.csv*. This file contains the top 10 albums for each year from 1990 to 2021. Each row corresponds to a unique album.

#### 6.8.1.1

Print the first 5 rows of the data.

#### 6.8.1.2

How many rows and columns are there in the data?

#### 6.8.1.3

Print the summary statistics of the data, and answer the following questions:

1. What proportion of albums have 15 or lesser tracks? Mention a range for the proportion.
2. What is the mean length of a track (in minutes)?

### 6.8.2 Practice exercise 2: Reading .txt data

Read the file *bestseller\_books.txt*. It contains top 50 best-selling books on amazon from 2009 to 2019. Identify the delimiter without opening the file with Notepad or a text-editing software. How many rows and columns are there in the dataset?

Alternatively, you can use the argument `sep = None`, and `engine = 'python'`. The default engine is C. However, the ‘python’ engine has a ‘sniffer’ tool which may identify the delimiter automatically.

### 6.8.3 Practice exercise 3: Reading HTML data

Read the table(s) consisting of attendance of spectators in FIFA worlds cup from this [page](#). Read only those table(s) that have the word '*reaching*' in them. How many rows and columns are there in the table(s)?

### 6.8.4 Practice exercise 4: Reading JSON data

Read the movies dataset from [here](#). How many rows and columns are there in the data?

# 7 Pandas Fundamentals

<IPython.core.display.Image object>

## 7.1 Introduction to Pandas Fundamentals

In the **Reading Data** chapter, we learned how to load data files into pandas DataFrames. Now we'll dive deeper into pandas' powerful capabilities for data manipulation and analysis.

Pandas is an essential tool in the data scientist or data analyst's toolkit due to its ability to handle and transform data with ease and efficiency.

### 7.1.1 Pandas in the Data Science Ecosystem

Pandas doesn't work in isolation - it's designed to integrate seamlessly with other Python libraries:

| Library             | Integration with Pandas         | Purpose                                      |
|---------------------|---------------------------------|----------------------------------------------|
| <b>NumPy</b>        | Built on NumPy arrays           | Mathematical operations and array processing |
| <b>Matplotlib</b>   | Direct plotting from DataFrames | Data visualization and charts                |
| <b>Seaborn</b>      | Native DataFrame support        | Statistical visualizations                   |
| <b>Scikit-learn</b> | Seamless data flow              | Machine learning algorithms                  |
| <b>SciPy</b>        | Statistical analysis            | Advanced statistical functions               |

### 7.1.2 Core Pandas Capabilities

Throughout this chapter, you'll master these essential pandas operations:

- **Creating Series & DataFrames:** From Python data structures
- **Filtering & Subsetting:** Extract exactly the data you need
- **Sorting & Ranking:** Organize data for analysis
- **Adding & Removing:** Modify DataFrame structure
- **Data Types:** Work with different types of data

**Learning Path:** We'll start with fundamental concepts and progressively build toward advanced data manipulation techniques, giving you a complete toolkit for real-world data analysis.

Now let's start by importing the essential Pandas library:

```
import pandas as pd
```

## 7.2 Creating Pandas Series & DataFrames

There are **two primary approaches** for creating pandas data structures, each serving different purposes in your data analysis workflow:

### 7.2.1 Method 1: Reading from External Data Sources

In real-world data science, you'll typically work with data stored in external files or databases. This is the most common approach for production analysis.

**Popular Data Sources:**

| Source Type | Pandas Function              | Use Case                                    |
|-------------|------------------------------|---------------------------------------------|
| CSV Files   | <code>pd.read_csv()</code>   | Most common - structured tabular data       |
| Excel Files | <code>pd.read_excel()</code> | Business data, reports with multiple sheets |



| Source Type          | Pandas Function             | Use Case                             |
|----------------------|-----------------------------|--------------------------------------|
| <b>JSON Files</b>    | <code>pd.read_json()</code> | API data, web services, nested data  |
| <b>SQL Databases</b> | <code>pd.read_sql()</code>  | Enterprise databases, large datasets |
| <b>HTML Tables</b>   | <code>pd.read_html()</code> | Web scraping, online data tables     |

**Pro Tip:** We covered these methods extensively in the **Reading Data** chapter. If you need a refresher on loading external data, refer back to that section!

## 7.2.2 Method 2: Creating from Python Data Structures

When you need to create data programmatically or work with small datasets for testing and learning, you can build DataFrames and Series directly from Python objects.

**Benefits of this approach:**

- **Testing & Learning:** Perfect for experimenting with pandas features
- **Data Generation:** Create synthetic data for analysis
- **Small Datasets:** Handle simple, manual data entry
- **Prototyping:** Quick data structure creation for proof-of-concepts

Let's explore how to create pandas data structures from scratch!

### 7.2.2.1 Creating Series from Python Data Structures

A **Series** is pandas' 1D data structure - think of it as a smart, labeled list or array.

#### 7.2.2.1.1 Method A: From a Python List

The simplest way to create a Series is from a Python list. Pandas automatically assigns integer indices starting from 0.

```
#Defining a Pandas Series
series_example = pd.Series(['these','are','english','words'])
series_example
```

```

0      these
1       are
2    english
3     words
dtype: object

```

**Key Observation:** Notice the automatic integer indices (0, 1, 2, 3) on the left!

**Custom Indexing:** You can provide meaningful labels instead of default numbers using the `index` parameter:

```

#Defining a Pandas Series with custom row labels
series_example = pd.Series(['these','are','english','words'], index = range(101,105))
series_example

```

```

101     these
102      are
103   english
104    words
dtype: object

```

#### 7.2.2.1.2 Method B: From a Python Dictionary

Dictionaries are perfect for creating Series with **meaningful labels**. The dictionary keys automatically become the Series index, and values become the data points.

**Perfect for:** Time series data, named categories, any key-value relationships

```

# Real-world example: US GDP per capita by year (1960-2021)
# Notice some years are missing - pandas handles this gracefully!
GDP_per_capita_dict = {'1960':3007,'1961':3067,'1962':3244,'1963':3375,'1964':3574,'1965':38

```

```

#Example 2: Creating a Pandas Series from a Dictionary
GDP_per_capita_series = pd.Series(GDP_per_capita_dict)
GDP_per_capita_series.head()

```

```

1960    3007
1961    3067
1962    3244
1963    3375
1964    3574
dtype: int64

```

## 7.2.2.2 Creating a DataFrame from Python Data Structures

### 7.2.2.2.1 Method A: From a list of Python Dictionary

You can create a DataFrame where keys are column names and values are lists representing column data.

```
#List of dictionary consisting of 52 playing cards of the deck
deck_list_of_dictionaries = [{'value':i, 'suit':c}
for c in ['spades', 'clubs', 'hearts', 'diamonds']
for i in range(2,15)]
```

```
#Example 3: Creating a Pandas DataFrame from a List of dictionaries
deck_df = pd.DataFrame(deck_list_of_dictionaries)
deck_df.head()
```

|   | value | suit   |
|---|-------|--------|
| 0 | 2     | spades |
| 1 | 3     | spades |
| 2 | 4     | spades |
| 3 | 5     | spades |
| 4 | 6     | spades |

### 7.2.2.2.2 Method B: From a Python Dictionary

You can create a DataFrame where keys are column names and values are lists representing column data.

```
#Example 4: Creating a Pandas DataFrame from a Dictionary
dict_data = {'A': [1, 2, 3, 4, 5],
             'B': [20, 10, 50, 40, 30],
             'C': [100, 200, 300, 400, 500]}

dict_df = pd.DataFrame(dict_data)
dict_df
```

|   | A | B  | C   |
|---|---|----|-----|
| 0 | 1 | 20 | 100 |
| 1 | 2 | 10 | 200 |
| 2 | 3 | 50 | 300 |

|   | A | B  | C   |
|---|---|----|-----|
| 3 | 4 | 40 | 400 |
| 4 | 5 | 30 | 500 |

## 7.3 Data Selection and Filtering

Now that you understand series and dataframe are two fundamental data structures in pandas, let's master the art of **extracting exactly the data you need**. Data selection and filtering are fundamental skills that you'll use in every pandas project.

### 7.3.1 Basic Selection

#### 7.3.1.1 Extracting Column(s)

The first step when working with a DataFrame is often to extract one or more columns. To do this effectively, it's helpful to understand the internal structure of a DataFrame. Conceptually, you can think of a DataFrame as a dictionary of lists, where the keys are column names, and the values are lists or arrays containing data for the respective columns.

#### Loading Our Practice Dataset

Let's work with a real movie ratings dataset to practice our selection techniques:

```
import pandas as pd
movie_ratings = pd.read_csv('./datasets/movie_ratings.csv')
movie_ratings.head()
```

|   | Title              | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source   | Major |
|---|--------------------|----------|-----------------|-------------------|--------------|-------------|----------|-------|
| 0 | Opal Dreams        |          | 14443           | 14443             | 9000000      | Nov 22 2006 | PG/PG-13 |       |
| 1 | Major Dundee       |          | 14873           | 14873             | 3800000      | Apr 07 1965 | PG/PG-13 |       |
| 2 | The Informers      |          | 315000          | 315000            | 18000000     | Apr 24 2009 | R        |       |
| 3 | Buffalo Soldiers   |          | 353743          | 353743            | 15000000     | Jul 25 2003 | R        |       |
| 4 | The Last Sin Eater |          | 388390          | 388390            | 2200000      | Feb 09 2007 | PG/PG-13 |       |

#### 7.3.1.1.1 Method 1: Single Column Extraction

Each column represents a **feature** of your data. There are **two ways** to extract a single column:

**A) Bracket Notation:** `df['column_name']` (Recommended)

- Always works
- Handles column names with spaces
- Clear and explicit

**B) Dot Notation:** `df.column_name` (Limited use)

- Fails with spaces in column names
- Conflicts with DataFrame methods
- Shorter syntax for simple names

```
movie_ratings.Title
```

```
0           Opal Dreams
1         Major Dundee
2         The Informers
3       Buffalo Soldiers
4       The Last Sin Eater
...
2223          King Arthur
2224             Mulan
2225          Robin Hood
2226  Robin Hood: Prince of Thieves
2227          Spiceworld
Name: Title, Length: 2228, dtype: object
```

```
# Understanding the DataFrame structure:
# DataFrames work like dictionaries with enhanced features!
movie_ratings_dict = {
    'Title': ['Opal Dreams', 'Major Dundee', 'The Informers', 'Buffalo Soldiers', 'The Last
    'US Gross': [14443, 14873, 315000, 353743, 388390],
    'Worldwide Gross': [14443, 14873, 315000, 353743, 388390],
    'Production Budget': [9000000, 3800000, 18000000, 15000000, 2200000]
}
```

#### Dictionary Analogy:

Just like accessing dictionary values by key:

```
movie_ratings_dict['Title']
```

```
['Opal Dreams',  
 'Major Dundee',  
 'The Informers',  
 'Buffalo Soldiers',  
 'The Last Sin Eater']
```

### DataFrame Column Extraction:

Similarly, we extract DataFrame columns by column name:

```
movie_ratings['Title']
```

```
0           Opal Dreams  
1           Major Dundee  
2           The Informers  
3           Buffalo Soldiers  
4           The Last Sin Eater  
...  
2223          King Arthur  
2224             Mulan  
2225          Robin Hood  
2226  Robin Hood: Prince of Thieves  
2227             Spiceworld  
Name: Title, Length: 2228, dtype: object
```

#### 7.3.1.1.2 Method 2: Multiple Column Selection

**Key Rule:** Use a **list of column names** inside the brackets:

```
# Single column → Series  
df['column']  
  
# Multiple columns → DataFrame  
df[['col1', 'col2']]
```

**Notice:** Double brackets `[[ ]]` create a list, which tells pandas you want multiple columns!

```
movie_ratings[['Title', 'US Gross', 'Worldwide Gross' ]]
```

|      | Title                         | US Gross  | Worldwide Gross |
|------|-------------------------------|-----------|-----------------|
| 0    | Opal Dreams                   | 14443     | 14443           |
| 1    | Major Dundee                  | 14873     | 14873           |
| 2    | The Informers                 | 315000    | 315000          |
| 3    | Buffalo Soldiers              | 353743    | 353743          |
| 4    | The Last Sin Eater            | 388390    | 388390          |
| ...  | ...                           | ...       | ...             |
| 2223 | King Arthur                   | 51877963  | 203877963       |
| 2224 | Mulan                         | 120620254 | 303500000       |
| 2225 | Robin Hood                    | 105269730 | 310885538       |
| 2226 | Robin Hood: Prince of Thieves | 165493908 | 390500000       |
| 2227 | Spiceworld                    | 29342592  | 56042592        |

### 7.3.1.2 Extracting Row(s)

In many cases, we need to filter rows based on specific conditions or a combination of multiple conditions. Next, let's explore how to use these conditions effectively to extract rows that meet our criteria, whether it's a single condition or multiple conditions combined

#### 7.3.1.2.1 Single Condition Filtering

Filter rows in a DataFrame based on **one logical condition**.

This is the most common way to extract a subset of data, such as selecting all rows where a column's values meet a specific criterion (e.g., greater than, equal to, not equal).

For Example:

```
# extracting the rows that have IMDB Rating greater than 8
movie_ratings[movie_ratings['IMDB Rating'] > 8]
```

|     | Title               | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source | Major |
|-----|---------------------|----------|-----------------|-------------------|--------------|-------------|--------|-------|
| 21  | Gandhi, My Father   | 240425   | 1375194         | 5000000           | Aug 03 2007  | Other       |        |       |
| 56  | Ed Wood             | 5828466  | 5828466         | 18000000          | Sep 30 1994  | R           |        |       |
| 67  | Requiem for a Dream | 3635482  | 7390108         | 4500000           | Oct 06 2000  | Other       |        |       |
| 164 | Trainspotting       | 16501785 | 24000785        | 3100000           | Jul 19 1996  | R           |        |       |
| 181 | The Wizard of Oz    | 28202232 | 28202232        | 2777000           | Aug 25 2039  | G           |        |       |
| ... | ...                 | ...      | ...             | ...               | ...          | ...         |        |       |

|      | Title        | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source   | Major |
|------|--------------|----------|-----------------|-------------------|--------------|-------------|----------|-------|
| 2090 | Finding Nemo |          | 339714978       | 867894287         | 94000000     | May 30 2003 | G        |       |
| 2092 | Toy Story 3  |          | 410640665       | 1046340665        | 200000000    | Jun 18 2010 | G        |       |
| 2094 | Avatar       |          | 760167650       | 2767891499        | 237000000    | Dec 18 2009 | PG/PG-13 |       |
| 2130 | Scarface     |          | 44942821        | 44942821          | 25000000     | Dec 09 1983 | Other    |       |
| 2194 | The Departed |          | 133311000       | 290539042         | 90000000     | Oct 06 2006 | R        |       |

### 7.3.1.2.2 Multiple Conditions Filtering

When filtering with more than one condition, use **logical operators**:

| Operator | Purpose                                         | Example                         |
|----------|-------------------------------------------------|---------------------------------|
| &        | <b>AND</b> – both conditions must be True       | (df['A'] > 5) & (df['B'] < 10)  |
| \        | <b>OR</b> – at least one condition must be True | (df['A'] > 5) \  (df['B'] < 10) |
| ~        | <b>NOT</b> – negates the condition              | ~(df['A'] > 5)                  |

**Important Rule:** Always wrap each condition in **parentheses ()** to avoid operator precedence errors.

```
# extracting the rows that have IMDB Rating greater than 8 and US Gross less than 1000000
movie_ratings[(movie_ratings['IMDB Rating'] > 8) & (movie_ratings['US Gross'] < 1000000)]
```

|     | Title             | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source | Major |
|-----|-------------------|----------|-----------------|-------------------|--------------|-------------|--------|-------|
| 21  | Gandhi, My Father |          | 240425          | 1375194           | 5000000      | Aug 03 2007 | Other  |       |
| 636 | Lake of Fire      |          | 25317           | 25317             | 6000000      | Oct 03 2007 | Other  |       |

### 7.3.1.2.3 Negating Conditions with the ~ Operator

The tilde (~) operator in pandas is used to **negate boolean conditions**, making it especially useful when you want to *exclude* certain rows.

Instead of selecting rows that satisfy a condition, ~ allows you to filter rows that **do not** meet that condition. This is helpful for refining queries and focusing on the remaining data.

For example, if you want all rows where **IMDB Rating** is *not* equal to 8, you can write:



```
# Excluding the rows that have IMDB Rating that equals 8 using the tilde ~
movie_ratings[~(movie_ratings['IMDB Rating'] == 8)]
```

|      | Title                         | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source      | Major |
|------|-------------------------------|----------|-----------------|-------------------|--------------|-------------|-------------|-------|
| 0    | Opal Dreams                   |          | 14443           | 14443             | 9000000      |             | Nov 22 2006 | I     |
| 1    | Major Dundee                  |          | 14873           | 14873             | 3800000      |             | Apr 07 1965 | I     |
| 2    | The Informers                 |          | 315000          | 315000            | 18000000     |             | Apr 24 2009 | I     |
| 3    | Buffalo Soldiers              |          | 353743          | 353743            | 15000000     |             | Jul 25 2003 | I     |
| 4    | The Last Sin Eater            |          | 388390          | 388390            | 2200000      |             | Feb 09 2007 | I     |
| ...  | ...                           |          | ...             | ...               | ...          |             | ...         | ...   |
| 2223 | King Arthur                   |          | 51877963        | 203877963         | 90000000     |             | Jul 07 2004 | I     |
| 2224 | Mulan                         |          | 120620254       | 303500000         | 90000000     |             | Jun 19 1998 | C     |
| 2225 | Robin Hood                    |          | 105269730       | 310885538         | 210000000    |             | May 14 2010 | I     |
| 2226 | Robin Hood: Prince of Thieves |          | 165493908       | 390500000         | 50000000     |             | Jun 14 1991 | I     |
| 2227 | Spiceworld                    |          | 29342592        | 56042592          | 25000000     |             | Jan 23 1998 | I     |

### Alternative: Using !=

Another way to exclude rows where a column equals a certain value is to use the **not-equal operator !=**:

```
# using the != operator
movie_ratings[movie_ratings['IMDB Rating'] != 8]
```

|      | Title                         | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source      | Major |
|------|-------------------------------|----------|-----------------|-------------------|--------------|-------------|-------------|-------|
| 0    | Opal Dreams                   |          | 14443           | 14443             | 9000000      |             | Nov 22 2006 | I     |
| 1    | Major Dundee                  |          | 14873           | 14873             | 3800000      |             | Apr 07 1965 | I     |
| 2    | The Informers                 |          | 315000          | 315000            | 18000000     |             | Apr 24 2009 | I     |
| 3    | Buffalo Soldiers              |          | 353743          | 353743            | 15000000     |             | Jul 25 2003 | I     |
| 4    | The Last Sin Eater            |          | 388390          | 388390            | 2200000      |             | Feb 09 2007 | I     |
| ...  | ...                           |          | ...             | ...               | ...          |             | ...         | ...   |
| 2223 | King Arthur                   |          | 51877963        | 203877963         | 90000000     |             | Jul 07 2004 | I     |
| 2224 | Mulan                         |          | 120620254       | 303500000         | 90000000     |             | Jun 19 1998 | C     |
| 2225 | Robin Hood                    |          | 105269730       | 310885538         | 210000000    |             | May 14 2010 | I     |
| 2226 | Robin Hood: Prince of Thieves |          | 165493908       | 390500000         | 50000000     |             | Jun 14 1991 | I     |
| 2227 | Spiceworld                    |          | 29342592        | 56042592          | 25000000     |             | Jan 23 1998 | I     |

### 7.3.2 Advanced Selection: `.loc[]` and `.iloc[]`

When you need **precise control** over both rows and columns, pandas provides two powerful indexers that let you select exactly what you need.

#### 7.3.2.1 `.loc[]` vs `.iloc[]`: The Key Difference

| Indexer              | Uses                          | Best For                 | Example                                        |
|----------------------|-------------------------------|--------------------------|------------------------------------------------|
| <code>.loc[]</code>  | <b>Labels</b><br>(names)      | Human-readable selection | <code>df.loc['row_name', 'column_name']</code> |
| <code>.iloc[]</code> | <b>Positions</b><br>(numbers) | Programmatic selection   | <code>df.iloc[0, 1]</code>                     |

#### Mental Model

Think of your DataFrame like a **spreadsheet**:

- `.loc[]` works like clicking on **named** rows/columns (A1, B2, etc.)
- `.iloc[]` works like clicking on **position** numbers (row 0, column 1, etc.)

#### Memory Tip:

- `loc` = **L**abels/**L**ocation names
- `iloc` = **i**nteger **l**ocations

#### Preparing Our Data: Sorting by IMDB Rating

To demonstrate `.loc[]` and `.iloc[]` effectively, let's first sort our movies by rating so we can easily select the best ones:

```
movie_ratings_sorted = movie_ratings.sort_values(by = 'IMDB Rating', ascending = False)
movie_ratings_sorted.head()
```

|      | Title                    | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source      | Major |
|------|--------------------------|----------|-----------------|-------------------|--------------|-------------|-------------|-------|
| 182  | The Shawshank Redemption |          | 28241469        | 28241469          | 25000000     |             | Sep 23 1994 | R     |
| 2084 | Inception                |          | 285630280       | 753830280         | 160000000    |             | Jul 16 2010 | PG    |
| 790  | Schindler's List         |          | 96067179        | 321200000         | 25000000     |             | Dec 15 1993 | R     |
| 1962 | Pulp Fiction             |          | 107928762       | 212928762         | 8000000      |             | Oct 14 1994 | R     |
| 561  | The Dark Knight          |          | 533345358       | 1022345358        | 185000000    |             | Jul 18 2008 | PG    |

### 7.3.2.2 Using .loc[] - Label-Based Selection

Syntax: `df.loc[row_indexer, column_indexer]`

Key Features:

- Uses **actual labels** (index names, column names)
- **Inclusive** of both endpoints in slicing
- Can combine with boolean conditions
- Most **intuitive** for data analysis

**Example:** Let's subset the Title, Worldwide Gross, Production Budget, and IMDB Rating of the top 3 movies using specific row indices that obtained from the above sorting table.

```
# Select specific rows by index and specific columns by name
movies_subset = movie_ratings_sorted.loc[[182, 2084, 790], ['Title', 'IMDB Rating', 'US Gross']]
print(" Selected movies with key financial and rating data:")
movies_subset
```

Selected movies with key financial and rating data:

|      | Title                    | IMDB Rating | US Gross | Worldwide Gross | Production Budget |           |
|------|--------------------------|-------------|----------|-----------------|-------------------|-----------|
| 182  | The Shawshank Redemption | 9.2         |          | 28241469        | 28241469          | 25000000  |
| 2084 | Inception                |             | 9.1      | 285630280       | 753830280         | 160000000 |
| 790  | Schindler's List         |             | 8.9      | 96067179        | 321200000         | 25000000  |

The `:` symbol in `.loc` is a slicing operator that represents a range or all elements in the specified dimension (rows or columns). Use `:` alone to select all rows/columns, or with start/end points to slice specific parts of the DataFrame.

```
# Select ALL rows (:) but only specific columns
movies_subset = movie_ratings_sorted.loc[:, ['Title', 'Worldwide Gross', 'Production Budget']]
print(" All movies with financial and rating columns only:")
movies_subset
```

All movies with financial and rating columns only:

|      | Title                         | Worldwide Gross | Production Budget | IMDB Rating |
|------|-------------------------------|-----------------|-------------------|-------------|
| 182  | The Shawshank Redemption      | 28241469        | 25000000          | 9.2         |
| 2084 | Inception                     | 753830280       | 160000000         | 9.1         |
| 790  | Schindler's List              | 321200000       | 25000000          | 8.9         |
| 1962 | Pulp Fiction                  | 212928762       | 8000000           | 8.9         |
| 561  | The Dark Knight               | 1022345358      | 185000000         | 8.9         |
| ...  | ...                           | ...             | ...               | ...         |
| 1051 | Glitter                       | 4273372         | 8500000           | 2.0         |
| 1495 | Disaster Movie                | 34690901        | 20000000          | 1.7         |
| 1116 | Crossover                     | 7009668         | 5600000           | 1.7         |
| 805  | From Justin to Kelly          | 4922166         | 12000000          | 1.6         |
| 1147 | Super Babies: Baby Geniuses 2 | 9109322         | 20000000          | 1.4         |

```
# Select a RANGE of rows (182 to 561) and specific columns
movies_subset = movie_ratings_sorted.loc[182:561, ['Title', 'Worldwide Gross', 'Production Budget', 'IMDB Rating']]
print("Movies from index 182 to 561 (inclusive):")
movies_subset
```

Movies from index 182 to 561 (inclusive):

|      | Title                    | Worldwide Gross | Production Budget | IMDB Rating |
|------|--------------------------|-----------------|-------------------|-------------|
| 182  | The Shawshank Redemption | 28241469        | 25000000          | 9.2         |
| 2084 | Inception                | 753830280       | 160000000         | 9.1         |
| 790  | Schindler's List         | 321200000       | 25000000          | 8.9         |
| 1962 | Pulp Fiction             | 212928762       | 8000000           | 8.9         |
| 561  | The Dark Knight          | 1022345358      | 185000000         | 8.9         |

### 7.3.2.3 Combining .loc with Boolean Conditions

**Power Move:** Filter rows AND select columns simultaneously using boolean logic!

```
# extracting the rows that have IMDB Rating greater than 8 or US Gross less than 1000000, on the same time
movie_ratings[(movie_ratings['IMDB Rating'] > 8) & (movie_ratings['US Gross'] < 1000000)][['Title', 'Worldwide Gross', 'Production Budget', 'IMDB Rating']]

#using loc to extract the rows that have IMDB Rating greater than 8 or US Gross less than 1000000, on the same time
movie_ratings.loc[(movie_ratings['IMDB Rating'] > 8) & (movie_ratings['US Gross'] < 1000000)][['Title', 'Worldwide Gross', 'Production Budget', 'IMDB Rating']]
```

|     | Title             | IMDB Rating |
|-----|-------------------|-------------|
| 21  | Gandhi, My Father | 8.1         |
| 636 | Lake of Fire      | 8.4         |

#### 7.3.2.4 Using .iloc[] - Position-Based Selection

Syntax: `df.iloc[row_positions, column_positions]`

Key Features:

- Uses **integer positions** (0-based indexing like Python lists)
- **Exclusive** of end point in slicing (like Python slicing)
- Great for **systematic sampling** or **first/last N records**
- Think: “integer location”

|               | Position Type                        | Example | Description              |
|---------------|--------------------------------------|---------|--------------------------|
| <b>Single</b> | <code>df.iloc[0, 1]</code>           |         | First row, second column |
| <b>List</b>   | <code>df.iloc[[0, 2], [1, 3]]</code> |         | Specific positions       |
| <b>Slice</b>  | <code>df.iloc[0:3, 1:4]</code>       |         | Range (exclusive end)    |
| <b>All</b>    | <code>df.iloc[:, :]</code>           |         | All rows and columns     |

Memory Tip:

- `iloc` = integer positions,
- `loc` = label names

<IPython.core.display.Image object>

```
# let's check the movie_ratings_sorted DataFrame
movie_ratings_sorted.head()
```

|      | Title                    | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source      | Major |
|------|--------------------------|----------|-----------------|-------------------|--------------|-------------|-------------|-------|
| 182  | The Shawshank Redemption |          | 28241469        | 28241469          | 25000000     |             | Sep 23 1994 | R     |
| 2084 | Inception                |          | 285630280       | 753830280         | 160000000    |             | Jul 16 2010 | PG    |
| 790  | Schindler's List         |          | 96067179        | 321200000         | 25000000     |             | Dec 15 1993 | R     |
| 1962 | Pulp Fiction             |          | 107928762       | 212928762         | 8000000      |             | Oct 14 1994 | R     |
| 561  | The Dark Knight          |          | 533345358       | 1022345358        | 185000000    |             | Jul 18 2008 | PG    |

After sorting, the position-based index changes, while the label-based index remains unchanged. Let's pass the position-based index to `iloc` to retrieve the top 2 rows from the `movie_ratings_sorted` DataFrame.

```
# Get first 2 rows by POSITION (0 and 1), all columns
top_2_movies = movie_ratings_sorted.iloc[0:2, :]
print(" Top 2 movies by position:")
top_2_movies
```

Top 2 movies by position:

|      | Title                    | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source      | Major |
|------|--------------------------|----------|-----------------|-------------------|--------------|-------------|-------------|-------|
| 182  | The Shawshank Redemption |          | 28241469        | 28241469          | 25000000     |             | Sep 23 1994 | R     |
| 2084 | Inception                |          | 285630280       | 753830280         | 160000000    |             | Jul 16 2010 | PG    |

It is important to note that the endpoint is excluded in an `iloc` slice.

For comparison, let's pass the same argument to `loc` and see what it returns.

```
# Same range [0:2] with loc - uses LABELS, includes endpoint
loc_result = movie_ratings_sorted.loc[0:2, :]
print(" Using .loc[0:2,:] - includes rows with labels 0, 1, AND 2:")
loc_result
```

Using `.loc[0:2,:]` - includes rows with labels 0, 1, AND 2:

|      | Title                  | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source | Major   |
|------|------------------------|----------|-----------------|-------------------|--------------|-------------|--------|---------|
| 0    | Opal Dreams            |          | 14443           | 14443             | 9000000      | Nov 22 2006 |        | PG/PG-1 |
| 851  | Star Trek: Generations |          | 75671262        | 120000000         | 38000000     | Nov 18 1994 |        | PG/PG-1 |
| 140  | Tuck Everlasting       |          | 19161999        | 19344615          | 15000000     | Oct 11 2002 |        | PG/PG-1 |
| 708  | De-Lovely              |          | 13337299        | 18396382          | 4000000      | Jun 25 2004 |        | PG/PG-1 |
| 705  | Flyboys                |          | 13090630        | 14816379          | 60000000     | Sep 22 2006 |        | PG/PG-1 |
| ...  | ...                    |          | ...             | ...               | ...          | ...         |        | ...     |
| 955  | The Brothers Solomon   |          | 900926          | 900926            | 10000000     | Sep 07 2007 |        | R       |
| 1637 | Drumline               |          | 56398162        | 56398162          | 20000000     | Dec 13 2002 |        | PG/PG-1 |
| 1610 | Hollywood Homicide     |          | 30207785        | 51107785          | 75000000     | Jun 13 2003 |        | PG/PG-1 |
| 569  | Doom                   |          | 28212337        | 54612337          | 70000000     | Oct 21 2005 |        | R       |
| 2    | The Informers          |          | 315000          | 315000            | 18000000     | Apr 24 2009 |        | R       |

As you can see, `:` is used in the same way as in `loc` to denote a range of the index. All rows between label indices 0 and 2, inclusive of both ends, are returned.

```
# Select first 10 rows and specific column positions [0, 2, 3, 9]
movies_iloc_subset = movie_ratings_sorted.iloc[0:10, [0, 2, 3, 9]]
print(" First 10 movies with columns at positions 0, 2, 3, 9:")
movies_iloc_subset
```

First 10 movies with columns at positions 0, 2, 3, 9:

|      | Title                                             | Worldwide Gross | Production Budget | IMDB Rating |     |
|------|---------------------------------------------------|-----------------|-------------------|-------------|-----|
| 182  | The Shawshank Redemption                          |                 | 28241469          | 25000000    | 9.2 |
| 2084 | Inception                                         |                 | 753830280         | 160000000   | 9.1 |
| 790  | Schindler's List                                  |                 | 321200000         | 25000000    | 8.9 |
| 1962 | Pulp Fiction                                      |                 | 212928762         | 8000000     | 8.9 |
| 561  | The Dark Knight                                   |                 | 1022345358        | 185000000   | 8.9 |
| 2092 | Toy Story 3                                       |                 | 1046340665        | 200000000   | 8.9 |
| 487  | The Lord of the Rings: The Fellowship of the Ring |                 | 868621686         | 109000000   | 8.8 |
| 335  | Fight Club                                        |                 | 100853753         | 65000000    | 8.8 |
| 497  | The Lord of the Rings: The Return of the King     |                 | 1133027325        | 94000000    | 8.8 |
| 1081 | C'era una volta il West                           |                 | 5321508           | 5000000     | 8.8 |

**Critical Thinking Question:** Can you use `.iloc` for conditional filtering (like `df.iloc[df['column'] > 5]`)?

**Answer:** **No!** `.iloc` only accepts **integer positions**, not boolean conditions. For conditional filtering, you must use `.loc` or boolean indexing directly.

### 7.3.2.5 Summary: `.loc` vs `.iloc` - Choose Your Weapon!

| Feature           | <code>.loc</code> (Label-Based)              | <code>.iloc</code> (Position-Based)          |
|-------------------|----------------------------------------------|----------------------------------------------|
| <b>Uses</b>       | <b>Labels/Names</b>                          | <b>Integer Positions</b>                     |
| <b>Slicing</b>    | <code>[start:end]</code> <b>includes</b> end | <code>[start:end]</code> <b>excludes</b> end |
| <b>Examples</b>   | <code>df.loc['row_name', 'col_name']</code>  | <code>df.iloc[0, 1]</code>                   |
| <b>Conditions</b> | <code>df.loc[df['col'] &gt; 5]</code>        | No boolean conditions                        |
| <b>Best For</b>   | <b>Business logic</b> filtering              | <b>Systematic</b> sampling                   |

**Mental Model:**

- `.loc` = “Give me the data **by name/label**”
- `.iloc` = “Give me the data **by position number**”

### 7.3.3 Finding Extremes in pandas: Values, Labels, and Positions

#### 7.3.3.1 Finding extreme values

When you need to find extreme values, pandas offers powerful methods:

|                    | Method        | Purpose              | Returns                  |
|--------------------|---------------|----------------------|--------------------------|
| <code>min()</code> | <b>Actual</b> | <b>minimum</b> value | The minimum value itself |
| <code>max()</code> | <b>Actual</b> | <b>maximum</b> value | The maximum value itself |

```
# find the max/min worldwide gross
max_gross = max(movie_ratings["Worldwide Gross"])
min_gross = min(movie_ratings["Worldwide Gross"])

print("Highest gross:", max_gross)
print("Minimum gross:", min_gross)
```

```
Highest gross: 2767891499
Minimum gross: 884
```

#### 7.3.3.2 Finding extreme labels

| Method                | Purpose                 | Returns     | Use With          |
|-----------------------|-------------------------|-------------|-------------------|
| <code>idxmax()</code> | <b>Label</b> of maximum | Index label | <code>.loc</code> |
| <code>idxmin()</code> | <b>Label</b> of minimum | Index label | <code>.loc</code> |

**Pro Tip:** Use `idx` methods to **locate** the extreme values, then use `.loc` to get the full row!

`min()` / `max()` vs. `idxmin()` / `idxmax()`

- Use `min()` / `max()` when you only need the **extreme value** itself.
- Use `idxmin()` / `idxmax()` when you need to know **where that extreme occurs** — i.e., the **index label**, so you can fetch the corresponding row.



```
# Find the INDEX LABELS of movies with max/min worldwide gross
max_gross_index = movie_ratings_sorted['Worldwide Gross'].idxmax()
min_gross_index = movie_ratings_sorted['Worldwide Gross'].idxmin()

print(" Highest grossing movie index:", max_gross_index)
print(" Lowest grossing movie index:", min_gross_index)

# Now get the full details of these movies
print("\n Highest grossing movie:")
print(movie_ratings_sorted.loc[max_gross_index, ['Title', 'Worldwide Gross']])

print("\n Lowest grossing movie:")
print(movie_ratings_sorted.loc[min_gross_index, ['Title', 'Worldwide Gross']])
```

```
Highest grossing movie index: 2094
Lowest grossing movie index: 896
```

```
Highest grossing movie:
Title          Avatar
Worldwide Gross 2767891499
Name: 2094, dtype: object
```

```
Lowest grossing movie:
Title          In Her Line of Fire
Worldwide Gross          884
Name: 896, dtype: object
```

**Workflow:** `idxmin()/idxmax()` return **index labels** → use with `.loc` to extract full rows

**Example:** Find the index, then get the actual value:

```
# Get the actual gross values using the indices we found
print(" Max worldwide gross:", f"${movie_ratings_sorted.loc[max_gross_index, 'Worldwide Gross']}")
print(" Min worldwide gross:", f"${movie_ratings_sorted.loc[min_gross_index, 'Worldwide Gross']}")
```

```
Max worldwide gross: $2,767,891,499
Min worldwide gross: $884
```

### 7.3.3.3 Finding extreme positions

When you need **position numbers** instead of **label names**, you need to use `argmax()` and `argmin()`

| Method                | Purpose                    | Returns                 | Use With           |
|-----------------------|----------------------------|-------------------------|--------------------|
| <code>argmax()</code> | <b>Position</b> of maximum | Integer index (0-based) | <code>.iloc</code> |
| <code>argmin()</code> | <b>Position</b> of minimum | Integer index (0-based) | <code>.iloc</code> |

```
# Find POSITIONS (not labels) of movies with max/min worldwide gross
max_position = movie_ratings_sorted['Worldwide Gross'].argmax()
min_position = movie_ratings_sorted['Worldwide Gross'].argmin()

print(" Max gross at position:", max_position)
print(" Min gross at position:", min_position)

# Use iloc with positions to get the actual values
print("\n Max worldwide gross:", f"${movie_ratings_sorted.iloc[max_position, 2]:,.0f}")
print(" Min worldwide gross:", f"${movie_ratings_sorted.iloc[min_position, 2]:,.0f}")

# Get full movie details using positions
print("\n Movies with extreme gross values:")
print("Highest:", movie_ratings_sorted.iloc[max_position]['Title'])
print("Lowest:", movie_ratings_sorted.iloc[min_position]['Title'])
```

```
Max gross at position: 48
Min gross at position: 2149
```

```
Max worldwide gross: $2,767,891,499
Min worldwide gross: $884
```

```
Movies with extreme gross values:
Highest: Avatar
Lowest: In Her Line of Fire
```

### Pro Tips for Min/Max Operations

| Scenario                           | Recommended Method                            | Reason                              |
|------------------------------------|-----------------------------------------------|-------------------------------------|
| <b>Custom/Non-unique indices</b>   | <code>idxmax()</code> / <code>idxmin()</code> | Returns meaningful labels           |
| <b>Simple position-based work</b>  | <code>argmax()</code> / <code>argmin()</code> | Direct integer positions            |
| <b>Find max/min across rows</b>    | <code>df.idxmax(axis=1)</code>                | Column name with max value per row  |
| <b>Find max/min across columns</b> | <code>df.idxmax(axis=0)</code>                | Row index with max value per column |

Memory Aid:

- `idx` → **labels** → use with `.loc`
- `arg` → **positions** → use with `.iloc`

## 7.4 Sorting & Ranking: Organize data for analysis

### 7.4.1 Why Sorting and Ranking Matter

Sorting and ranking are fundamental operations in data analysis that help you:

- **Identify top performers** (best movies, highest sales)
- **Understand distributions** (from lowest to highest values)
- **Find outliers** (extreme values at the ends)
- **Create ordered presentations** (leaderboards, reports)

### 7.4.2 Basic Sorting with `sort_values()`

Syntax: `df.sort_values(by='column_name', ascending=True/False)`

|                          | Parameter                 | Purpose | Options                                         |
|--------------------------|---------------------------|---------|-------------------------------------------------|
| <code>by</code>          | Column(s) to sort by      |         | Single column or list of columns                |
| <code>ascending</code>   | Sort direction            |         | <b>True</b> (low→high), <b>False</b> (high→low) |
| <code>na_position</code> | Where to put NaN values   |         | 'first' or 'last'                               |
| <code>inplace</code>     | Modify original DataFrame |         | <b>True</b> or <b>False</b>                     |

```
# Sort movies by IMDB Rating (ascending - lowest to highest)
movies_by_rating_asc = movie_ratings.sort_values(by='IMDB Rating', ascending=True)
print(" Movies sorted by IMDB Rating (worst to best):")
print(movies_by_rating_asc[['Title', 'IMDB Rating']].head())

print("\n" + "="*50)

# Sort movies by IMDB Rating (descending - highest to lowest)
movies_by_rating_desc = movie_ratings.sort_values(by='IMDB Rating', ascending=False)
print(" Movies sorted by IMDB Rating (best to worst):")
print(movies_by_rating_desc[['Title', 'IMDB Rating']].head())
```

Movies sorted by IMDB Rating (worst to best):

|      | Title                         | IMDB Rating |
|------|-------------------------------|-------------|
| 1147 | Super Babies: Baby Geniuses 2 | 1.4         |
| 805  | From Justin to Kelly          | 1.6         |
| 1495 | Disaster Movie                | 1.7         |
| 1116 | Crossover                     | 1.7         |
| 1051 | Glitter                       | 2.0         |

=====

Movies sorted by IMDB Rating (best to worst):

|      | Title                    | IMDB Rating |
|------|--------------------------|-------------|
| 182  | The Shawshank Redemption | 9.2         |
| 2084 | Inception                | 9.1         |
| 790  | Schindler's List         | 8.9         |
| 1962 | Pulp Fiction             | 8.9         |
| 561  | The Dark Knight          | 8.9         |

### 7.4.3 Sorting by Multiple Columns

**Real-world scenario:** Sort movies by **genre first**, then by **worldwide gross** within each genre

**Syntax:** `df.sort_values(by=['column1', 'column2'], ascending=[True, False])`

```
movie_ratings
```

|   | Title       | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source      | Major |
|---|-------------|----------|-----------------|-------------------|--------------|-------------|-------------|-------|
| 0 | Opal Dreams |          |                 | 14443             | 14443        | 9000000     | Nov 22 2006 | I     |

|      | Title                         | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source      | Major |
|------|-------------------------------|----------|-----------------|-------------------|--------------|-------------|-------------|-------|
| 1    | Major Dundee                  |          |                 | 14873             | 14873        | 3800000     | Apr 07 1965 | I     |
| 2    | The Informers                 |          |                 | 315000            | 315000       | 18000000    | Apr 24 2009 | I     |
| 3    | Buffalo Soldiers              |          |                 | 353743            | 353743       | 15000000    | Jul 25 2003 | I     |
| 4    | The Last Sin Eater            |          |                 | 388390            | 388390       | 2200000     | Feb 09 2007 | I     |
| ...  | ...                           |          |                 | ...               | ...          | ...         | ...         | ...   |
| 2223 | King Arthur                   |          |                 | 51877963          | 203877963    | 90000000    | Jul 07 2004 | I     |
| 2224 | Mulan                         |          |                 | 120620254         | 303500000    | 90000000    | Jun 19 1998 | O     |
| 2225 | Robin Hood                    |          |                 | 105269730         | 310885538    | 210000000   | May 14 2010 | I     |
| 2226 | Robin Hood: Prince of Thieves |          |                 | 165493908         | 390500000    | 50000000    | Jun 14 1991 | I     |
| 2227 | Spiceworld                    |          |                 | 29342592          | 56042592     | 25000000    | Jan 23 1998 | I     |

```
# Multi-column sorting: Genre (A-Z), then Worldwide Gross (high to low)
movies_multi_sort = movie_ratings.sort_values(
    by=['Major Genre', 'Worldwide Gross'],
    ascending=[True, False] # Genre A-Z, Gross high-to-low
)

# Pro tip: Check how many unique genres we have
print(f"\n Total unique genres: {movie_ratings['Major Genre'].nunique()}")
print(" Genres:", sorted(movie_ratings['Major Genre'].unique()))

print(" Movies sorted by Major Genre (A-Z), then by Worldwide Gross (high-to-low):")
movies_multi_sort[['Major Genre', 'Title', 'Worldwide Gross']].head(10)
```

Total unique genres: 6

Genres: ['Action/Adventure', 'Comedy', 'Documentary', 'Drama', 'Horror/Thriller', 'Western/

Movies sorted by Major Genre (A-Z), then by Worldwide Gross (high-to-low):

|      |                  | Major Genre                                   | Title | Worldwide Gross |
|------|------------------|-----------------------------------------------|-------|-----------------|
| 2094 | Action/Adventure | Avatar                                        |       | 2767891499      |
| 497  | Action/Adventure | The Lord of the Rings: The Return of the King |       | 1133027325      |
| 895  | Action/Adventure | Pirates of the Caribbean: Dead Man's Chest    |       | 1065659812      |
| 2092 | Action/Adventure | Toy Story 3                                   |       | 1046340665      |
| 496  | Action/Adventure | Alice in Wonderland                           |       | 1023291110      |
| 561  | Action/Adventure | The Dark Knight                               |       | 1022345358      |
| 495  | Action/Adventure | Harry Potter and the Sorcerer's Stone         |       | 976457891       |

|     |                  | Major Genre                               | Title | Worldwide Gross |
|-----|------------------|-------------------------------------------|-------|-----------------|
| 894 | Action/Adventure | Pirates of the Caribbean: At World's End  |       | 960996492       |
| 494 | Action/Adventure | Harry Potter and the Order of the Phoenix |       | 938468864       |
| 493 | Action/Adventure | Harry Potter and the Half-Blood Prince    |       | 937499905       |

## 7.4.4 Ranking Methods with rank()

**Purpose:** Assign numerical ranks to values (1st, 2nd, 3rd, etc.)

| Method    | Handling Ties         | Example (values: [1, 2, 2, 4]) |
|-----------|-----------------------|--------------------------------|
| 'average' | Average of tied ranks | [1.0, 2.5, 2.5, 4.0]           |
| 'min'     | Minimum rank for ties | [1, 2, 2, 4]                   |
| 'max'     | Maximum rank for ties | [1, 3, 3, 4]                   |
| 'first'   | Order of appearance   | [1, 2, 3, 4]                   |
| 'dense'   | No gaps in ranks      | [1, 2, 2, 3]                   |

**Use Cases:** Create leaderboards, percentile rankings, performance tiers

```
# Create rankings for IMDB ratings (higher rating = better rank)
movie_ratings['Rating_Rank'] = movie_ratings['IMDB Rating'].rank(ascending=False, method='min')

# Create rankings for worldwide gross (higher gross = better rank)
movie_ratings['Gross_Rank'] = movie_ratings['Worldwide Gross'].rank(ascending=False, method='min')

# Show top 10 movies by IMDB rating with their ranks
top Rated = movie_ratings.nsmallest(10, 'Rating_Rank')
print(" Top 10 Movies by IMDB Rating:")
print(top Rated[['Title', 'IMDB Rating', 'Rating_Rank', 'Worldwide Gross', 'Gross_Rank']].to_string(index=False))

print("\n" + "="*60)

# Compare: Movies that are highly rated vs highly grossing
print("\n Rating vs Gross Performance Analysis:")
comparison = movie_ratings[['Title', 'IMDB Rating', 'Rating_Rank', 'Worldwide Gross', 'Gross_Rank']]
print(comparison.to_string(index=False))
```

Top 10 Movies by IMDB Rating:

|  | Title                    | IMDB Rating | Rating_Rank | Worldwide Gross |
|--|--------------------------|-------------|-------------|-----------------|
|  | The Shawshank Redemption | 9.2         | 1.0         | 28241469        |

|                                                   |                  |     |     |            |
|---------------------------------------------------|------------------|-----|-----|------------|
|                                                   | Inception        | 9.1 | 2.0 | 753830280  |
|                                                   | The Dark Knight  | 8.9 | 3.0 | 1022345358 |
|                                                   | Schindler's List | 8.9 | 3.0 | 321200000  |
|                                                   | Pulp Fiction     | 8.9 | 3.0 | 212928762  |
|                                                   | Toy Story 3      | 8.9 | 3.0 | 1046340665 |
|                                                   | Cidade de Deus   | 8.8 | 7.0 | 28763397   |
|                                                   | Fight Club       | 8.8 | 7.0 | 100853753  |
| The Lord of the Rings: The Fellowship of the Ring |                  | 8.8 | 7.0 | 868621686  |
| The Lord of the Rings: The Return of the King     |                  | 8.8 | 7.0 | 1133027325 |

=====

#### Rating vs Gross Performance Analysis:

|                                    | Title               | IMDB Rating | Rating_Rank | Worldwide Gross | Gross_Rank |
|------------------------------------|---------------------|-------------|-------------|-----------------|------------|
|                                    | Opal Dreams         | 6.5         | 994.0       | 14443           | 2223.0     |
|                                    | Major Dundee        | 6.7         | 828.0       | 14873           | 2222.0     |
|                                    | The Informers       | 5.2         | 1808.0      | 315000          | 2179.0     |
|                                    | Buffalo Soldiers    | 6.9         | 665.0       | 353743          | 2176.0     |
|                                    | The Last Sin Eater  | 5.7         | 1535.0      | 388390          | 2172.0     |
| The City of Your Final Destination |                     | 6.6         | 914.0       | 493296          | 2165.0     |
|                                    | The Claim           | 6.5         | 994.0       | 622023          | 2154.0     |
|                                    | Texas Rangers       | 5.0         | 1873.0      | 623374          | 2152.0     |
|                                    | Ride With the Devil | 6.4         | 1058.0      | 630779          | 2149.0     |
|                                    | Karakter            | 7.8         | 167.0       | 713413          | 2142.0     |

### 7.4.5 Percentile Ranking

Percentile ranking allows you to convert raw ranks into **percentiles (0–100%)**, giving a clearer sense of each value's **relative position** in the dataset.

This is especially useful when comparing across datasets of different sizes or when you want a normalized measure of ranking.

In pandas, you can compute percentile ranks using the `rank()` method with the parameter `pct=True`:

```
# Calculate percentile rankings (0-100%)
movie_ratings['Rating_Percentile'] = movie_ratings['IMDB Rating'].rank(pct=True) * 100
movie_ratings['Gross_Percentile'] = movie_ratings['Worldwide Gross'].rank(pct=True) * 100

# Interpret percentiles
print(" Percentile Interpretation Guide:")
print("• 90th+ percentile = Top 10% (Excellent)")
```

```

print("• 75th+ percentile = Top 25% (Very Good)")
print("• 50th+ percentile = Above Average")
print("• Below 50th = Below Average")

print("\n Top performers in both rating AND gross:")
top_both = movie_ratings[
    (movie_ratings['Rating_Percentile'] >= 90) &
    (movie_ratings['Gross_Percentile'] >= 90)
][['Title', 'IMDB Rating', 'Rating_Percentile', 'Worldwide Gross', 'Gross_Percentile']]

if len(top_both) > 0:
    print(top_both.round(1).to_string(index=False))
else:
    print(" No movies in top 10% for both rating AND gross!")

# Show some examples with percentile context
print("\n Sample movies with percentile context:")
sample = movie_ratings[['Title', 'IMDB Rating', 'Rating_Percentile', 'Worldwide Gross', 'Gross_Percentile']]
print(sample.round(1).to_string(index=False))

```

#### Percentile Interpretation Guide:

- 90th+ percentile = Top 10% (Excellent)
- 75th+ percentile = Top 25% (Very Good)
- 50th+ percentile = Above Average
- Below 50th = Below Average

#### Top performers in both rating AND gross:

|  | Title                                             | IMDB Rating | Rating_Percentile | Worldwide Gross |
|--|---------------------------------------------------|-------------|-------------------|-----------------|
|  | The Green Mile                                    | 8.4         | 98.3              |                 |
|  | Shutter Island                                    | 8.0         | 94.8              |                 |
|  | The Curious Case of Benjamin Button               | 8.0         | 94.8              |                 |
|  | Jurassic Park 3                                   | 7.9         | 93.3              |                 |
|  | Gone with the Wind                                | 8.2         | 96.9              |                 |
|  | The Exorcist                                      | 8.1         | 96.1              |                 |
|  | The Bourne Ultimatum                              | 8.2         | 96.9              |                 |
|  | Jaws                                              | 8.3         | 97.7              |                 |
|  | Shrek                                             | 8.0         | 94.8              |                 |
|  | How to Train Your Dragon                          | 8.2         | 96.9              |                 |
|  | Casino Royale                                     | 8.0         | 94.8              |                 |
|  | Forrest Gump                                      | 8.6         | 99.1              |                 |
|  | The Lord of the Rings: The Fellowship of the Ring | 8.8         | 99.6              |                 |
|  | Jurassic Park                                     | 7.9         | 93.3              |                 |



|                                                        |     |       |
|--------------------------------------------------------|-----|-------|
| The Lord of the Rings: The Two Towers                  | 8.7 | 99.4  |
| The Lord of the Rings: The Return of the King          | 8.8 | 99.6  |
| Batman Begins                                          | 8.3 | 97.7  |
| X2                                                     | 7.8 | 91.5  |
| 300                                                    | 7.8 | 91.5  |
| Iron Man                                               | 7.9 | 93.3  |
| The Dark Knight                                        | 8.9 | 99.8  |
| A Beautiful Mind                                       | 8.0 | 94.8  |
| The Pursuit of Happyness                               | 7.8 | 91.5  |
| Schindler's List                                       | 8.9 | 99.8  |
| The Fugitive                                           | 7.8 | 91.5  |
| Star Trek                                              | 8.2 | 96.9  |
| Pirates of the Caribbean: The Curse of the Black Pearl | 8.0 | 94.8  |
| As Good as it Gets                                     | 7.8 | 91.5  |
| Inglourious Basterds                                   | 8.4 | 98.3  |
| Se7en                                                  | 8.7 | 99.4  |
| American Beauty                                        | 8.6 | 99.1  |
| Toy Story                                              | 8.2 | 96.9  |
| Slumdog Millionaire                                    | 8.3 | 97.7  |
| Raiders of the Lost Ark                                | 8.7 | 99.4  |
| The Last Samurai                                       | 7.8 | 91.5  |
| Gladiator                                              | 8.3 | 97.7  |
| The Matrix                                             | 8.7 | 99.4  |
| The Hangover                                           | 7.9 | 93.3  |
| Saving Private Ryan                                    | 8.5 | 98.8  |
| Toy Story 2                                            | 8.0 | 94.8  |
| Aladdin                                                | 7.8 | 91.5  |
| Terminator 2: Judgment Day                             | 8.5 | 98.8  |
| WALL-E                                                 | 8.5 | 98.8  |
| Ratatouille                                            | 8.1 | 96.1  |
| The Incredibles                                        | 8.1 | 96.1  |
| The Sixth Sense                                        | 8.2 | 96.9  |
| Up                                                     | 8.4 | 98.3  |
| Inception                                              | 9.1 | 100.0 |
| The Lion King                                          | 8.2 | 96.9  |
| ET: The Extra-Terrestrial                              | 7.9 | 93.3  |
| Finding Nemo                                           | 8.2 | 96.9  |
| Toy Story 3                                            | 8.9 | 99.8  |
| Avatar                                                 | 8.3 | 97.7  |
| The Departed                                           | 8.5 | 98.8  |

Sample movies with percentile context:

| Title | IMDB Rating | Rating_Percentile | Worldwide Gross | Gross_Percentile |
|-------|-------------|-------------------|-----------------|------------------|
|-------|-------------|-------------------|-----------------|------------------|

|                    |     |      |        |     |
|--------------------|-----|------|--------|-----|
| Opal Dreams        | 6.5 | 54.0 | 14443  | 0.3 |
| Major Dundee       | 6.7 | 61.0 | 14873  | 0.3 |
| The Informers      | 5.2 | 18.2 | 315000 | 2.2 |
| Buffalo Soldiers   | 6.9 | 68.4 | 353743 | 2.4 |
| The Last Sin Eater | 5.7 | 29.9 | 388390 | 2.6 |

## 7.4.6 Advanced Sorting Techniques

### 7.4.6.1 Custom Sorting with `sort_values(key=function)`

The `key` parameter in `sort_values()` lets you apply a **transformation function** to column values **before sorting**.

This is useful when the natural order of values is not the same as the order you want to sort by.

```
# Custom sorting: Sort movie titles by length (shortest to longest)
movies_by_title_length = movie_ratings.sort_values(
    by='Title',
    key=lambda x: x.str.len() # Sort by title length, not alphabetically
)

print(" Movies sorted by title length (shortest to longest):")
title_length_sample = movies_by_title_length[['Title', 'IMDB Rating']].head(8)
for idx, row in title_length_sample.iterrows():
    print(f" '{row['Title']}' ({len(row['Title'])} chars) - {row['IMDB Rating']}")

print("\n" + "="*50)

# Sort by absolute deviation from average rating (find most "average" movies)
avg_rating = movie_ratings['IMDB Rating'].mean()
movies_by_avg_deviation = movie_ratings.sort_values(
    by='IMDB Rating',
    key=lambda x: abs(x - avg_rating) # Sort by distance from average
)

print(f"\n Movies closest to average rating ({avg_rating:.1f}):")
avg_movies = movies_by_avg_deviation[['Title', 'IMDB Rating']].head(5)
for idx, row in avg_movies.iterrows():
    deviation = abs(row['IMDB Rating'] - avg_rating)
    print(f" '{row['Title']}' - Rating: {row['IMDB Rating']} (±{deviation:.2f} from avg)")
```

Movies sorted by title length (shortest to longest):

```
'9' (1 chars) - 7.8
'54' (2 chars) - 5.6
'Up' (2 chars) - 8.4
'Pi' (2 chars) - 7.5
'X2' (2 chars) - 7.8
'21' (2 chars) - 6.7
'Woo' (3 chars) - 3.4
'Saw' (3 chars) - 7.7
```

=====

Movies closest to average rating (6.2):

```
'Beyond Borders' - Rating: 6.2 ( $\pm 0.04$  from avg)
'Come Early Morning' - Rating: 6.2 ( $\pm 0.04$  from avg)
'Bad Boys II' - Rating: 6.2 ( $\pm 0.04$  from avg)
'Drop Dead Gorgeous' - Rating: 6.2 ( $\pm 0.04$  from avg)
'The Sisterhood of the Traveling Pants 2' - Rating: 6.2 ( $\pm 0.04$  from avg)
```

#### 7.4.6.2 Index Sorting with `sort_index()`

Sort DataFrame by **row indices** or **column names** rather than values

```
# First, let's see the current index order
print(" Current index range:")
print(f"First 5 indices: {list(movie_ratings.index[:5])}")
print(f>Last 5 indices: {list(movie_ratings.index[-5:])}")

# Sort by index in ascending order (restore original row order)
movies_index_sorted = movie_ratings.sort_index()
print(f"\n After sorting by index (ascending):")
print(f"First 5 indices: {list(movies_index_sorted.index[:5])}")

# Sort by index in descending order
movies_index_desc = movie_ratings.sort_index(ascending=False)
print(f"\n After sorting by index (descending):")
print(f"First 5 indices: {list(movies_index_desc.index[:5])}")

# Sort columns alphabetically by column names
print(f"\n Original column order:")
print(f"Columns: {list(movie_ratings.columns)}")
```

```

movies_cols_sorted = movie_ratings.sort_index(axis=1) # axis=1 for columns
print(f"\n Columns after alphabetical sorting:")
print(f"Columns: {list(movies_cols_sorted.columns)}")

```

Current index range:

First 5 indices: [0, 1, 2, 3, 4]

Last 5 indices: [2223, 2224, 2225, 2226, 2227]

After sorting by index (ascending):

First 5 indices: [0, 1, 2, 3, 4]

After sorting by index (descending):

First 5 indices: [2227, 2226, 2225, 2224, 2223]

Original column order:

Columns: ['Title', 'US Gross', 'Worldwide Gross', 'Production Budget', 'Release Date', 'MPAA']

Columns after alphabetical sorting:

Columns: ['Creative Type', 'Gross\_Percentile', 'Gross\_Rank', 'IMDB Rating', 'IMDB Votes', 'MPAA']

## 7.4.7 Top-N and Bottom-N Selection

When you want the **largest or smallest N values** in a Series or DataFrame, pandas provides efficient methods that avoid a full sort:

- **nlargest(n)** → returns the top **n** rows with the highest values
- **nsmallest(n)** → returns the bottom **n** rows with the lowest values

These are **faster than sorting** the entire DataFrame with `.sort_values()`, especially for large datasets.

```

# Top 5 highest-grossing movies (using nlargest)
print(" Top 5 Highest-Grossing Movies:")
top_gross = movie_ratings.nlargest(5, 'Worldwide Gross')[['Title', 'Worldwide Gross', 'IMDB Rating']]
for idx, row in top_gross.iterrows():
    print(f" {row['Title']} - ${row['Worldwide Gross']:,.0f} ( {row['IMDB Rating']})")

print("\n" + "="*50)

# Top 5 highest-rated movies (using nlargest)

```

```

print("\n Top 5 Highest-Rated Movies:")
topRated = movie_ratings.nlargest(5, 'IMDB Rating')[['Title', 'IMDB Rating', 'Worldwide Gross']]
for idx, row in topRated.iterrows():
    print(f" {row['Title']} - {row['IMDB Rating']} ({row['Worldwide Gross']:,.0f})")

print("\n" + "="*50)

# Bottom 5 lowest-rated movies (using nsmallest)
print("\n Bottom 5 Lowest-Rated Movies:")
bottomRated = movie_ratings.nsmallest(5, 'IMDB Rating')[['Title', 'IMDB Rating', 'Worldwide Gross']]
for idx, row in bottomRated.iterrows():
    print(f" {row['Title']} - {row['IMDB Rating']} ({row['Worldwide Gross']:,.0f})")

print("\n" + "="*50)

# Bottom 5 lowest-grossing movies (using nsmallest)
print("\n Bottom 5 Lowest-Grossing Movies:")
bottomGross = movie_ratings.nsmallest(5, 'Worldwide Gross')[['Title', 'Worldwide Gross', 'IMDB Rating']]
for idx, row in bottomGross.iterrows():
    print(f" {row['Title']} - ${row['Worldwide Gross']:,.0f} ( {row['IMDB Rating']})")

```

#### Top 5 Highest-Grossing Movies:

Avatar - \$2,767,891,499 ( 8.3)

Titanic - \$1,842,879,955 ( 7.4)

The Lord of the Rings: The Return of the King - \$1,133,027,325 ( 8.8)

Pirates of the Caribbean: Dead Man's Chest - \$1,065,659,812 ( 7.3)

Toy Story 3 - \$1,046,340,665 ( 8.9)

=====

#### Top 5 Highest-Rated Movies:

The Shawshank Redemption - 9.2 (\$28,241,469)

Inception - 9.1 (\$753,830,280)

The Dark Knight - 8.9 (\$1,022,345,358)

Schindler's List - 8.9 (\$321,200,000)

Pulp Fiction - 8.9 (\$212,928,762)

=====

#### Bottom 5 Lowest-Rated Movies:

Super Babies: Baby Geniuses 2 - 1.4 (\$9,109,322)

From Justin to Kelly - 1.6 (\$4,922,166)

Crossover - 1.7 (\$7,009,668)  
Disaster Movie - 1.7 (\$34,690,901)  
Son of the Mask - 2.0 (\$59,918,422)

=====

Bottom 5 Lowest-Grossing Movies:  
In Her Line of Fire - \$884 ( 3.5)  
The Californians - \$4,134 ( 5.1)  
Peace, Propaganda and the Promised Land - \$4,930 ( 3.0)  
Say Uncle - \$5,361 ( 5.7)  
London - \$12,667 ( 7.7)

### 7.4.8 Performance Comparison: Sorting Methods

| Method                     | Best For                  | Performance               | Use Case                            |
|----------------------------|---------------------------|---------------------------|-------------------------------------|
| <code>sort_values()</code> | <b>Full sorting</b>       | Slower for large data     | Complete ranking, detailed analysis |
| <code>nlargest(n)</code>   | <b>Top N only</b>         | <b>Faster</b> for small N | Leaderboards, top performers        |
| <code>nsmallest(n)</code>  | <b>Bottom N only</b>      | <b>Faster</b> for small N | Finding outliers, worst performers  |
| <code>rank()</code>        | <b>Relative positions</b> | Moderate                  | Percentiles, competition rankings   |

**Pro Tip:** Use `nlargest()`/`nsmallest()` when you only need the top/bottom few records!

### 7.4.9 Real-World Sorting Scenarios

Let's apply sorting and ranking to solve common business questions:

```
# Scenario 1: Find "Hidden Gems" - High rating but low gross
print(" HIDDEN GEMS: Great movies that didn't make much money")
print("="*55)

# Create efficiency ratio: Rating per dollar (higher = better value)
movie_ratings['Rating_per_Gross'] = movie_ratings['IMDB Rating'] / (movie_ratings['Worldwide Gross'] / 1000000)
```

```

hidden_gems = movie_ratings[
    (movie_ratings['IMDB Rating'] >= 7.5) &
    (movie_ratings['Worldwide Gross'] < movie_ratings['Worldwide Gross'].median())
].nlargest(5, 'IMDB Rating')

for idx, row in hidden_gems.iterrows():
    print(f" {row['Title']}")
    print(f"    Rating: {row['IMDB Rating']} |    Gross: ${row['Worldwide Gross']:,.0f}")
    print()

print("="*55)

# Scenario 2: Find "Blockbuster Hits" - High gross AND high rating
print("\n BLOCKBUSTER HITS: Great movies that made tons of money")
print("="*55)

blockbusters = movie_ratings[
    (movie_ratings['IMDB Rating'] >= 7.0) &
    (movie_ratings['Worldwide Gross'] >= movie_ratings['Worldwide Gross'].quantile(0.8))
].sort_values(['IMDB Rating', 'Worldwide Gross'], ascending=[False, False]).head(5)

for idx, row in blockbusters.iterrows():
    print(f" {row['Title']}")
    print(f"    Rating: {row['IMDB Rating']} |    Gross: ${row['Worldwide Gross']:,.0f}")
    print()

print("="*55)

# Scenario 3: ROI Analysis - Best return on investment
print("\n ROI CHAMPIONS: Best gross-to-budget ratio")
print("="*55)

# Calculate ROI ratio (avoid division by zero)
movie_ratings['ROI_Ratio'] = movie_ratings['Worldwide Gross'] / movie_ratings['Production Budget']

top_roi = movie_ratings.nlargest(5, 'ROI_Ratio')
for idx, row in top_roi.iterrows():
    roi = row['ROI_Ratio']
    print(f" {row['Title']}")
    print(f"    ROI: {roi:.1f}x | Budget: ${row['Production Budget']:,.0f} → Gross: ${row['Worldwide Gross']:,.0f}")
    print()

```

HIDDEN GEMS: Great movies that didn't make much money

=====

The Shawshank Redemption

Rating: 9.2 | Gross: \$28,241,469

Cidade de Deus

Rating: 8.8 | Gross: \$28,763,397

C'era una volta il West

Rating: 8.8 | Gross: \$5,321,508

The Town

Rating: 8.7 | Gross: \$33,180,607

Memento

Rating: 8.7 | Gross: \$39,665,950

=====

BLOCKBUSTER HITS: Great movies that made tons of money

=====

Inception

Rating: 9.1 | Gross: \$753,830,280

Toy Story 3

Rating: 8.9 | Gross: \$1,046,340,665

The Dark Knight

Rating: 8.9 | Gross: \$1,022,345,358

Schindler's List

Rating: 8.9 | Gross: \$321,200,000

Pulp Fiction

Rating: 8.9 | Gross: \$212,928,762

=====

ROI CHAMPIONS: Best gross-to-budget ratio

=====

Paranormal Activity

ROI: 12918.0x | Budget: \$15,000 → Gross: \$193,770,453



Tarnation

ROI: 5330.3x | Budget: \$218 → Gross: \$1,162,014

Super Size Me

ROI: 454.3x | Budget: \$65,000 → Gross: \$29,529,368

The Brothers McMullen

ROI: 417.1x | Budget: \$25,000 → Gross: \$10,426,506

The Blair Witch Project

ROI: 413.8x | Budget: \$600,000 → Gross: \$248,300,000

## 7.4.10 \*Key Takeaways: Sorting & Ranking Mastery\*\*

### Essential Methods

- `sort_values()`: Complete sorting by column values
- `rank()`: Assign numerical ranks (1st, 2nd, 3rd...)
- `nlargest()` / `smallest()`: Efficient top/bottom N selection
- `sort_index()`: Sort by row indices or column names

### Decision Framework

```
# Quick reference for method selection
if "I need top/bottom N only":
    use_nlargest_or_smallest()
elif "I need complete ordering":
    use_sort_values()
elif "I need relative positions":
    use_rank()
elif "I need custom ordering logic":
    use_sort_values_with_key_function()
```

### Pro Tips

1. **Performance:** `nlargest(5)` beats `sort_values().head(5)` for large datasets
2. **Multiple columns:** `sort_values(['col1', 'col2'], ascending=[True, False])`
3. **Percentiles:** `rank(pct=True) * 100` for 0-100% percentile scores
4. **Custom logic:** Use `key=lambda x: transform(x)` for complex sorting rules
5. **Stability:** pandas sorting is **stable** (preserves original order for ties)

*Next up: Learn how to add and remove your sorted data!*

## 7.5 Adding & Removing: Modify DataFrame structure

### 7.5.1 Why DataFrame Structure Modification Matters

Real-world data analysis often requires:

- **Adding new columns** (calculated fields, derived metrics)
- **Removing columns** (irrelevant features, memory optimization)
- **Removing rows** (outliers, invalid data, filtering)

### 7.5.2 Adding Columns to DataFrames

There are multiple ways to add new columns, each with different use cases:

#### 7.5.2.1 Method 1: Direct Assignment

The simplest way to add a column with a constant value or calculated field. The added column will be put at the end of the dataframe

```
# Let's work with our movie dataset to demonstrate column operations
print("Original DataFrame shape:", movie_ratings.shape)
print("Original columns:", list(movie_ratings.columns))

# Method 1a: Add a constant value column
movie_ratings['Data_Source'] = 'IMDB'
print(f"\n Added constant column 'Data_Source'")

# Method 1b: Add calculated column based on existing data
movie_ratings['Profit'] = movie_ratings['Worldwide Gross'] - movie_ratings['Production Budget']
print(f" Added calculated column 'Profit'")

# Show results
print(f"\n New DataFrame shape:", movie_ratings.shape)
print(" Sample of new columns:")
movie_ratings[['Title', 'Profit', 'Data_Source']].head()
```

Original DataFrame shape: (2228, 19)

Original columns: ['Title', 'US Gross', 'Worldwide Gross', 'Production Budget', 'Release Date', 'Genre', 'Director', 'Cast', 'Box Office', 'Runtime', 'MPAA Rating', 'IMDB Rating', 'IMDB Votes', 'Metacritic Score', 'Rotten Tomatoes Score', 'Rotten Tomatoes Audience Score', 'Rotten Tomatoes Critics Score', 'Rotten Tomatoes Freshness', 'Rotten Tomatoes Consensus', 'Rotten Tomatoes Trailer Score', 'Rotten Tomatoes Trailer Score 2']

Added constant column 'Data\_Source'

Added calculated column 'Profit'

New DataFrame shape: (2228, 19)

Sample of new columns:

|   | Title              | Profit    | Data_Source |
|---|--------------------|-----------|-------------|
| 0 | Opal Dreams        | -8985557  | IMDB        |
| 1 | Major Dundee       | -3785127  | IMDB        |
| 2 | The Informers      | -17685000 | IMDB        |
| 3 | Buffalo Soldiers   | -14646257 | IMDB        |
| 4 | The Last Sin Eater | -1811610  | IMDB        |

### 7.5.2.2 Method 3: Using insert() for Specific Positioning

When you need to **control exactly where** the new column appears

```
movies_copy['Release Date']
```

```
0      Nov 22 2006
1      Apr 07 1965
2      Apr 24 2009
3      Jul 25 2003
4      Feb 09 2007
```

...

```
2223    Jul 07 2004
2224    Jun 19 1998
2225    May 14 2010
2226    Jun 14 1991
2227    Jan 23 1998
```

Name: Release Date, Length: 2228, dtype: object

```
# Method 3: insert() for precise column positioning
# Create a working copy to demonstrate insert()
movies_copy = movie_ratings.copy()

print(" Original column order (first 5):")
print(list(movies_copy.columns[:5]))

# Insert a column at position 2 (after Title and Genre)
```

```
# 1) Parse the date column (e.g., "Nov 22 2006")
movies_copy["Release Date"] = pd.to_datetime(movies_copy["Release Date"], format="%b %d %Y",

movies_copy.insert(2, 'Release_Decade', (movies_copy['Release Date'].dt.year // 10) * 10)

# Insert another column at the beginning (position 0)
movies_copy.insert(0, 'Movie_ID', range(1, len(movies_copy) + 1))

print("\n New column order (first 6):")
print(list(movies_copy.columns[:6]))

# Note: insert() modifies the DataFrame in-place
print(f"\n DataFrame shape after inserts: {movies_copy.shape}")

print("\n Sample with positioned columns:")
movies_copy[['Movie_ID', 'Title', 'Major Genre', 'Release_Decade', 'IMDB Rating']].head()
```

Original column order (first 5):

```
['Title', 'US Gross', 'Worldwide Gross', 'Production Budget', 'Release Date']
```

New column order (first 6):

```
['Movie_ID', 'Title', 'US Gross', 'Release_Decade', 'Worldwide Gross', 'Production Budget']
```

DataFrame shape after inserts: (2228, 21)

Sample with positioned columns:

|   |   | Movie_ID | Title              | Major Genre     | Release_Decade | IMDB Rating |
|---|---|----------|--------------------|-----------------|----------------|-------------|
| 0 | 1 |          | Opal Dreams        | Drama           | 2000           | 6.5         |
| 1 | 2 |          | Major Dundee       | Western/Musical | 1960           | 6.7         |
| 2 | 3 |          | The Informers      | Horror/Thriller | 2000           | 5.2         |
| 3 | 4 |          | Buffalo Soldiers   | Comedy          | 2000           | 6.9         |
| 4 | 5 |          | The Last Sin Eater | Drama           | 2000           | 5.7         |

### 7.5.3 Removing Columns from DataFrames

The most common and flexible way to remove columns is with `.drop()`.

- By default, `.drop()` works on **rows**, so you need to specify `axis=1` (or `columns=`) to drop columns.
- You can remove **one column** or **multiple columns** at the same time.
- `.drop()` returns a **new DataFrame** unless you use `inplace=True`.

```
# Setup: Create a working copy with extra columns for deletion demo
movies_for_deletion = movie_ratings.copy()

# Add some temporary columns to demonstrate deletion
movies_for_deletion['Temp_Col1'] = 'Delete_Me'
movies_for_deletion['Temp_Col2'] = range(len(movies_for_deletion))
movies_for_deletion['Temp_Col3'] = movies_for_deletion['IMDB Rating'] * 2

print(" DataFrame before deletion:")
print(f"Shape: {movies_for_deletion.shape}")
print(f"Columns: {list(movies_for_deletion.columns)}")

# Using drop() - most common and flexible
print(f"\n Using drop()")

# Drop single column
movies_method1 = movies_for_deletion.drop('Temp_Col1', axis=1)
print(f"After dropping 1 column: {movies_method1.shape}")

# Drop multiple columns
movies_method1b = movies_for_deletion.drop(['Temp_Col1', 'Temp_Col2'], axis=1)
print(f"After dropping 2 columns: {movies_method1b.shape}")

print(f"\n Final comparison - all methods result in same shape: {movies_method1b.shape}")
```

DataFrame before deletion:

Shape: (2228, 22)

Columns: ['Title', 'US Gross', 'Worldwide Gross', 'Production Budget', 'Release Date', 'MPAA

Using drop()

After dropping 1 column: (2228, 21)

After dropping 2 columns: (2228, 20)

Final comparison - all methods result in same shape: (2228, 20)

## 7.5.4 Removing Rows from DataFrames

There are several ways to remove or filter out unwanted rows in a DataFrame:

- using `drop()`
- boolean filtering

### 7.5.4.1 Method 1: Using `.drop()`

- Works by specifying the **row index labels** to remove.
- Returns a new DataFrame unless you use `inplace=True`.

### 7.5.4.2 Method2: Boolean Filtering

- Keep only rows that meet a condition (the most common method in data analysis).
- More flexible than `drop()`, especially when conditions depend on column values.

```
# Row removal demonstrations
print(" Original dataset info:")
print(f"Total movies: {len(movie_ratings)}")
print(f"IMDB Rating range: {movie_ratings['IMDB Rating'].min():.1f} - {movie_ratings['IMDB Rating'].max():.1f}")

# Method 1: Remove rows by index using drop()
print(f"\n Method 1: Remove specific rows by index")
movies_drop_index = movie_ratings.drop([0, 1, 2]) # Remove first 3 movies
print(f"After dropping first 3 rows: {len(movies_drop_index)} movies")

# Method 2: Remove rows by condition (Boolean filtering)
print(f"\n Method 2: Remove rows by condition")

# Remove movies with low ratings (< 5.0)
movies_no_low_rating = movie_ratings[movie_ratings['IMDB Rating'] >= 5.0]
removed_count = len(movie_ratings) - len(movies_no_low_rating)
print(f"Removed {removed_count} movies with rating < 5.0")
print(f"Remaining: {len(movies_no_low_rating)} movies")

# Remove movies with missing/zero gross
movies_with_gross = movie_ratings[movie_ratings['Worldwide Gross'] > 0]
removed_no_gross = len(movie_ratings) - len(movies_with_gross)
print(f"Removed {removed_no_gross} movies with no gross data")
```

```

print(f"Remaining: {len(movies_with_gross)} movies")

# Summary of filtering effects
print(f"\n Filtering Summary:")
print(f"Original dataset: {len(movie_ratings)} movies")
print(f"After rating filter: {len(movies_no_low_rating)} movies ({len(movies_no_low_rating)/len(movies)}%)")
print(f"After gross filter: {len(movies_with_gross)} movies ({len(movies_with_gross)/len(movies)}%)")

```

Original dataset info:  
Total movies: 2228  
IMDB Rating range: 1.4 - 9.2

Method 1: Remove specific rows by index  
After dropping first 3 rows: 2225 movies

Method 2: Remove rows by condition  
Removed 324 movies with rating < 5.0  
Remaining: 1904 movies  
Removed 0 movies with no gross data  
Remaining: 2228 movies

Filtering Summary:  
Original dataset: 2228 movies  
After rating filter: 1904 movies (85.5%)  
After gross filter: 2228 movies (100.0%)

## 7.5.5 Method Comparison Summary

| Operation             | Method                         | Best For             | In-Place? | Performance |
|-----------------------|--------------------------------|----------------------|-----------|-------------|
| <b>Add Column</b>     | <code>df['col'] = value</code> | Simple assignment    | Yes       | Fastest     |
| <b>Add Column</b>     | <code>df.insert()</code>       | Specific positioning | Yes       | Medium      |
| <b>Remove Columns</b> | <code>df.drop()</code>         | Flexible removal     | No        | Fast        |
| <b>Remove Rows</b>    | Boolean filtering              | Conditional removal  | No        | Fast        |
| <b>Remove Rows</b>    | <code>df.drop()</code>         | Index-based removal  | No        | Fast        |

## 7.6 Data Types: Working with Different Data Formats

### 7.6.1 Why Data Types Matter in Data Science

Data types are **fundamental** to pandas operations:

- **Functionality:** Type-specific methods (`.str`, `.dt` accessors)
- **Speed:** Vectorized operations work best with proper types
- **Data Integrity:** Prevent incorrect calculations and comparisons

### 7.6.2 Complete Pandas Data Types Reference

| Data Type                    | Description           | Memory Efficient       | Example         | Common Use Cases     |
|------------------------------|-----------------------|------------------------|-----------------|----------------------|
| <code>int64</code>           | 64-bit integers       | Efficient              | 1, 2, 3         | Counts, IDs, years   |
| <code>float64</code>         | 64-bit floating point | Efficient              | 1.5, 2.7, 3.14  | Measurements, ratios |
| <code>object</code>          | Strings/Mixed types   | Memory heavy           | 'text', [1,2,3] | Names, categories    |
| <code>category</code>        | Categorical data      | <b>Super efficient</b> | 'A', 'B', 'C'   | Fixed categories     |
| <code>datetime64[ns]</code>  | Date and time         | Efficient              | 2024-01-15      | Timestamps, dates    |
| <code>timedelta64[ns]</code> | Time differences      | Efficient              | 5 days          | Duration, intervals  |
| <code>bool</code>            | True/False values     | <b>Ultra efficient</b> | True, False     | Flags, conditions    |
| <code>string</code>          | Pure string type      | Moderate               | 'pandas 2.0+'   | Text analysis (new)  |

### 7.6.3 Data Type Detection and Inspection

```
# Comprehensive data type exploration
print(" DATA TYPE ANALYSIS")
print("="*40)

# Basic data type information
print(" Basic DataFrame Info:")
print(f"Shape: {movie_ratings.shape}")
print(f"\n Data Types Overview:")
print(movie_ratings.dtypes)
```



```
# Data type categorization
print(f"\n Columns by Data Type:")
numeric_cols = movie_ratings.select_dtypes(include=['int64', 'float64']).columns.tolist()
object_cols = movie_ratings.select_dtypes(include=['object']).columns.tolist()
datetime_cols = movie_ratings.select_dtypes(include=['datetime64']).columns.tolist()

print(f" Numeric columns ({len(numeric_cols)}): {numeric_cols}")
print(f" Object columns ({len(object_cols)}): {object_cols}")
print(f" Datetime columns ({len(datetime_cols)}): {datetime_cols}")
```

## DATA TYPE ANALYSIS

=====

Basic DataFrame Info:

Shape: (2228, 19)

Data Types Overview:

|                   |         |
|-------------------|---------|
| Title             | object  |
| US Gross          | int64   |
| Worldwide Gross   | int64   |
| Production Budget | int64   |
| Release Date      | object  |
| MPAA Rating       | object  |
| Source            | object  |
| Major Genre       | object  |
| Creative Type     | object  |
| IMDB Rating       | float64 |
| IMDB Votes        | int64   |
| Rating_Rank       | float64 |
| Gross_Rank        | float64 |
| Rating_Percentile | float64 |
| Gross_Percentile  | float64 |
| Rating_per_Gross  | float64 |
| ROI_Ratio         | float64 |
| Data_Source       | object  |
| Profit            | int64   |

dtype: object

Columns by Data Type:

Numeric columns (12): ['US Gross', 'Worldwide Gross', 'Production Budget', 'IMDB Rating', 'IMDB Votes', 'Rating\_Rank', 'Gross\_Rank', 'Rating\_Percentile', 'Gross\_Percentile', 'Rating\_per\_Gross', 'ROI\_Ratio', 'Profit']  
Object columns (7): ['Title', 'Release Date', 'MPAA Rating', 'Source', 'Major Genre', 'Creative Type', 'Data\_Source']  
Datetime columns (0): []

## 7.6.4 Smart Data Type Filtering with select\_dtypes()

**Purpose:** Efficiently select columns based on their data types for targeted analysis

```
# Select columns by data type for targeted analysis
print(" DATA TYPE FILTERING EXAMPLES")
print("="*45)

# 1. Select object (string/categorical) columns
print(" Object/String Columns:")
object_data = movie_ratings.select_dtypes(include='object')
print(f"Columns: {list(object_data.columns)}")
print("Sample data:")
print(object_data.head(3))

print("\n" + "="*45)
```

```
DATA TYPE FILTERING EXAMPLES
=====

Object/String Columns:
Columns: ['Title', 'Release Date', 'MPAA Rating', 'Source', 'Major Genre', 'Creative Type',
Sample data:

      Title Release Date MPAA Rating      Source \
0   Opal Dreams  Nov 22 2006    PG/PG-13  Adapted screenplay
1  Major Dundee  Apr 07 1965    PG/PG-13  Adapted screenplay
2 The Informers  Apr 24 2009         R    Adapted screenplay

      Major Genre Creative Type Data_Source
0          Drama      Fiction          IMDB
1 Western/Musical      Fiction          IMDB
2 Horror/Thriller      Fiction          IMDB

=====
```

```
# 2. Select numeric columns (int + float)
print(" Numeric Columns:")
numeric_data = movie_ratings.select_dtypes(include='number')
print(f"Columns: {list(numeric_data.columns)}")
print("Sample data:")
print(numeric_data.head(3))

print("\n" + "="*45)
```



```

1          450.480737    0.003914  -3785127
2          16.507937     0.017500 -17685000

```

=====

#### Advanced Type Filtering:

Integer columns: ['US Gross', 'Worldwide Gross', 'Production Budget', 'IMDB Votes', 'Profit']

Float columns: ['IMDB Rating', 'Rating\_Rank', 'Gross\_Rank', 'Rating\_Percentile', 'Gross\_Percentile']

Non-object columns: ['US Gross', 'Worldwide Gross', 'Production Budget', 'IMDB Rating', 'IMDB Votes', 'Profit']

#### Statistical Summary for Numeric Data:

|       | US Gross     | Worldwide Gross | Production Budget | IMDB Rating \ |
|-------|--------------|-----------------|-------------------|---------------|
| count | 2.228000e+03 | 2.228000e+03    | 2.228000e+03      | 2228.00       |
| mean  | 5.076370e+07 | 1.019370e+08    | 3.816055e+07      | 6.24          |
| std   | 6.643081e+07 | 1.648589e+08    | 3.782604e+07      | 1.24          |
| min   | 0.000000e+00 | 8.840000e+02    | 2.180000e+02      | 1.40          |
| 25%   | 9.646188e+06 | 1.320737e+07    | 1.200000e+07      | 5.50          |
| 50%   | 2.838649e+07 | 4.266892e+07    | 2.600000e+07      | 6.40          |
| 75%   | 6.453140e+07 | 1.200000e+08    | 5.300000e+07      | 7.10          |
| max   | 7.601676e+08 | 2.767891e+09    | 3.000000e+08      | 9.20          |

|       | IMDB Votes | Rating_Rank | Gross_Rank | Rating_Percentile \ |
|-------|------------|-------------|------------|---------------------|
| count | 2228.00    | 2228.00     | 2228.00    | 2228.00             |
| mean  | 33585.15   | 1087.83     | 1114.50    | 50.02               |
| std   | 47325.65   | 646.48      | 643.31     | 28.86               |
| min   | 18.00      | 1.00        | 1.00       | 0.04                |
| 25%   | 6659.25    | 535.00      | 557.00     | 24.71               |
| 50%   | 18169.00   | 1058.00     | 1114.50    | 50.76               |
| 75%   | 40092.75   | 1653.00     | 1671.25    | 74.37               |
| max   | 519541.00  | 2228.00     | 2228.00    | 100.00              |

|       | Gross_Percentile | Rating_per_Gross | ROI_Ratio | Profit        |
|-------|------------------|------------------|-----------|---------------|
| count | 2228.00          | 2228.00          | 2228.00   | 2.228000e+03  |
| mean  | 50.02            | 6.35             | 12.59     | 6.377647e+07  |
| std   | 28.87            | 95.26            | 296.50    | 1.422922e+08  |
| min   | 0.04             | 0.00             | 0.00      | -9.463523e+07 |
| 25%   | 25.03            | 0.05             | 0.79      | -3.290465e+06 |
| 50%   | 50.02            | 0.14             | 1.79      | 1.450338e+07  |
| 75%   | 75.02            | 0.45             | 3.62      | 7.412914e+07  |
| max   | 100.00           | 3959.28          | 12918.03  | 2.530891e+09  |

#### Unique Values in Object Columns:

```

Title           : 2220 unique values
Release Date    : 1064 unique values

```

```

MPAA Rating      :    4 unique values
  Values: ['PG/PG-13', 'R', 'Other', 'G']
Source           :    2 unique values
  Values: ['Adapted screenplay', 'Original Screenplay']
Major Genre      :    6 unique values
Creative Type    :    2 unique values
  Values: ['Fiction', 'Non-Fiction']
Data_Source      :    1 unique values
  Values: ['IMDB']

```

## 7.6.5 Data Type Conversion

Often, after the initial reading, we need to convert the datatypes of some of the columns to make them suitable for analysis, as the available functions and operations depend on the column's data type. For example, the datatype of Release Date in the DataFrame `movie_ratings` is object. To perform datetime related computations on this variable, we'll need to convert it to a datetime format. We'll use the Pandas function `to_datetime()` to convert it to a datetime format. Similar functions such as `to_numeric()`, `to_string()` etc., can be used for other conversions.

In Pandas, the `errors='coerce'` parameter is often used in the context of data conversion, specifically when using the `pd.to_numeric` function. This argument tells Pandas to convert values that it can and set the ones it cannot convert to `NaN`. It's a way of gracefully handling errors without raising an exception. Read the textbook for an example

### 7.6.5.1 Core Conversion Functions

| Function                      | Purpose              | Error Handling               | Example                              |
|-------------------------------|----------------------|------------------------------|--------------------------------------|
| <code>pd.to_numeric()</code>  | String → Number      | <code>errors='coerce'</code> | '123' → 123                          |
| <code>pd.to_datetime()</code> | String →<br>DateTime | <code>errors='coerce'</code> | '2024-01-01' →<br>datetime           |
| <code>astype()</code>         | Force type change    | Raises errors                | <code>df['col'].astype('int')</code> |
| <code>pd.Categorical()</code> | Create categories    | N/A                          | Efficient<br>categorical data        |

**Pro Tip:** `errors='coerce'` converts invalid values to `NaN` instead of throwing errors!

```

# Comprehensive data type analysis before conversion
print(" DATA TYPE CONVERSION EXAMPLE")
print("="*40)

```

```

# Check current data type and sample values
print(f" Release Date column analysis:")
print(f"Current data type: {movie_ratings['Release Date'].dtypes}")
print(f"Sample values:")
print(movie_ratings['Release Date'].head())

# Check for problematic values
print(f"\n Data Quality Check:")
print(f"Non-null values: {movie_ratings['Release Date'].count()} / {len(movie_ratings)}")
print(f"Unique values: {movie_ratings['Release Date'].nunique()}")
print(f>Data range: {movie_ratings['Release Date'].min()} to {movie_ratings['Release Date'].max()}")

# Check for potential conversion issues
print(f"\n Potential Issues:")
non_numeric = movie_ratings['Release Date'].apply(lambda x: not str(x).isdigit() if pd.notnull(x))
if non_numeric.any():
    print(f"Non-numeric values found: {non_numeric.sum()}")
    print(f"Examples: {movie_ratings.loc[non_numeric, 'Release Date'].head().tolist()}")
else:
    print(" All values appear to be numeric years")

```

#### DATA TYPE CONVERSION EXAMPLE

=====

```

Release Date column analysis:
Current data type: object
Sample values:
0    Nov 22 2006
1    Apr 07 1965
2    Apr 24 2009
3    Jul 25 2003
4    Feb 09 2007
Name: Release Date, dtype: object

```

```

Data Quality Check:
Non-null values: 2228 / 2228
Unique values: 1064
Data range: Apr 01 1988 to Sep 30 2006

```

```

Potential Issues:
Non-numeric values found: 2228
Examples: ['Nov 22 2006', 'Apr 07 1965', 'Apr 24 2009', 'Jul 25 2003', 'Feb 09 2007']

```

Next, we'll convert the `Release Date` column in the `DataFrame` to the `datetime` format to facilitate further analysis.

### 7.6.5.2 Comprehensive Type Conversion Examples

```
# COMPREHENSIVE TYPE CONVERSION EXAMPLES
print(" TYPE CONVERSION DEMONSTRATIONS")
print("="*45)

# 1. DateTime Conversion (Year → Full Date)
print(" 1. DateTime Conversion:")
print(f"Before: {movie_ratings['Release Date'].dtype}")

# Convert years to datetime (assuming January 1st)
movie_ratings['Release Date'] = pd.to_datetime(movie_ratings['Release Date'], format='mixed')
print(f"After: {movie_ratings['Release Date'].dtype}")
print("Sample converted values:")
print(movie_ratings['Release Date'].head())

print("\n" + "="*45)

# 2. Numeric Conversion Example
print(" 2. Numeric Conversion with Error Handling:")

# Create sample data with mixed types for demonstration
sample_data = pd.Series(['100', '200.5', 'invalid', '300', None])
print(f"Original data: {sample_data.tolist()}")

# Safe conversion with error handling
numeric_converted = pd.to_numeric(sample_data, errors='coerce')
print(f"After to_numeric(): {numeric_converted.tolist()}")
print(f>Data type: {numeric_converted.dtype}")

print("\n" + "="*45)

# 3. Custom astype() Conversions
print(" 3. Custom Type Conversions with astype():")

# Create sample numeric data
sample_numeric = pd.Series([1.7, 2.3, 3.9, 4.1, 5.8])
```

```

print(f"Original (float): {sample_numeric.tolist()}")

# Convert float to int (truncates decimals)
int_converted = sample_numeric.astype('int')
print(f"As integer: {int_converted.tolist()}")

# Convert to string
str_converted = sample_numeric.astype('string')
print(f"As string: {str_converted.tolist()}")
print(f"String dtype: {str_converted.dtype}")

print("\n Final DataFrame dtypes after conversions:")
print(movie_ratings.dtypes)

```

#### TYPE CONVERSION DEMONSTRATIONS

=====

##### 1. DateTime Conversion:

Before: datetime64[ns]

After: datetime64[ns]

Sample converted values:

0 2006-11-22

1 1965-04-07

2 2009-04-24

3 2003-07-25

4 2007-02-09

Name: Release Date, dtype: datetime64[ns]

=====

##### 2. Numeric Conversion with Error Handling:

Original data: ['100', '200.5', 'invalid', '300', None]

After to\_numeric(): [100.0, 200.5, nan, 300.0, nan]

Data type: float64

=====

##### 3. Custom Type Conversions with astype():

Original (float): [1.7, 2.3, 3.9, 4.1, 5.8]

As integer: [1, 2, 3, 4, 5]

As string: ['1.7', '2.3', '3.9', '4.1', '5.8']

String dtype: string

Final DataFrame dtypes after conversions:

Title object



```

US Gross                int64
Worldwide Gross         int64
Production Budget       int64
Release Date            datetime64[ns]
MPAA Rating             object
Source                  object
Major Genre             object
Creative Type           object
IMDB Rating             float64
IMDB Votes              int64
Rating_Rank             float64
Gross_Rank              float64
Rating_Percentile       float64
Gross_Percentile        float64
Rating_per_Gross        float64
ROI_Ratio               float64
Data_Source             object
Profit                  int64
dtype: object

```

## 7.6.6 Working with DateTime Data: The .dt Accessor

**Power Tool:** The `.dt` accessor unlocks **30+ datetime methods** for temporal analysis and feature engineering.

### 7.6.6.1 Complete DateTime Component Extraction

| Component            | Method                      | Output                 | Business Use Case                    |
|----------------------|-----------------------------|------------------------|--------------------------------------|
| <b>Date Parts</b>    | <code>.dt.year</code>       | 2024                   | Annual trends, cohort analysis       |
|                      | <code>.dt.month</code>      | 1-12                   | Seasonal patterns, monthly reporting |
|                      | <code>.dt.day</code>        | 1-31                   | Daily analysis, month-end effects    |
|                      | <code>.dt.quarter</code>    | 1-4                    | Quarterly business cycles            |
| <b>Time Parts</b>    | <code>.dt.hour</code>       | 0-23                   | Hourly patterns, business hours      |
|                      | <code>.dt.minute</code>     | 0-59                   | Minute-level precision               |
|                      | <code>.dt.second</code>     | 0-59                   | Second-level timing                  |
|                      | <code>.dt.weekday</code>    | 0=Monday, 6=Sunday     | Weekday/weekend analysis             |
| <b>Calendar Info</b> | <code>.dt.day_name()</code> | 'Monday', 'Tuesday'... | Human-readable days                  |

| Component       | Method                         | Output                     | Business Use Case     |
|-----------------|--------------------------------|----------------------------|-----------------------|
| <b>Advanced</b> | <code>.dt.month_name()</code>  | January',<br>'February'... | Human-readable months |
|                 | <code>.dt.dayofyear</code>     | 1-366                      | Day within year       |
|                 | <code>.dt.week</code>          | 1-53                       | Week number           |
|                 | <code>.dt.is_leap_year</code>  | True/False                 | Leap year detection   |
|                 | <code>.dt.days_in_month</code> | 28-31                      | Month length          |

### 7.6.6.2 Practical DateTime Feature Engineering

```
# COMPREHENSIVE DATETIME FEATURE ENGINEERING
print(" DATETIME FEATURE EXTRACTION")
print("="*45)

# 1. Extract multiple date components
movie_ratings['Release_Year'] = movie_ratings['Release Date'].dt.year
movie_ratings['Release_Month'] = movie_ratings['Release Date'].dt.month
movie_ratings['Release_Quarter'] = movie_ratings['Release Date'].dt.quarter
movie_ratings['Release_Weekday'] = movie_ratings['Release Date'].dt.weekday
movie_ratings['Release_DayName'] = movie_ratings['Release Date'].dt.day_name()
movie_ratings['Release_MonthName'] = movie_ratings['Release Date'].dt.month_name()

# Display extracted features
print(" Extracted DateTime Features:")
datetime_features = ['Release Date', 'Release_Year', 'Release_Month', 'Release_Quarter', 'Re
print(movie_ratings[datetime_features].head())

print("\n" + "="*45)

# 2. Create business-relevant features
print(" Business-Relevant DateTime Features:")

# Decade classification
movie_ratings['Decade'] = (movie_ratings['Release_Year'] // 10) * 10
print(" Movies by Decade:")
print(movie_ratings['Decade'].value_counts().sort_index())

# Era classification
movie_ratings['Era'] = movie_ratings['Release_Year'].apply(
    lambda x: 'Silent Era' if x < 1930 else
```

```

        'Golden Age' if x < 1960 else
        'New Hollywood' if x < 1980 else
        'Modern Era' if x < 2000 else
        'Digital Era'
    )

print("\n Movies by Era:")
print(movie_ratings['Era'].value_counts())

# Season classification
movie_ratings['Release_Season'] = movie_ratings['Release_Month'].apply(
    lambda x: 'Winter' if x in [12, 1, 2] else
              'Spring' if x in [3, 4, 5] else
              'Summer' if x in [6, 7, 8] else
              'Fall'
)

print("\n Movies by Release Season:")
print(movie_ratings['Release_Season'].value_counts())

print("\n Sample with All Features:")
feature_cols = ['Title', 'Release_Year', 'Decade', 'Era', 'Release_Season', 'Release_DayName']
print(movie_ratings[feature_cols].head())

```

#### DATETIME FEATURE EXTRACTION

=====

Extracted DateTime Features:

|   | Release Date | Release_Year | Release_Month | Release_Quarter | Release_DayName |
|---|--------------|--------------|---------------|-----------------|-----------------|
| 0 | 2006-11-22   | 2006         | 11            | 4               | Wednesday       |
| 1 | 1965-04-07   | 1965         | 4             | 2               | Wednesday       |
| 2 | 2009-04-24   | 2009         | 4             | 2               | Friday          |
| 3 | 2003-07-25   | 2003         | 7             | 3               | Friday          |
| 4 | 2007-02-09   | 2007         | 2             | 1               | Friday          |

=====

Business-Relevant DateTime Features:

Movies by Decade:

| Decade |    |
|--------|----|
| 1950   | 1  |
| 1960   | 6  |
| 1970   | 8  |
| 1980   | 26 |

```

1990      595
2000     1536
2010       54
2030        2
Name: count, dtype: int64

```

```

Movies by Era:
Era
Digital Era      1592
Modern Era       621
New Hollywood    14
Golden Age        1
Name: count, dtype: int64

```

```

Movies by Release Season:
Release_Season
Fall      631
Summer    571
Winter    521
Spring    505
Name: count, dtype: int64

```

```

Sample with All Features:
      Title  Release_Year  Decade      Era Release_Season \
0   Opal Dreams      2006   2000  Digital Era         Fall
1   Major Dundee     1965   1960  New Hollywood      Spring
2   The Informers     2009   2000  Digital Era      Spring
3  Buffalo Soldiers     2003   2000  Digital Era      Summer
4  The Last Sin Eater     2007   2000  Digital Era      Winter

```

```

Release_DayName
0   Wednesday
1   Wednesday
2   Friday
3   Friday
4   Friday

```

### 7.6.6.3 Time Duration Calculations

**Business Value:** Calculate elapsed time, age of records, time-to-event analysis

```
# let's calculate the days since release till Jan 1st 2024
movie_ratings['Days Since Release'] = (pd.Timestamp.today().normalize() - movie_ratings['Release Date']).dt.days
movie_ratings.head()
```

|   | Title              | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source   | Major |
|---|--------------------|----------|-----------------|-------------------|--------------|-------------|----------|-------|
| 0 | Opal Dreams        |          | 14443           | 14443             | 9000000      | 2006-11-22  | PG/PG-13 |       |
| 1 | Major Dundee       |          | 14873           | 14873             | 3800000      | 1965-04-07  | PG/PG-13 |       |
| 2 | The Informers      |          | 315000          | 315000            | 18000000     | 2009-04-24  | R        |       |
| 3 | Buffalo Soldiers   |          | 353743          | 353743            | 15000000     | 2003-07-25  | R        |       |
| 4 | The Last Sin Eater |          | 388390          | 388390            | 2200000      | 2007-02-09  | PG/PG-13 |       |

**Filtering Data:** Use extracted datetime components to filter rows. Aggregation and grouping using datetime components will be covered in future chapters.

```
# Filter rows where the release month is January
january_releases = movie_ratings[movie_ratings['Release Date'].dt.month == 1]
january_releases.head()
```

|    | Title             | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source   | Major |
|----|-------------------|----------|-----------------|-------------------|--------------|-------------|----------|-------|
| 15 | Thr3e             |          | 1008849         | 1060418           | 2400000      | 2007-01-05  | PG/PG-13 |       |
| 57 | Impostor          |          | 6114237         | 6114237           | 40000000     | 2002-01-04  | PG/PG-13 |       |
| 62 | The Last Station  |          | 6616974         | 6616974           | 18000000     | 2010-01-15  | R        |       |
| 63 | The Big Bounce    |          | 6471394         | 6626115           | 50000000     | 2004-01-30  | PG/PG-13 |       |
| 84 | Not Easily Broken |          | 10572742        | 10572742          | 5000000      | 2009-01-09  | PG/PG-13 |       |

## 7.6.7 Working with object Data

In pandas, the **object** data type is a flexible data type that can store a mix of text (strings), mixed types, or arbitrary Python objects. It is commonly used for string data and is a key part of working with categorical or unstructured data in pandas.

Similar to datetime objects having a **dt** accessor, a number of specialized string methods are available when using the **str** accessor. These methods have in general matching names with the equivalent built-in string methods for single elements, but are applied element-wise on each of the values of the columns.

### 7.6.7.1 the str accessor in pandas

The `str` accessor in pandas provides a wide range of string methods that allow for efficient and convenient text processing on an entire Series of strings. Here are some commonly used `str` methods:

- String splitting: `str.split()`
- String joining: `str.join()`
- Substrings: `str.slice(start, stop)`, `str[0]`
- String Case Conversion: `str.lower()`, `str.upper()`, `str.capitalize()`
- Whitespace Removal: `str.strip()`, `str.lstrip()`, `str.rstrip()`
- Replacing and Removing: `str.replace('old', 'new')`
- Pattern matching and extraction: `str.contains('pattern')`, `startswith('prefix')`, `endswith('suffix')`
- String length and counting: `str.len()`, `str.count()`

Let's use the well-known titanic dataset to illustrate how to manipulate string columns in pandas dataframe next

```
titanic = pd.read_csv('./Datasets/titanic.csv')
titanic.head()
```

|   | PassengerId | Survived | Pclass | Name                                              | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |   |
|---|-------------|----------|--------|---------------------------------------------------|-----|-------|-------|--------|------|-------|----------|---|
| 0 | 1           | 0        | 3      | Braund, Mr. Owen Harris                           |     |       |       |        | 22.0 | 1     | 0        | 0 |
| 1 | 2           | 1        | 1      | Cumings, Mrs. John Bradley (Florence Briggs Th... |     |       |       |        | 38.0 | 1     | 0        | 0 |
| 2 | 3           | 1        | 3      | Heikkinen, Miss. Laina                            |     |       |       |        | 26.0 | 0     | 0        | 0 |
| 3 | 4           | 1        | 1      | Futrelle, Mrs. Jacques Heath (Lily May Peel)      |     |       |       |        | 35.0 | 1     | 0        | 0 |
| 4 | 5           | 0        | 3      | Allen, Mr. William Henry                          |     |       |       |        | 35.0 | 0     | 0        | 0 |

```
titanic.Name
```

```
0          Braund, Mr. Owen Harris
1  Cumings, Mrs. John Bradley (Florence Briggs Th...
2          Heikkinen, Miss. Laina
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)
4          Allen, Mr. William Henry
...
886          Montvila, Rev. Juozas
887          Graham, Miss. Margaret Edith
888  Johnston, Miss. Catherine Helen "Carrie"
889          Behr, Mr. Karl Howell
```

```
890                                Dooley, Mr. Patrick
Name: Name, Length: 891, dtype: object
```

The `Name` column varies in length and contains passengers' last names, titles, and first names. By extracting the title from the `Name` column, we could infer the **sex** of the passenger and add it as a new feature. This could potentially be a significant predictor of survival, as the “ladies first” principle was often applied during the Titanic evacuation. Adding this feature may enhance the model's ability to predict whether a passenger survived.

```
# Let's check the length of the name of each passenger
titanic["Name"].str.len()
```

```
0      23
1      51
2      22
3      44
4      24
..
886    21
887    28
888    40
889    21
890    19
Name: Name, Length: 891, dtype: int64
```

```
# check what is the maximum length of the name
titanic.loc[titanic["Name"].str.len().idxmax(), "Name"]
```

```
'Penasco y Castellana, Mrs. Victor de Satode (Maria Josefa Perez de Soto y Vallejo)'
```

```
# get the observations that contains the word 'Mrs'
titanic[titanic.Name.str.contains('Mrs.')]

```

|    | PassengerId | Survived | Pclass | Name                                              | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |   |
|----|-------------|----------|--------|---------------------------------------------------|-----|-------|-------|--------|------|-------|----------|---|
| 1  | 2           | 1        | 1      | Cumings, Mrs. John Bradley (Florence Briggs Th... |     |       |       |        | 38.0 | 1     | 0        | 0 |
| 3  | 4           | 1        | 1      | Futrelle, Mrs. Jacques Heath (Lily May Peel)      |     |       |       |        | 35.0 | 1     | 0        | 0 |
| 8  | 9           | 1        | 3      | Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg) |     |       |       |        | 27.0 | 0     | 2        | 0 |
| 9  | 10          | 1        | 2      | Nasser, Mrs. Nicholas (Adele Achem)               |     |       |       |        | 14.0 | 1     | 0        | 0 |
| 15 | 16          | 1        | 2      | Hewlett, Mrs. (Mary D Kingcome)                   |     |       |       |        | 55.0 | 0     | 0        | 0 |

| PassengerId | Survived | Pclass | Name | Age                                              | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|--------------------------------------------------|-------|-------|--------|------|-------|----------|
| ...         | ...      | ...    | ...  | ...                                              | ...   | ...   | ...    | ...  | ...   | ...      |
| 871         | 872      | 1      | 1    | Beckwith, Mrs. Richard Leonard (Sallie Monypeny) |       |       |        | 47.0 | 1     | ...      |
| 874         | 875      | 1      | 2    | Abelson, Mrs. Samuel (Hannah Wozosky)            |       |       |        | 28.0 | 1     | ...      |
| 879         | 880      | 1      | 1    | Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)    |       |       |        | 56.0 | 0     | ...      |
| 880         | 881      | 1      | 2    | Shelley, Mrs. William (Imanita Parrish Hall)     |       |       |        | 25.0 | 0     | ...      |
| 885         | 886      | 0      | 3    | Rice, Mrs. William (Margaret Norton)             |       |       |        | 39.0 | 0     | ...      |

```
# get the observations that contains the word 'Mrs'
titanic.Name.str.count('Mrs.').sum()
```

129

```
# split the name column into two columns
titanic.Name.str.split(',', expand=True)
```

|     |           |                                            | <u>0</u> | <u>1</u> |
|-----|-----------|--------------------------------------------|----------|----------|
| 0   | Braund    | Mr. Owen Harris                            |          |          |
| 1   | Cumings   | Mrs. John Bradley (Florence Briggs Thayer) |          |          |
| 2   | Heikkinen | Miss. Laina                                |          |          |
| 3   | Futrelle  | Mrs. Jacques Heath (Lily May Peel)         |          |          |
| 4   | Allen     | Mr. William Henry                          |          |          |
| ... | ...       | ...                                        |          |          |
| 886 | Montvila  | Rev. Juozas                                |          |          |
| 887 | Graham    | Miss. Margaret Edith                       |          |          |
| 888 | Johnston  | Miss. Catherine Helen "Carrie"             |          |          |
| 889 | Behr      | Mr. Karl Howell                            |          |          |
| 890 | Dooley    | Mr. Patrick                                |          |          |

```
# get the last part of the split
titanic.Name.str.split(',', expand=True).get(1)
```

```
0      Mr. Owen Harris
1  Mrs. John Bradley (Florence Briggs Thayer)
2      Miss. Laina
3  Mrs. Jacques Heath (Lily May Peel)
4      Mr. William Henry
...
```



```

886                                Rev. Juozas
887                                Miss. Margaret Edith
888                                Miss. Catherine Helen "Carrie"
889                                Mr. Karl Howell
890                                Mr. Patrick
Name: 1, Length: 891, dtype: object

```

## 7.7 Working with Numerical Data

In data science, **80%** of insights come from numerical analysis. So numerical data analysis matters!

### 7.7.1 Basic Descriptive Statistics

#### 7.7.1.1 The `describe()` Method - Your Data's Report Card

```
movie_ratings.describe()
```

|       | US Gross     | Worldwide Gross | Production Budget | Release Date                  | IMDB Rating | IMDB Votes | Rating      |
|-------|--------------|-----------------|-------------------|-------------------------------|-------------|------------|-------------|
| count | 2.228000e+03 | 2.228000e+03    | 2.228000e+03      | 2228                          |             |            | 2228.000000 |
| mean  | 5.076370e+07 | 1.019370e+08    | 3.816055e+07      | 2002-07-22 01:12:23.267504512 |             |            | 6.239004    |
| min   | 0.000000e+00 | 8.840000e+02    | 2.180000e+02      | 1953-02-05 00:00:00           |             |            | 1.400000    |
| 25%   | 9.646188e+06 | 1.320737e+07    | 1.200000e+07      | 1999-07-29 12:00:00           |             |            | 5.500000    |
| 50%   | 2.838649e+07 | 4.266892e+07    | 2.600000e+07      | 2002-12-26 00:00:00           |             |            | 6.400000    |
| 75%   | 6.453140e+07 | 1.200000e+08    | 5.300000e+07      | 2006-06-06 18:00:00           |             |            | 7.100000    |
| max   | 7.601676e+08 | 2.767891e+09    | 3.000000e+08      | 2039-12-15 00:00:00           |             |            | 9.200000    |
| std   | 6.643081e+07 | 1.648589e+08    | 3.782604e+07      | NaN                           |             |            | 1.243285    |

```

# Comprehensive analysis of numerical data
print(" NUMERICAL DATA ANALYSIS OVERVIEW")
print("="*50)

# Display basic info about our dataset
print("\n Dataset Overview:")
print(f"Shape: {movie_ratings.shape}")
print(f"Numerical columns: {movie_ratings.select_dtypes(include=['number']).columns.tolist()}")

# Enhanced describe() with interpretation

```

```

print("\n COMPREHENSIVE STATISTICAL SUMMARY:")
print("="*40)
desc_stats = movie_ratings.describe()
print(desc_stats)

# Interpret the results
print("\n Key Insights from describe():")
print("-"*30)

for col in movie_ratings.select_dtypes(include=['number']).columns:
    if col in desc_stats.columns:
        mean_val = desc_stats.loc['mean', col]
        median_val = desc_stats.loc['50%', col]
        std_val = desc_stats.loc['std', col]

        print(f"\n {col}:")
        print(f"    • Average: {mean_val:.2f}")
        print(f"    • Median: {median_val:.2f}")
        print(f"    • Spread (Std): {std_val:.2f}")

        # Interpret skewness (very rough heuristic using mean vs. median)
        if pd.notna(mean_val) and pd.notna(median_val) and pd.notna(std_val) and std_val > 0:
            skew_direction = "right" if mean_val > median_val else "left"
            if abs(mean_val - median_val) > 0.1 * std_val:
                print(f"    • Distribution: Skewed {skew_direction}")
            else:
                print(f"    • Distribution: Roughly symmetric")
        else:
            print(f"    • Distribution: Not assessed (insufficient variation or NaNs)")

```

## NUMERICAL DATA ANALYSIS OVERVIEW

=====

### Dataset Overview:

Shape: (2228, 29)

Numerical columns: ['US Gross', 'Worldwide Gross', 'Production Budget', 'IMDB Rating', 'IMDB

### COMPREHENSIVE STATISTICAL SUMMARY:

=====

|       | US Gross     | Worldwide Gross | Production Budget \ |
|-------|--------------|-----------------|---------------------|
| count | 2.228000e+03 | 2.228000e+03    | 2.228000e+03        |
| mean  | 5.076370e+07 | 1.019370e+08    | 3.816055e+07        |

|     |              |              |              |
|-----|--------------|--------------|--------------|
| min | 0.000000e+00 | 8.840000e+02 | 2.180000e+02 |
| 25% | 9.646188e+06 | 1.320737e+07 | 1.200000e+07 |
| 50% | 2.838649e+07 | 4.266892e+07 | 2.600000e+07 |
| 75% | 6.453140e+07 | 1.200000e+08 | 5.300000e+07 |
| max | 7.601676e+08 | 2.767891e+09 | 3.000000e+08 |
| std | 6.643081e+07 | 1.648589e+08 | 3.782604e+07 |

|       | Release Date                  | IMDB Rating | IMDB Votes    | Rating_Rank \ |
|-------|-------------------------------|-------------|---------------|---------------|
| count | 2228                          | 2228.000000 | 2228.000000   | 2228.000000   |
| mean  | 2002-07-22 01:12:23.267504512 | 6.239004    | 33585.154847  | 1087.825853   |
| min   | 1953-02-05 00:00:00           | 1.400000    | 18.000000     | 1.000000      |
| 25%   | 1999-07-29 12:00:00           | 5.500000    | 6659.250000   | 535.000000    |
| 50%   | 2002-12-26 00:00:00           | 6.400000    | 18169.000000  | 1058.000000   |
| 75%   | 2006-06-06 18:00:00           | 7.100000    | 40092.750000  | 1653.000000   |
| max   | 2039-12-15 00:00:00           | 9.200000    | 519541.000000 | 2228.000000   |
| std   | NaN                           | 1.243285    | 47325.651561  | 646.475234    |

|       | Gross_Rank  | Rating_Percentile | Gross_Percentile | Rating_per_Gross \ |
|-------|-------------|-------------------|------------------|--------------------|
| count | 2228.000000 | 2228.000000       | 2228.000000      | 2228.000000        |
| mean  | 1114.499551 | 50.022442         | 50.022442        | 6.354252           |
| min   | 1.000000    | 0.044883          | 0.044883         | 0.002999           |
| 25%   | 557.000000  | 24.708259         | 25.033662        | 0.051100           |
| 50%   | 1114.500000 | 50.763016         | 50.022442        | 0.141194           |
| 75%   | 1671.250000 | 74.371634         | 75.022442        | 0.451079           |
| max   | 2228.000000 | 100.000000        | 100.000000       | 3959.276018        |
| std   | 643.312910  | 28.863900         | 28.873991        | 95.256652          |

|       | ROI_Ratio    | Profit        | Release_Year | Release_Month \ |
|-------|--------------|---------------|--------------|-----------------|
| count | 2228.000000  | 2.228000e+03  | 2228.000000  | 2228.000000     |
| mean  | 12.594830    | 6.377647e+07  | 2002.005386  | 7.101436        |
| min   | 0.000884     | -9.463523e+07 | 1953.000000  | 1.000000        |
| 25%   | 0.787529     | -3.290465e+06 | 1999.000000  | 4.000000        |
| 50%   | 1.787569     | 1.450338e+07  | 2002.000000  | 7.000000        |
| 75%   | 3.623547     | 7.412914e+07  | 2006.000000  | 10.000000       |
| max   | 12918.030200 | 2.530891e+09  | 2039.000000  | 12.000000       |
| std   | 296.498330   | 1.422922e+08  | 5.524324     | 3.399271        |

|       | Release_Quarter | Release_Weekday | Decade      | Days Since Release |
|-------|-----------------|-----------------|-------------|--------------------|
| count | 2228.000000     | 2228.000000     | 2228.000000 | 2228.000000        |
| mean  | 2.680880        | 3.713645        | 1997.127469 | 8473.949731        |
| min   | 1.000000        | 0.000000        | 1950.000000 | -5186.000000       |
| 25%   | 2.000000        | 4.000000        | 1990.000000 | 7058.250000        |
| 50%   | 3.000000        | 4.000000        | 2000.000000 | 8317.000000        |

|     |          |          |             |              |
|-----|----------|----------|-------------|--------------|
| 75% | 4.000000 | 4.000000 | 2000.000000 | 9562.500000  |
| max | 4.000000 | 6.000000 | 2030.000000 | 26538.000000 |
| std | 1.111257 | 0.757334 | 5.918766    | 2013.967471  |

Key Insights from describe():

-----

US Gross:

- Average: 50763702.60
- Median: 28386493.50
- Spread (Std): 66430812.73
- Distribution: Skewed right

Worldwide Gross:

- Average: 101937019.30
- Median: 42668922.50
- Spread (Std): 164858850.98
- Distribution: Skewed right

Production Budget:

- Average: 38160549.18
- Median: 26000000.00
- Spread (Std): 37826043.95
- Distribution: Skewed right

IMDB Rating:

- Average: 6.24
- Median: 6.40
- Spread (Std): 1.24
- Distribution: Skewed left

IMDB Votes:

- Average: 33585.15
- Median: 18169.00
- Spread (Std): 47325.65
- Distribution: Skewed right

Rating\_Rank:

- Average: 1087.83
- Median: 1058.00
- Spread (Std): 646.48
- Distribution: Roughly symmetric

Gross\_Rank:

- Average: 1114.50
- Median: 1114.50
- Spread (Std): 643.31
- Distribution: Roughly symmetric

Rating\_Percentile:

- Average: 50.02
- Median: 50.76
- Spread (Std): 28.86
- Distribution: Roughly symmetric

Gross\_Percentile:

- Average: 50.02
- Median: 50.02
- Spread (Std): 28.87
- Distribution: Roughly symmetric

Rating\_per\_Gross:

- Average: 6.35
- Median: 0.14
- Spread (Std): 95.26
- Distribution: Roughly symmetric

ROI\_Ratio:

- Average: 12.59
- Median: 1.79
- Spread (Std): 296.50
- Distribution: Roughly symmetric

Profit:

- Average: 63776470.12
- Median: 14503383.00
- Spread (Std): 142292233.66
- Distribution: Skewed right

Release\_Year:

- Average: 2002.01
- Median: 2002.00
- Spread (Std): 5.52
- Distribution: Roughly symmetric

Release\_Month:

- Average: 7.10
- Median: 7.00
- Spread (Std): 3.40
- Distribution: Roughly symmetric

Release\_Quarter:

- Average: 2.68
- Median: 3.00
- Spread (Std): 1.11
- Distribution: Skewed left

Release\_Weekday:

- Average: 3.71
- Median: 4.00
- Spread (Std): 0.76
- Distribution: Skewed left

Decade:

- Average: 1997.13
- Median: 2000.00
- Spread (Std): 5.92
- Distribution: Skewed left

Days Since Release:

- Average: 8473.95
- Median: 8317.00
- Spread (Std): 2013.97
- Distribution: Roughly symmetric

## 7.7.2 Individual Statistical Methods

Beyond `describe()`, pandas offers **granular statistical methods** for specific analysis needs:

|  | Method                                  | Purpose            | Business Example                     |
|--|-----------------------------------------|--------------------|--------------------------------------|
|  | <code>mean()</code>                     | Average value      | Average customer age, mean sales     |
|  | <code>median()</code>                   | Middle value       | Median income, typical order size    |
|  | <code>mode()</code>                     | Most frequent      | Most common product rating           |
|  | <code>std()</code>                      | Standard deviation | Risk assessment, quality control     |
|  | <code>var()</code>                      | Variance           | Data volatility, consistency         |
|  | <code>min()</code> / <code>max()</code> | Range bounds       | Performance benchmarks               |
|  | <code>quantile()</code>                 | Percentiles        | Customer segments, outlier detection |

|                      | Method          | Purpose | Business Example                 |
|----------------------|-----------------|---------|----------------------------------|
| <code>sum()</code>   | Total           |         | Revenue totals, inventory counts |
| <code>count()</code> | Non-null values |         | Data completeness                |

### 7.7.2.1 Summary Statistics: Rows vs Columns with axis Parameter

**Critical Concept:** The `axis` parameter controls the **direction** of calculation: - **axis=0** (default): Calculate **down** the rows (column-wise aggregation) - **axis=1**: Calculate **across** the columns (row-wise aggregation)

**Memory Trick:** - `axis=0` → “**0** rows away” → Column results - `axis=1` → “**1** column away” → Row results

```
movie_ratings.head()
```

|   | Title              | US Gross | Worldwide Gross | Production Budget | Release Date | MPAA Rating | Source   | Major |
|---|--------------------|----------|-----------------|-------------------|--------------|-------------|----------|-------|
| 0 | Opal Dreams        |          | 14443           | 14443             | 9000000      | 2006-11-22  | PG/PG-13 |       |
| 1 | Major Dundee       |          | 14873           | 14873             | 3800000      | 1965-04-07  | PG/PG-13 |       |
| 2 | The Informers      |          | 315000          | 315000            | 18000000     | 2009-04-24  | R        |       |
| 3 | Buffalo Soldiers   |          | 353743          | 353743            | 15000000     | 2003-07-25  | R        |       |
| 4 | The Last Sin Eater |          | 388390          | 388390            | 2200000      | 2007-02-09  | PG/PG-13 |       |

```
movie_ratings.columns
```

```
Index(['Title', 'US Gross', 'Worldwide Gross', 'Production Budget',
      'Release Date', 'MPAA Rating', 'Source', 'Major Genre', 'Creative Type',
      'IMDB Rating', 'IMDB Votes', 'Rating_Rank', 'Gross_Rank',
      'Rating_Percentile', 'Gross_Percentile', 'Rating_per_Gross',
      'ROI_Ratio', 'Data_Source', 'Profit', 'Release_Year', 'Release_Month',
      'Release_Quarter', 'Release_Weekday', 'Release_DayName',
      'Release_MonthName', 'Decade', 'Era', 'Release_Season',
      'Days Since Release'],
      dtype='object')
```

```
# INDIVIDUAL STATISTICAL METHODS DEMONSTRATION
print(" INDIVIDUAL STATISTICAL METHODS")
print("="*50)
```

```

# Select numerical columns for analysis
numeric_cols = ['IMDB Rating', 'IMDB Votes']
print(f"\nAnalyzing columns: {numeric_cols}")

print("\n COLUMN-WISE STATISTICS (axis=0 - default):")
print("-"*45)
print("Mean values (average per column):")
print(movie_ratings[numeric_cols].mean())

print("\nMedian values (middle value per column):")
print(movie_ratings[numeric_cols].median())

print("\nStandard deviation (spread per column):")
print(movie_ratings[numeric_cols].std().round(2))

print("\n RANGE AND DISTRIBUTION:")
print("-"*30)
print("Minimum values:")
print(movie_ratings[numeric_cols].min())

print("\nMaximum values:")
print(movie_ratings[numeric_cols].max())

print("\nQuantiles (25%, 50%, 75%):")
print(movie_ratings[numeric_cols].quantile([0.25, 0.5, 0.75]))

# Business interpretation example
print("\n BUSINESS INSIGHTS:")
print("-"*20)
avg_rating = movie_ratings['IMDB Rating'].mean()
median_votes = movie_ratings['IMDB Votes'].median()

print(f" Average movie quality: {avg_rating:.1f}/10")
print(f" Typical movie popularity: {median_votes:,.0f} votes")

```

#### INDIVIDUAL STATISTICAL METHODS

=====

Analyzing columns: ['IMDB Rating', 'IMDB Votes']

COLUMN-WISE STATISTICS (axis=0 - default):

-----



Mean values (average per column):

IMDB Rating           6.239004

IMDB Votes           33585.154847

dtype: float64

Median values (middle value per column):

IMDB Rating           6.4

IMDB Votes           18169.0

dtype: float64

Standard deviation (spread per column):

IMDB Rating           1.24

IMDB Votes           47325.65

dtype: float64

#### RANGE AND DISTRIBUTION:

-----  
Minimum values:

IMDB Rating           1.4

IMDB Votes           18.0

dtype: float64

Maximum values:

IMDB Rating           9.2

IMDB Votes           519541.0

dtype: float64

Quantiles (25%, 50%, 75%):

|      | IMDB Rating | IMDB Votes |
|------|-------------|------------|
| 0.25 | 5.5         | 6659.25    |
| 0.50 | 6.4         | 18169.00   |
| 0.75 | 7.1         | 40092.75   |

#### BUSINESS INSIGHTS:

-----  
Average movie quality: 6.2/10

Typical movie popularity: 18,169 votes

```
# AXIS PARAMETER DEMONSTRATION - Critical for Data Science!
print(" UNDERSTANDING THE AXIS PARAMETER")
print("=*50)
```

```

# Create a sample DataFrame for clear demonstration
sample_data = {
    'Q1_Sales': [100, 150, 120, 200],
    'Q2_Sales': [110, 160, 130, 210],
    'Q3_Sales': [105, 155, 125, 205],
    'Q4_Sales': [115, 165, 135, 215]
}
quarterly_sales = pd.DataFrame(sample_data,
                                index=['North', 'South', 'East', 'West'])

print(" Sample Quarterly Sales Data:")
print(quarterly_sales)

print("\n AXIS=0 (Column-wise): Statistics DOWN the rows")
print("-"*50)
print("Mean sales per quarter (across all regions):")
col_means = quarterly_sales.mean(axis=0) # Default behavior
print(col_means)

print("\nSum sales per quarter (total across regions):")
col_sums = quarterly_sales.sum(axis=0)
print(col_sums)

print("\n AXIS=1 (Row-wise): Statistics ACROSS the columns")
print("-"*50)
print("Mean sales per region (across all quarters):")
row_means = quarterly_sales.mean(axis=1)
print(row_means)

print("\nTotal sales per region (across all quarters):")
row_sums = quarterly_sales.sum(axis=1)
print(row_sums)

print("\n BUSINESS INTERPRETATION:")
print("-"*25)
best_quarter = col_sums.idxmax()
best_region = row_sums.idxmax()
print(f" Best performing quarter: {best_quarter} (${col_sums[best_quarter]:,})")
print(f" Best performing region: {best_region} (${row_sums[best_region]:,})")

# Practical example with movie data
print(f"\n MOVIE DATA EXAMPLE:")

```

```

print("-"*20)
print("Average rating and votes per movie (axis=1):")
movie_sample = movie_ratings[['IMDB Rating', 'IMDB Votes']].head(3)
print("Sample movies:")
print(movie_sample)
print("\nAverage across rating dimensions per movie:")
print(movie_sample.mean(axis=1)) # Average across columns for each row

```

#### UNDERSTANDING THE AXIS PARAMETER

=====

Sample Quarterly Sales Data:

|       | Q1_Sales | Q2_Sales | Q3_Sales | Q4_Sales |
|-------|----------|----------|----------|----------|
| North | 100      | 110      | 105      | 115      |
| South | 150      | 160      | 155      | 165      |
| East  | 120      | 130      | 125      | 135      |
| West  | 200      | 210      | 205      | 215      |

AXIS=0 (Column-wise): Statistics DOWN the rows

-----

Mean sales per quarter (across all regions):

```

Q1_Sales    142.5
Q2_Sales    152.5
Q3_Sales    147.5
Q4_Sales    157.5
dtype: float64

```

Sum sales per quarter (total across regions):

```

Q1_Sales    570
Q2_Sales    610
Q3_Sales    590
Q4_Sales    630
dtype: int64

```

AXIS=1 (Row-wise): Statistics ACROSS the columns

-----

Mean sales per region (across all quarters):

```

North    107.5
South    157.5
East     127.5
West     207.5
dtype: float64

```

```
Total sales per region (across all quarters):
North      430
South      630
East       510
West       830
dtype: int64
```

#### BUSINESS INTERPRETATION:

```
-----
Best performing quarter: Q4_Sales ($630)
Best performing region: West ($830)
```

#### MOVIE DATA EXAMPLE:

```
-----
Average rating and votes per movie (axis=1):
Sample movies:
   IMDB Rating  IMDB Votes
0           6.5         468
1           6.7        2588
2           5.2        7595

Average across rating dimensions per movie:
0      237.25
1    1297.35
2    3800.10
dtype: float64
```

### 7.7.3 Summary: Working with Numerical Data

| Concept          | Purpose           | Key Methods                                                                                |
|------------------|-------------------|--------------------------------------------------------------------------------------------|
| Basic Statistics | Data overview     | <code>describe()</code> , <code>mean()</code> , <code>median()</code> , <code>std()</code> |
| Axis Operations  | Direction control | <code>axis=0</code> (columns), <code>axis=1</code> (rows)                                  |

## 7.8 Understanding inplace=True vs inplace=False

### 7.8.1 Why This Matters: Memory, Performance, and Safety

The `inplace` parameter is one of the **most important concepts** in pandas for:

- **Memory management** (avoid unnecessary copies)
- **Data integrity** (prevent accidental modifications)
- **Performance optimization** (reduce overhead)
- **Debugging** (track data transformations)

## 7.8.2 Core Concept: Modify vs Create

| Parameter                  | Behavior                 | Memory       | Original Data              | Return Value       | Use Case                     |
|----------------------------|--------------------------|--------------|----------------------------|--------------------|------------------------------|
| <code>inplace=False</code> | <b>Creates new copy</b>  | Higher usage | <b>Unchanged</b>           | Modified DataFrame | <b>Safe exploration</b>      |
| <code>inplace=True</code>  | <b>Modifies original</b> | Lower usage  | <b>Changed permanently</b> | None               | <b>Final transformations</b> |

**Critical Rule:** `inplace=True` returns None, not the modified DataFrame!

```
# Comprehensive inplace demonstration
print(" INPLACE PARAMETER DEMONSTRATION")
print("="*50)

# Create sample business data
original_data = pd.DataFrame({
    'Employee_ID': [101, 102, 103, 104, 105],
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eva'],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR'],
    'Salary': [75000, 65000, 80000, 70000, 68000],
    'Years_Experience': [5, 3, 7, 4, 6]
})

print(" Original DataFrame:")
print(original_data)
print(f" DataFrame ID: {id(original_data)}") # Memory address

# Method 1: inplace=False (DEFAULT)
print(f"\n Method 1: inplace=False (Creates New Copy)")
print("-" * 45)

# Add bonus column without modifying original
df_with_bonus = original_data.assign(Bonus=original_data['Salary'] * 0.1)
print(" New DataFrame with bonus:")
```

```

print(df_with_bonus[['Name', 'Salary', 'Bonus']].head(3))
print(f" New DataFrame ID: {id(df_with_bonus)}")

print(f"\n Original DataFrame (unchanged):")
print(original_data.head(3))
print(f" Original ID still: {id(original_data)}")

# Method 2: inplace=True (Modifies Original)
print(f"\n Method 2: inplace=True (Modifies Original)")
print("-" * 45)

# Store original ID for comparison
original_id = id(original_data)

# Add bonus column in place
original_data['Bonus'] = original_data['Salary'] * 0.1
print(" Original DataFrame after in-place modification:")
print(original_data[['Name', 'Salary', 'Bonus']].head(3))
print(f" Same DataFrame ID: {id(original_data)}")
print(f" ID unchanged: {original_id == id(original_data)}")

```

```

      B
0  4
1  5
2  6
   A  B
0  1  4
1  2  5
2  3  6
      B
0  4
1  5
2  6

```

### 7.8.3 Common Methods Supporting inplace Parameter

```

# Demonstrate inplace with different pandas methods
print(" METHODS SUPPORTING INPLACE PARAMETER")
print("-"*50)

```

```

# Create sample dataset for demonstrations
sample_df = pd.DataFrame({
    'Product': ['Laptop', 'Phone', 'Tablet', 'Phone', 'Laptop'],
    'Price': [1200, 800, 600, 850, 1100],
    'Category': ['Electronics', 'Electronics', 'Electronics', 'Electronics', 'Electronics'],
    'Stock': [10, 0, 15, 8, 12]
})

print(" Sample Dataset:")
print(sample_df)

# 1. DROP operations
print(f"\n 1. DROP OPERATIONS")
print("-" * 30)

# Drop with inplace=False (default)
df_copy = sample_df.copy()
result = df_copy.drop('Category', axis=1) # inplace=False by default
print(" After drop(inplace=False) - returns new DataFrame:")
print(f"Original shape: {df_copy.shape}, New shape: {result.shape}")

# Drop with inplace=True
df_copy2 = sample_df.copy()
return_value = df_copy2.drop('Category', axis=1, inplace=True)
print(f" After drop(inplace=True) - modifies original:")
print(f"Modified shape: {df_copy2.shape}, Return value: {return_value}")

# 2. RENAME operations
print(f"\n 2. RENAME OPERATIONS")
print("-" * 30)

df_copy3 = sample_df.copy()
# Rename with inplace=False
renamed = df_copy3.rename(columns={'Product': 'Item_Name'})
print(f" rename(inplace=False): Original columns unchanged")
print(f"Original: {list(df_copy3.columns)}")
print(f"New: {list(renamed.columns)}")

# Rename with inplace=True
df_copy3.rename(columns={'Product': 'Item_Name'}, inplace=True)
print(f" rename(inplace=True): Original columns changed")
print(f"Modified: {list(df_copy3.columns)}")

```

```

# 3. SORT operations
print(f"\n 3. SORT OPERATIONS")
print("-" * 30)

df_copy4 = sample_df.copy()
# Sort with inplace=False
sorted_df = df_copy4.sort_values('Price')
print(f" sort_values(inplace=False):")
print(f"Original order preserved: {list(df_copy4['Price'])}")
print(f"New sorted order: {list(sorted_df['Price'])}")

# Sort with inplace=True
df_copy4.sort_values('Price', inplace=True)
print(f" sort_values(inplace=True):")
print(f"Original modified: {list(df_copy4['Price'])}")

# 4. FILLNA operations
print(f"\n 4. FILLNA OPERATIONS")
print("-" * 30)

# Create data with NaN
nan_df = pd.DataFrame({
    'A': [1, None, 3, None, 5],
    'B': [None, 2, None, 4, None]
})

print("Original with NaN:")
print(nan_df)

# fillna with inplace=False
filled = nan_df.fillna(0)
print(f"\n fillna(inplace=False) - original unchanged:")
print("Original still has NaN:", nan_df.isna().sum().sum() > 0)
print("New has no NaN:", filled.isna().sum().sum() == 0)

# fillna with inplace=True
nan_df.fillna(0, inplace=True)
print(f" fillna(inplace=True) - original modified:")
print("Original now has no NaN:", nan_df.isna().sum().sum() == 0)

```

METHODS SUPPORTING INPLACE PARAMETER



=====

Sample Dataset:

|   | Product | Price | Category    | Stock |
|---|---------|-------|-------------|-------|
| 0 | Laptop  | 1200  | Electronics | 10    |
| 1 | Phone   | 800   | Electronics | 0     |
| 2 | Tablet  | 600   | Electronics | 15    |
| 3 | Phone   | 850   | Electronics | 8     |
| 4 | Laptop  | 1100  | Electronics | 12    |

## 1. DROP OPERATIONS

-----

After drop(inplace=False) - returns new DataFrame:

Original shape: (5, 4), New shape: (5, 3)

After drop(inplace=True) - modifies original:

Modified shape: (5, 3), Return value: None

## 2. RENAME OPERATIONS

-----

rename(inplace=False): Original columns unchanged

Original: ['Product', 'Price', 'Category', 'Stock']

New: ['Item\_Name', 'Price', 'Category', 'Stock']

rename(inplace=True): Original columns changed

Modified: ['Item\_Name', 'Price', 'Category', 'Stock']

## 3. SORT OPERATIONS

-----

sort\_values(inplace=False):

Original order preserved: [1200, 800, 600, 850, 1100]

New sorted order: [600, 800, 850, 1100, 1200]

sort\_values(inplace=True):

Original modified: [600, 800, 850, 1100, 1200]

## 4. FILLNA OPERATIONS

-----

Original with NaN:

|   | A   | B   |
|---|-----|-----|
| 0 | 1.0 | NaN |
| 1 | NaN | 2.0 |
| 2 | 3.0 | NaN |
| 3 | NaN | 4.0 |
| 4 | 5.0 | NaN |

fillna(inplace=False) - original unchanged:

```
Original still has NaN: True
New has no NaN: True
fillna(inplace=True) - original modified:
Original now has no NaN: True
```

## Complete Methods Reference

| Method               | Supports inplace | Default | Memory Impact             | Use Case                    |
|----------------------|------------------|---------|---------------------------|-----------------------------|
| <b>drop()</b>        | Yes              | False   | High without<br>inplace   | Remove<br>columns/rows      |
| <b>dropna()</b>      | Yes              | False   | High without<br>inplace   | Remove<br>missing<br>values |
| <b>fillna()</b>      | Yes              | False   | High without<br>inplace   | Fill missing<br>values      |
| <b>rename()</b>      | Yes              | False   | Medium without<br>inplace | Rename<br>columns/index     |
| <b>sort_values()</b> | Yes              | False   | High without<br>inplace   | Sort by<br>values           |
| <b>sort_index()</b>  | Yes              | False   | High without<br>inplace   | Sort by index               |
| <b>reset_index()</b> | Yes              | False   | Medium without<br>inplace | Reset index                 |
| <b>set_index()</b>   | Yes              | False   | Medium without<br>inplace | Set new<br>index            |
| <b>replace()</b>     | Yes              | False   | High without<br>inplace   | Replace<br>values           |

## 7.9 Independent Practice

### 7.9.1 Practice exercise 1

#### 7.9.1.1

Read the file *Top 10 Albums By Year.csv*. This file contains the top 10 albums for each year from 1990 to 2021. Each row corresponds to a unique album.

|   | Year | Ranking | Artist       | Album                     | Worldwide Sales | CDs | Tracks | Album Length | Hours | Minutes | Seconds |
|---|------|---------|--------------|---------------------------|-----------------|-----|--------|--------------|-------|---------|---------|
| 0 | 1990 | 8       | Phil Collins | ... Serious Hits... Live! |                 |     |        |              | 99    | 56      | 20      |

|   | Year | Ranking | Artist           | Album                                      | Worldwide Sales | CDs | Tracks | Album Length | Hours    | Minutes | Seconds |
|---|------|---------|------------------|--------------------------------------------|-----------------|-----|--------|--------------|----------|---------|---------|
| 1 | 1990 | 1       | Madonna          | The Immaculate Collection                  |                 |     |        |              | 30000000 |         | 1       |
| 2 | 1990 | 10      | The Three Tenors | Carreras Domingo Pavarotti In Concert 1990 |                 |     |        |              | 8533000  |         | 1       |
| 3 | 1990 | 4       | MC Hammer        | Please Hammer Don't Hurt Em                |                 |     |        |              | 18000000 |         | 1       |
| 4 | 1990 | 6       | Movie Soundtrack | Aashiqui                                   |                 |     |        |              | 15000000 |         | 1       |

### 7.9.1.2

Print the summary statistics of the data, and answer the following question:

- Why is **Worldwide Sales** not included in the summary statistics table printed in the above step?

|       | Year        | Ranking    | Worldwide Sales | CDs | Tracks     | Hours      | Minutes    | Seconds    | mean_sales_per_year |     |
|-------|-------------|------------|-----------------|-----|------------|------------|------------|------------|---------------------|-----|
| count | 320.000000  | 320.000000 | 3.190000e+02    |     | 320.000000 | 320.000000 | 320.000000 | 320.000000 | 320.000000          | 320 |
| mean  | 2005.500000 | 5.50000    | 1.071093e+07    |     | 1.043750   | 14.306250  | 0.941406   | 56.478500  |                     | 338 |
| std   | 9.247553    | 2.87678    | 7.566796e+06    |     | 0.246528   | 5.868995   | 0.382895   | 22.970109  |                     | 137 |
| min   | 1990.000000 | 1.00000    | 1.909009e+06    |     | 1.000000   | 6.000000   | 0.320000   | 19.430000  |                     | 116 |
| 25%   | 1997.750000 | 3.00000    | 5.000000e+06    |     | 1.000000   | 12.000000  | 0.740000   | 44.137500  |                     | 264 |
| 50%   | 2005.500000 | 5.50000    | 8.255866e+06    |     | 1.000000   | 13.000000  | 0.860000   | 51.555000  |                     | 309 |
| 75%   | 2013.250000 | 8.00000    | 1.400000e+07    |     | 1.000000   | 15.000000  | 1.090000   | 65.112500  |                     | 390 |
| max   | 2021.000000 | 10.00000   | 4.500000e+07    |     | 4.000000   | 67.000000  | 5.070000   | 304.030000 |                     | 182 |

`dtype('O')`

### 7.9.1.3

Update the DataFrame so that **Worldwide Sales** is included in the summary statistics table. Print the summary statistics table.

**Hint:** Sometimes it may not be possible to convert an object to `numeric()`. For example, the object 'hi' cannot be converted to a `numeric()` by the python compiler. To avoid getting an error, use the `errors` argument of `to_numeric()` to force such conversions to NaN (missing value).

|       | Year        | Ranking   | Worldwide Sales | CDs | Tracks     | Hours      | Minutes    | Seconds    |     |
|-------|-------------|-----------|-----------------|-----|------------|------------|------------|------------|-----|
| count | 320.000000  | 320.00000 | 3.190000e+02    |     | 320.000000 | 320.000000 | 320.000000 | 320.000000 | 320 |
| mean  | 2005.500000 | 5.50000   | 1.071093e+07    |     | 1.043750   | 14.306250  | 0.941406   | 56.478500  | 338 |
| std   | 9.247553    | 2.87678   | 7.566796e+06    |     | 0.246528   | 5.868995   | 0.382895   | 22.970109  | 137 |

|     | Year        | Ranking  | Worldwide Sales | CDs      | Tracks    | Hours    | Minutes    | Seconds |  |  |
|-----|-------------|----------|-----------------|----------|-----------|----------|------------|---------|--|--|
| min | 1990.000000 | 1.00000  | 1.909009e+06    | 1.000000 | 6.000000  | 0.320000 | 19.430000  | 116     |  |  |
| 25% | 1997.750000 | 3.00000  | 5.000000e+06    | 1.000000 | 12.000000 | 0.740000 | 44.137500  | 264     |  |  |
| 50% | 2005.500000 | 5.50000  | 8.255866e+06    | 1.000000 | 13.000000 | 0.860000 | 51.555000  | 309     |  |  |
| 75% | 2013.250000 | 8.00000  | 1.400000e+07    | 1.000000 | 15.000000 | 1.090000 | 65.112500  | 390     |  |  |
| max | 2021.000000 | 10.00000 | 4.500000e+07    | 4.000000 | 67.000000 | 5.070000 | 304.030000 | 182     |  |  |

#### 7.9.1.4

Create a new column called `mean_sales_per_year` that computes the average worldwide sales per year for each album, assuming that the worldwide sales are as of 2022. Print the first 5 rows of the updated DataFrame.

|   | Year | Ranking | Artist           | Album                                      | Worldwide Sales | CDs | Tracks | Album Length | Hours      | Minutes | Seconds |
|---|------|---------|------------------|--------------------------------------------|-----------------|-----|--------|--------------|------------|---------|---------|
| 0 | 1990 | 8       | Phil Collins     | Serious Hits... Live!                      |                 |     |        |              | 9956520.0  |         | 1       |
| 1 | 1990 | 1       | Madonna          | The Immaculate Collection                  |                 |     |        |              | 30000000.0 |         | 1       |
| 2 | 1990 | 10      | The Three Tenors | Carreras Domingo Pavarotti In Concert 1990 |                 |     |        |              | 8533000.0  |         | 1       |
| 3 | 1990 | 4       | MC Hammer        | Please Hammer Don't Hurt Em                |                 |     |        |              | 18000000.0 |         | 1       |
| 4 | 1990 | 6       | Movie Soundtrack | Aashiqui                                   |                 |     |        |              | 15000000.0 |         | 1       |

#### 7.9.1.5

Find the album having the highest worldwide sales in the dataset, and its artist.

```
Sales: 9956520
Album: Serious Hits... Live!
Artist: Phil Collins
```

#### 7.9.1.6

Subset the data to include only Hip-Hop albums. How many Hip\_Hop albums are there?

```
Number of Hip-Hop albums: 42
```

```
Hip-Hop albums:
Number of Hip-Hop albums: 42
```

```
Hip-Hop albums:
```